



Instituto Tecnológico de Zacatepec

# TALLER DE BASE DE DATOS

Grupo: XB

## PROYECTO JAVA Y JDBC

INTEGRANTES  
EQUIPO

Autores:

- Cruz Martínez Arleth
- Ureña Nieto Miguel Eduardo
- Ortiz Tafoya Moisés
- Hernández Beltrán Emili Marian
- Garduño Nava Fernando

Asesor: Rodriguez Zabaleta Estela

Fecha: 29/05/2025

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Resumen .....                                                                  | 3  |
| Introducción .....                                                             | 4  |
| Planteamiento del problema.....                                                | 4  |
| Objetivo.....                                                                  | 4  |
| Objetivos específicos.....                                                     | 4  |
| Justificación .....                                                            | 5  |
| Marco Teórico .....                                                            | 5  |
| Java y JDBC.....                                                               | 5  |
| MySQL .....                                                                    | 5  |
| Triggers en MySQL .....                                                        | 5  |
| Metodología de Desarrollo Técnico-Aplicada .....                               | 5  |
| Fase 1: Análisis de requerimientos. ....                                       | 5  |
| Configuración de proyecto con Spring Boot .....                                | 6  |
| ¿Qué hace esta clase?.....                                                     | 8  |
| Requerimientos Técnicos Iniciales.....                                         | 14 |
| Arquitectura y Herramientas .....                                              | 15 |
| Diseño de la Base de Datos .....                                               | 15 |
| Fase 2: Diseño del Backend (API REST Con Spring Boot).....                     | 16 |
| Uso de relaciones JPA(Java Persistence API) en los Modelos implementados. .... | 16 |
| Código ejemplo:.....                                                           | 19 |
| Beneficios de usar relaciones JPA .....                                        | 20 |
| Fase 3: Implementación. ....                                                   | 21 |
| Método 1: getAllClients() .....                                                | 29 |
| Método 2: getClienteById(Long id).....                                         | 29 |
| Método 3: newCliente(Cliente clienteInfo).....                                 | 29 |
| Método 4: updateClienteById(Long id, Cliente clienteInfo).....                 | 30 |
| Método 5: deleteClienteById(Long id) .....                                     | 30 |
| Estructura modular: .....                                                      | 67 |
| CSS.....                                                                       | 68 |
| JS.....                                                                        | 73 |
| HTML .....                                                                     | 74 |
| Visualización Pagina web .....                                                 | 78 |
| Fase 4: Reglas de negocios. ....                                               | 79 |
| Validaciones y Triggers (MySQL).....                                           | 79 |

|                                                            |    |
|------------------------------------------------------------|----|
| Fase 5: .....                                              | 79 |
| Fase 6: .....                                              | 80 |
| Triggers.....                                              | 80 |
| Estructura de los Triggers dentro de la Base de Datos..... | 80 |
| Vista de ventas consolidada. ....                          | 81 |
| Automatización mediante Triggers.....                      | 82 |
| Resultados .....                                           | 87 |
| Formulario de cliente.....                                 | 87 |
| Formulario de productos .....                              | 88 |
| Formulario de categoría.....                               | 89 |
| Formulario de ventas.....                                  | 90 |
| Consultas.....                                             | 91 |
| Consultar clientes.....                                    | 91 |
| Consulta productos.....                                    | 91 |
| Consulta ventas.....                                       | 92 |
| Consulta categorías.....                                   | 92 |
| Consulta producto_cliente. ....                            | 93 |
| Consulta producto_venta.....                               | 93 |
| Consulta detalle_venta. ....                               | 94 |
| Conclusión.....                                            | 95 |

## Índice

### Resumen

Este proyecto describe el proceso de conexión entre un lenguaje de programación (Java) y un sistema gestor de base de datos (MySQL) mediante la API JDBC. Se desarrolla una aplicación en Spring Boot que facilita la creación de aplicaciones con una infraestructura ligera que elimina la mayoría de los sistemas antiguos a los que estamos acostumbrados. El objetivo de esta API es proporcionar un conjunto de herramientas para construir rápidamente aplicaciones de Spring que sean fáciles de configurar donde también permite realizar operaciones CRUD gracias a su eficiencia, código limpio, automatizaciones sobre la base de datos **factu**, simulando un sistema de ventas que gestiona productos, pedidos y clientes.

La base de datos, denominada **factu**, almacena información clave como productos disponibles, clientes registrados y pedidos realizados. También se hace uso de la automatización de asignación de puntos al cliente según el monto de la compra y la reducción de existencias de productos. Este proyecto refuerza los conocimientos adquiridos en

programación orientada a objetos y bases de datos, simula la necesidad real en entornos comerciales pequeños y medianos.

El trabajo incluye el diseño de la base de datos, la implementación del código en Java, el uso de triggers en MYSQL y la integración del conector JDBC. Se presentan evidencias funcionales y se evalúa el impacto de la correcta comunicación entre los sistemas.

## Introducción

El avance de la tecnología ha llegado a la necesidad constante de integrar sistemas de información que automaticen procesos esenciales en los negocios. La gestión de pedidos, control de inventarios y seguimiento de clientes son actividades que, en la actualidad, pueden y deben ser respaldadas por aplicaciones informáticas. Java, con su arquitectura multiplataforma y estructura orientada a objetos, se presenta como una herramienta eficaz para el desarrollo de dichas aplicaciones. En este contexto, el uso de JDBC permite que java interactúe directamente con bases de datos relacionales como MySQL, posibilitando el manejo de grandes volúmenes de información con eficiencia y seguridad.

La práctica se centra en el diseño, desarrollo y evaluación de una solución informática que integre una base de datos con una aplicación Java. A través de esta integración, se busca simular el funcionamiento de un sistema de gestión de pedidos, donde se controle tanto la información del cliente como las existencias del producto, y se implementen reglas de negocio automatizadas y más. Este documento presenta cada fase del desarrollo, desde la planificación hasta la validación de los resultados, aportando evidencias claras del proceso seguido.

## Planteamiento del problema

En muchos entornos comerciales, especialmente en pequeños negocios, se carece de sistemas automatizados para gestionar adecuadamente los pedidos de los clientes, el inventario de productos y programas de fidelidad. Esta carencia provoca errores humanos, pérdida de datos, falta de seguimiento.

El problema específico que se aborda en este proyecto, es: ¿Cómo diseñar e implementar un sitio web en Java que permita conectarse a una base de datos MySQL para gestionar pedidos de clientes, actualizar existencias de productos y asignar puntos de fidelidad según el monto de compra? Esto nos ayuda a guiar el desarrollo del sistema.

## Objetivo

Desarrollar una página web funcional que se conecte con una base de datos MySQL utilizando JDBC, permitiendo la gestión automatizada de pedidos, productos y clientes, e implementando triggers para operaciones lógicas como el control de inventario y control de eliminación.

## Objetivos específicos

- Diseñar el modelo relacional de la base de datos factu.
- Implementar el esquema de base de datos en MySQL y añadir registro de prueba.

- Programas en java las funcionalidades necesarias para insertar, modificar, eliminar y consultar datos.
- Establecer la conexión entre Java y MySQL mediante el driver JDBC.
- Incorporar triggers en MySQL para automatizar el historial de ventas y el historial de productos eliminados.

## Justificación

La realización de este proyecto responde a la necesidad de integrar el conocimiento teórico adquirido sobre bases de datos y programación orientada a objetos en una página web practica y funcional, al simular un sistema de pedidos, se abordan problemáticas reales y frecuentemente en empresas que aún no cuentan con soluciones automatizadas. Implementar este tipo de proyectos en el entorno académico permite consolidar sus habilidades técnicas.

Además, se promueve el trabajo colaborativo, la investigación aplicada y el uso de herramientas profesionales como MySQL y Java.

## Marco Teórico

### Java y JDBC

Java es un lenguaje de programación orientado a objetos, multiplataforma y con una sintaxis robusta y versátil. Su principal ventaja radica en su portabilidad. JDBC (Java Database Connectivity) es una Api proporcionada por Java que permite la conexión con bases de datos relacionales. JBBC actúa como un puente entre una ejecutar sentencias SQL, recuperar resultados y gestionar transacciones de forma eficiente.

### MySQL

MYSQL es uno de los sistemas de gestión de bases de datos más utilizados a nivel mundial. Es un software de código abierto que permite el almacenamiento estructurado de datos y su recuperación mediante el lenguaje SQL. Su arquitectura cliente-servidor permite que múltiples aplicaciones pueden interactuar con el simultáneamente, lo que lo convierte en una excelente opción para el desarrollo de aplicaciones empresariales.

### Triggers en MySQL

Los triggers son procedimientos almacenados que se ejecutan automáticamente en respuestas a ciertos eventos sobre una tabla. En este proyecto se utilizaron para controlar las existencias de productos y calcular los puntos acumulados por los clientes en función del total de cada compra. Esta lógica automatiza permite mantener la integridad de los datos y reducir errores.

## Metodología de Desarrollo Técnico-Aplicada

### Fase 1: Análisis de requerimientos.

Se definieron los requerimientos funcionales y técnicos del sistema, incluyendo:

- Uso del lenguaje Java con Spring Boot.

- Persistencia con JPA (Java Persistence API).
- Base de datos relacional en MySQL.
- Arquitectura en capas: modelos, controladores, servicios, repositorios, DTOs.
- Lombok

Tecnologías a utilizar:

- Backend: Spring Boot (Java), MySQL, Hibernate
- Frontend: HTML, CSS, JavaScript (Fetch API)
- Testing: Postman (para probar API)
- Desarrollo local: MySQL Workbench para base de datos

## Configuración de proyecto con Spring Boot

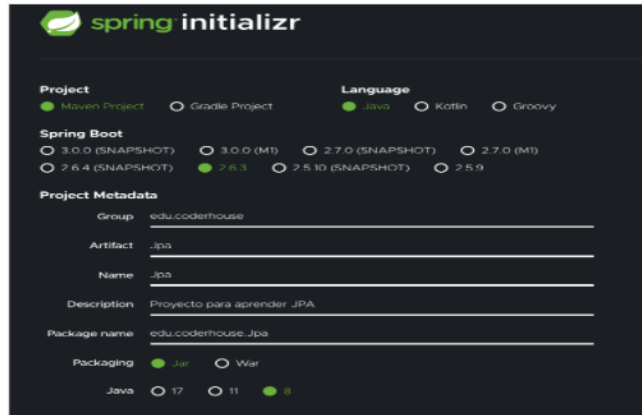
Asegúrate de tener instalado:

- **Java JDK 17 o superior** (ya estás usando Java 21).
- **Eclipse IDE for Enterprise Java Developers.**
- **Maven** (el proyecto es Mavenizado).
- **Base de datos MySQL** (si aplica en tu proyecto, parece que sí por la consola).

1. Ir al sitio [star.spring.io](https://star.spring.io)

# Creación del proyecto Spring Boot

1. Ir al sitio [start.spring.io](https://start.spring.io)
2. Configurar el proyecto de tipo Maven, lenguaje Java, Spring Boot versión 2.6.3, completar la metadata, el packaging va a ser en formato jar y la versión de Java va a ser la 8

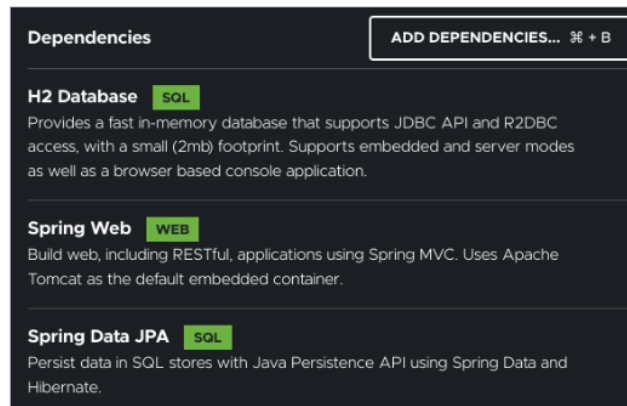


The screenshot shows the Spring Initializr web interface. Under 'Project', 'Maven Project' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', version '2.6.3' is selected. The 'Project Metadata' section includes fields for Group (edu.coderhouse), Artifact (jpa), Name (jpa), Description (Proyecto para aprender JPA), and Package name (edu.coderhouse-jpa). Under 'Packaging', 'Jar' is selected. Under 'Java', version '8' is selected.

2. Agregamos las siguientes configuraciones.

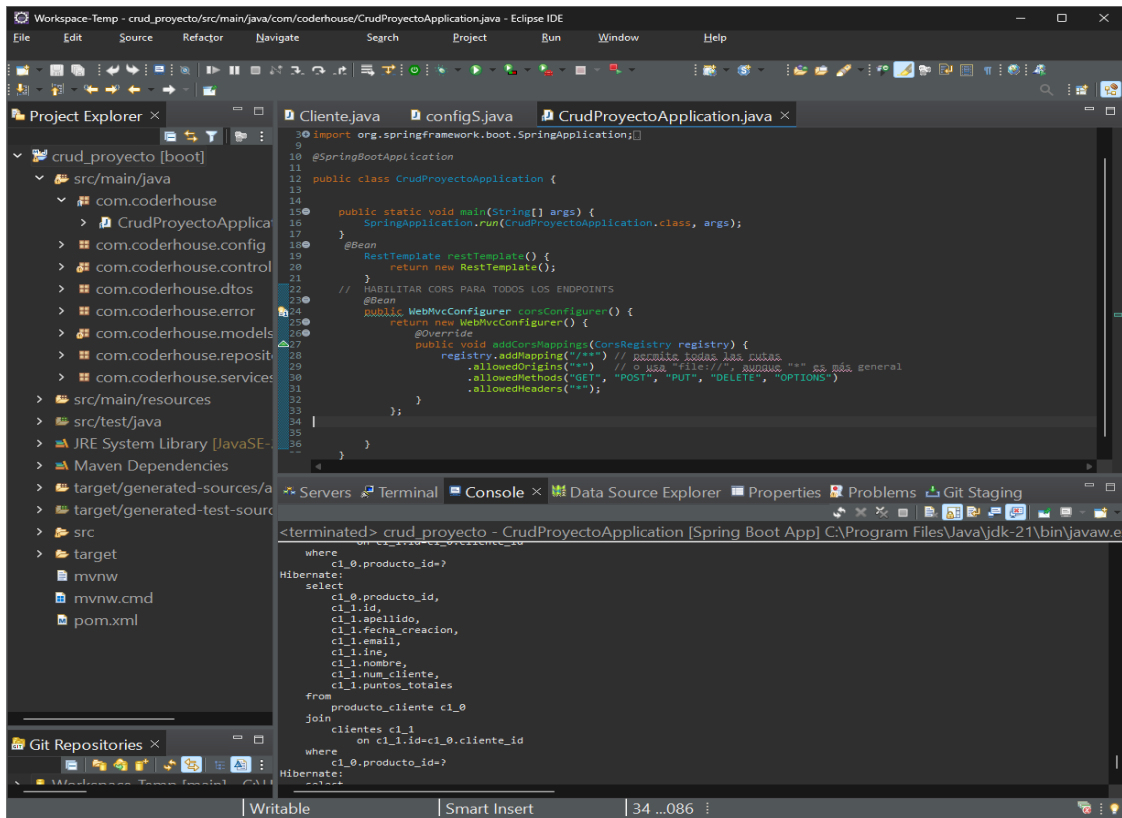
# Creación del proyecto Spring Boot

3. Se agregan las siguientes dependencias:
  - a. H2: Base de datos en memoria
  - b. Spring Web: La BBDD nos brinda una consola web entonces esta librería provee todo lo necesario para acceder a ella
  - c. Spring Data JPA: librería para implementar JPA con Hibernate



The screenshot shows the 'Dependencies' section of the Spring Initializr. There is a button 'ADD DEPENDENCIES... 96 + B'. Three dependencies are listed: 'H2 Database' (SQL), 'Spring Web' (WEB), and 'Spring Data JPA' (SQL). Each dependency has a brief description of its functionality.

3. En el IDE de ECLIPSE ya tenemos nuestro proyecto, pero se necesitan esas configuraciones mostradas anteriormente, por lo que ahora ya estructurando nuestro proyecto nos encontramos esta vista con nuestras carpetas configuradas.

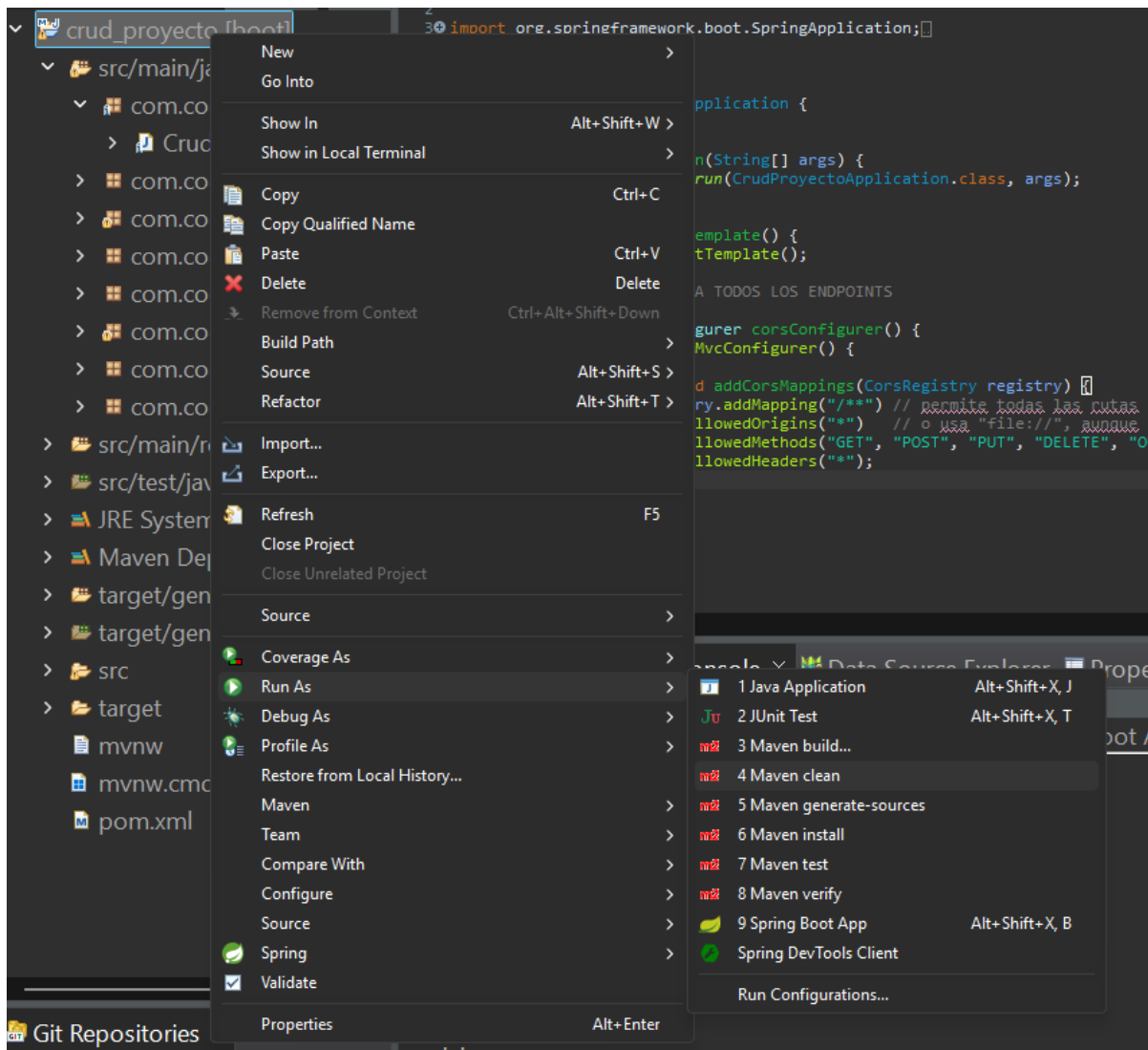


¿Qué hace esta clase?

1. **Es el punto de entrada principal** de tu aplicación Spring Boot.
2. **Está anotada con @SpringBootApplication, lo que:**
  - a. Habilita configuración automática (@EnableAutoConfiguration)
  - b. Escanea componentes (@ComponentScan)
  - c. Habilita configuración tipo Spring (@Configuration)
3. **Dentro tiene:**
  - a. main(): Ejecuta toda la aplicación.
  - b. restTemplate(): Permite hacer peticiones HTTP desde el servidor (por ejemplo, consumir otra API).
  - c. corsConfigurer(): Habilita CORS para aceptar peticiones desde cualquier frontend externo (como HTML+JS, React, etc.).
4. **Limpia y construye el proyecto**

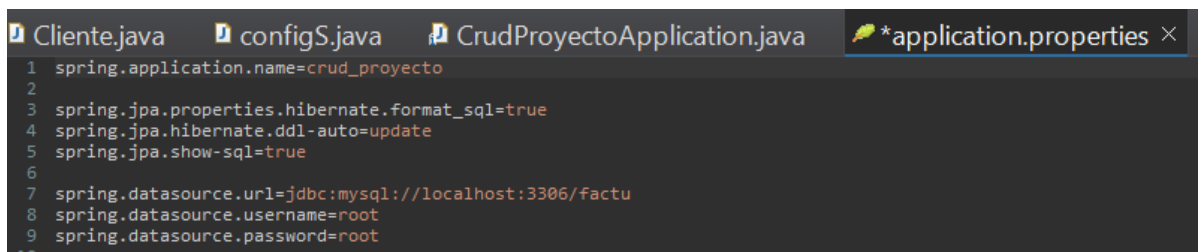
Haz clic derecho sobre el proyecto (crud\_proyecto) > Run As > Maven clean  
Luego: Run As > Maven install

Esto descargará dependencias y compilará el proyecto.



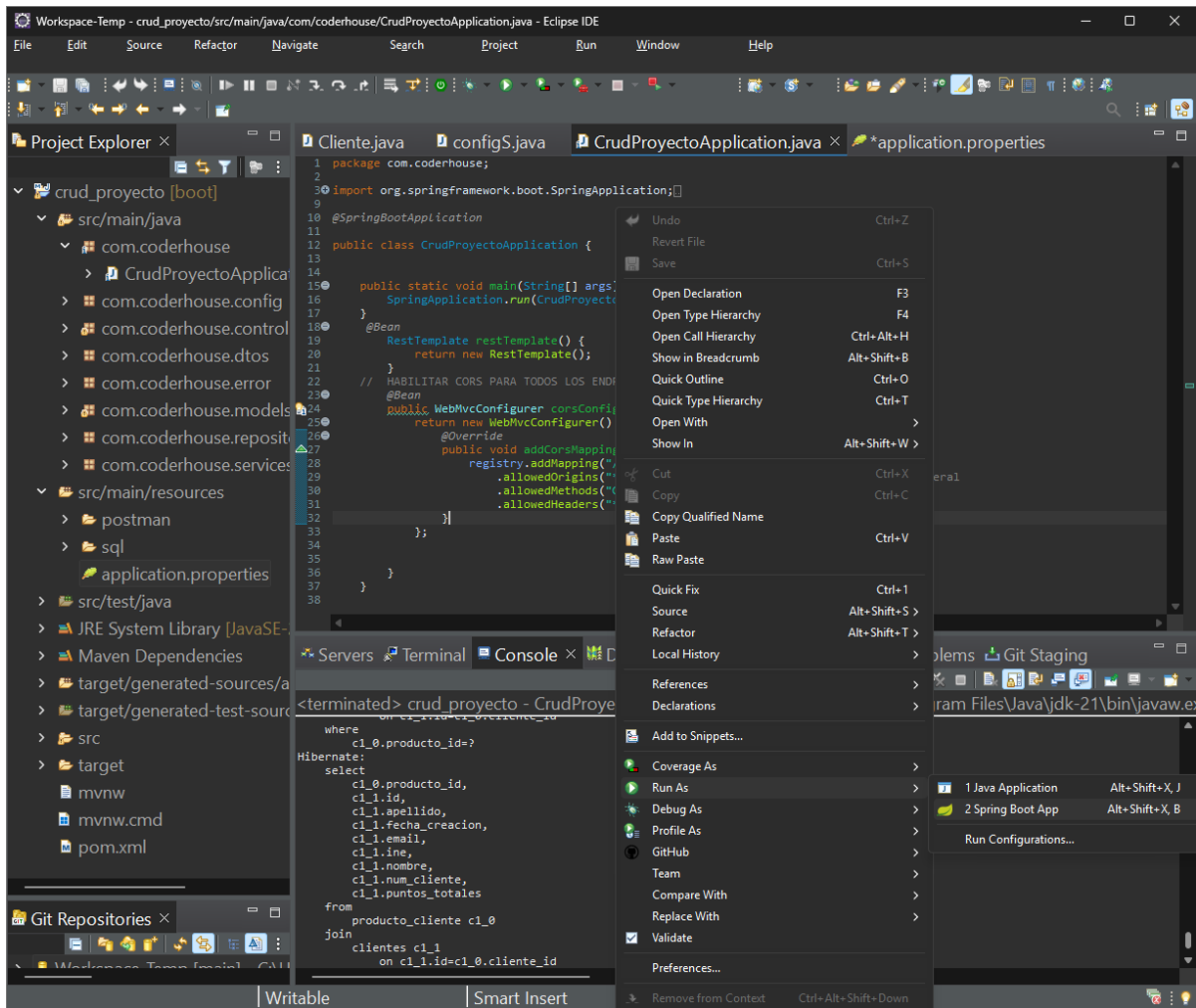
5. Revisa el archivo application.properties (o application.yml)

Confirma que tienes configurada correctamente la conexión a la base de datos:

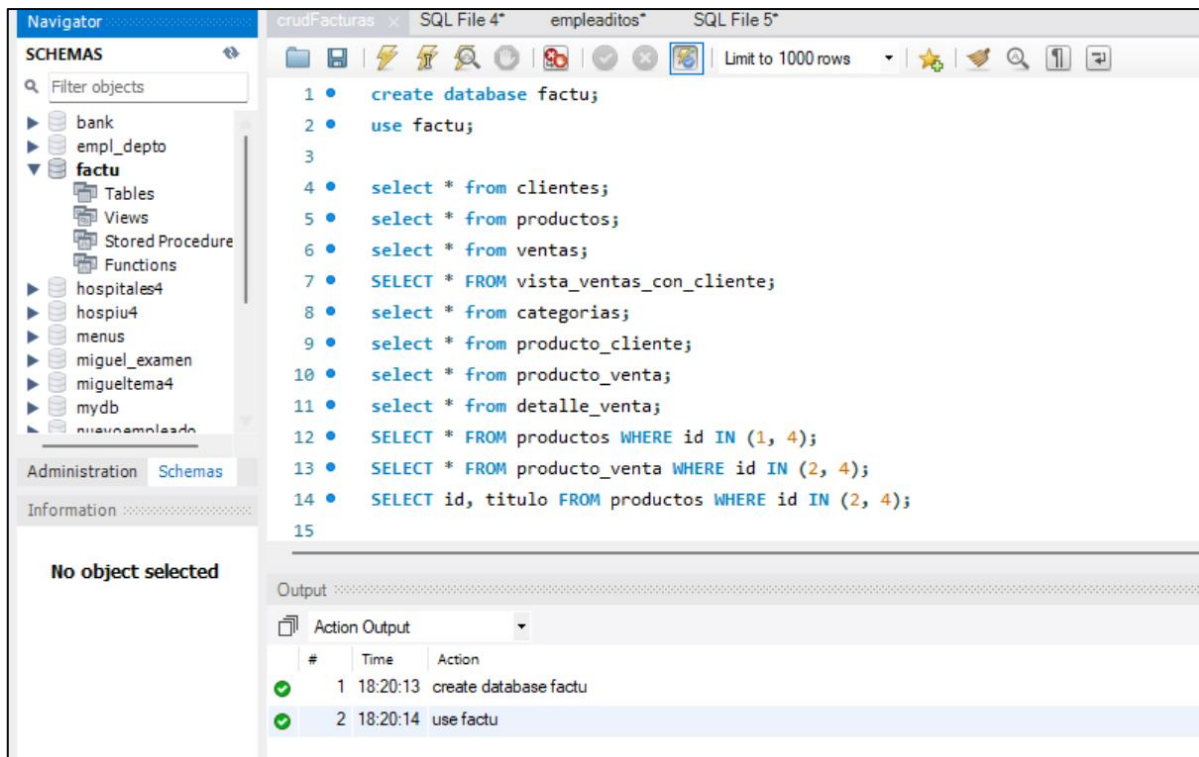


- Como podemos observar si esta correcta la ruta

6. Ejecutamos la aplicación.



- Seleccionamos la opción “2”.
7. Antes de correr nuestra aplicación creamos la base de datos en nuestro gestor de base de datos para que nuestra API entienda que se va a conectar a esa base de datos.
- Se crea correctamente, ahora si vamos al IDE.



8. Procedemos a correr nuestro proyecto:

- Cuando lo ejecutamos, nos da una vista que el proyecto se levantó de manera correcta y que también se corrieron las dependencias.

The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure for 'crud\_proyecto'. The main editor shows the 'CrudProjctoApplication.java' file. The Console on the right shows the application startup logs, including the following text:

```

2025-05-30T18:22:04.059-06:00 INFO 30168 --- [crud_proyecto] [
main] com.coderhouse.CrudProjctoApplication : Starti
2025-05-30T18:22:04.062-06:00 INFO 30168 --- [crud_proyecto] [
main] com.coderhouse.CrudProjctoApplication : No act
2025-05-30T18:22:04.783-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Multip
2025-05-30T18:22:04.785-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Bootst
2025-05-30T18:22:04.822-06:00 INFO 30168 --- [crud_proyecto] [
main] .RepositoryConfigurationExtensionSupport : Spring
2025-05-30T18:22:04.824-06:00 INFO 30168 --- [crud_proyecto] [
main] .RepositoryConfigurationExtensionSupport : Spring
2025-05-30T18:22:04.825-06:00 INFO 30168 --- [crud_proyecto] [
main] .RepositoryConfigurationExtensionSupport : Spring
2025-05-30T18:22:04.828-06:00 INFO 30168 --- [crud_proyecto] [
main] .RepositoryConfigurationExtensionSupport : Spring
2025-05-30T18:22:04.830-06:00 INFO 30168 --- [crud_proyecto] [
main] .RepositoryConfigurationExtensionSupport : Spring
2025-05-30T18:22:04.830-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Finish
2025-05-30T18:22:04.838-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Multip
2025-05-30T18:22:04.839-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Bootst
2025-05-30T18:22:04.874-06:00 INFO 30168 --- [crud_proyecto] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Finish
2025-05-30T18:22:05.340-06:00 INFO 30168 --- [crud_proyecto] [
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
2025-05-30T18:22:05.353-06:00 INFO 30168 --- [crud_proyecto] [
main] o.apache.catalina.core.StandardService : Starti
2025-05-30T18:22:05.354-06:00 INFO 30168 --- [crud_proyecto] [
main] o.apache.catalina.core.StandardEngine : Starti
2025-05-30T18:22:05.428-06:00 INFO 30168 --- [crud_proyecto] [
main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initia
2025-05-30T18:22:05.429-06:00 INFO 30168 --- [crud_proyecto] [
main] w.s.c.ServletWebServerApplicationContext : Root W
2025-05-30T18:22:05.578-06:00 INFO 30168 --- [crud_proyecto] [
main] o.hibernate.jpa.internal.util.LogHelper : HHH000
2025-05-30T18:22:05.641-06:00 INFO 30168 --- [crud_proyecto] [
main] org.hibernate.Version : HHH000
2025-05-30T18:22:05.671-06:00 INFO 30168 --- [crud_proyecto] [
main] o.h.c.internal.RegionFactoryInitiator : HHH000
2025-05-30T18:22:05.927-06:00 INFO 30168 --- [crud_proyecto] [
main] o.s.o.j.p.SpringPersistenceUnitInfo : No Loa
2025-05-30T18:22:05.957-06:00 INFO 30168 --- [crud_proyecto] [
main] com.zaxxer.hikari.HikariDataSource : Hikari
2025-05-30T18:22:06.338-06:00 INFO 30168 --- [crud_proyecto] [
main] com.zaxxer.hikari.pool.HikariPool : Hikari
2025-05-30T18:22:06.339-06:00 INFO 30168 --- [crud_proyecto] [
main] com.zaxxer.hikari.HikariDataSource : Hikari
2025-05-30T18:22:06.425-06:00 INFO 30168 --- [crud_proyecto] [
main] org.hibernate.orm.connections.pooling : HHH100
Database JDBC URL [Connecting through
Database driver: undefined/unknown
Database version: 8.0.40
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-05-30T18:22:07.414-06:00 INFO 30168 --- [crud_proyecto] [
main] o.h.e.t.j.p.i.itaPlatformInitiator : HHH000
Hibernate:
create table categorias (
  id bigint not null auto_increment,
  nombre varchar(255) not null,
  primary key (id)
)

```

- El hibernate Hibernate es un ORM (Object Relational Mapping) que traduce objetos Java en tablas de base de datos y viceversa. Esto evita escribir SQL manual para operaciones básicas (insertar, consultar, actualizar, eliminar), ya que Hibernate lo hace por ti.
- En este proyecto, se utiliza Hibernate como framework ORM (Object Relational Mapping), el cual permite mapear automáticamente las clases Java (entidades) con las tablas de la base de datos. Esto se logra usando anotaciones como @Entity, @Table, @Id, @OneToMany, @ManyToOne, entre otras. Hibernate se encarga de generar el SQL necesario para realizar operaciones CRUD, gestionar relaciones entre entidades y sincronizar los datos entre objetos y la base de datos, simplificando así el desarrollo y mantenimiento de la capa de persistencia del sistema.
- Hibernate aquí: Genera un INSERT INTO clientes (...) VALUES (...) automáticamente.

Convierte el objeto Cliente a una fila en la tabla.

Retorna el objeto ya guardado (con ID asignado por MySQL).

```
Servers Terminal Console Data Source Explorer Properties Problems Git Staging
crud_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 8.0.40
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-05-30T18:22:07.414-06:00 INFO 30168 --- [crud_proyecto] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000
Hibernate:
create table categorias (
  id bigint not null auto_increment,
  nombre varchar(255) not null,
  primary key (id)
) engine=InnoDB
Hibernate:
create table clientes (
  id bigint not null auto_increment,
  apellido varchar(255),
  fecha_creacion datetime(6),
  email varchar(255),
  ine integer not null,
  nombre varchar(255),
  num_cliente varchar(255),
  puntos_totales integer not null,
  primary key (id)
) engine=InnoDB
Hibernate:
create table detalle_venta (
  id bigint not null auto_increment,
  cantidad integer not null,
  precio_unitario float(53) not null,
  producto_venta_id bigint,
  venta_id bigint,
  primary key (id)
) engine=InnoDB
Hibernate:
create table producto_cliente (
  producto_id bigint not null,
  cliente_id bigint not null
) engine=InnoDB
Hibernate:
create table producto_venta (
  id bigint not null auto_increment,
  descripcion varchar(255),
  precio float(53) not null,
  stock integer not null,
  titulo varchar(255),
  categoria_id bigint,
  primary key (id)
) engine=InnoDB
table Smart Insert 20 ... 638
```

```
Servers Terminal Console Data Source Explorer Properties Problems Git Staging
crud_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20
primary key (id)
) engine=InnoDB
Hibernate:
create table productos (
  id bigint not null auto_increment,
  descripcion varchar(255),
  precio float(53) not null,
  stock integer not null,
  titulo varchar(255),
  categoria_id bigint,
  primary key (id)
) engine=InnoDB
Hibernate:
create table ventas (
  id bigint not null auto_increment,
  cantidad_productos integer not null,
  fecha datetime(6),
  precio_total float(53) not null,
  cliente_id bigint,
  primary key (id)
) engine=InnoDB
Hibernate:
alter table categorias
drop index UKqcg8b7hps1hioi9onqwjdt6y
Hibernate:
alter table categorias
add constraint UKqcg8b7hps1hioi9onqwjdt6y unique (nombre)
Hibernate:
alter table clientes
drop index UKk6e2jvvgm9hmica5xb6iss227
Hibernate:
alter table clientes
add constraint UKk6e2jvvgm9hmica5xb6iss227 unique (ine)
Hibernate:
alter table detalle_venta
add constraint FKil3sitipuouo4bwko73kf0210
foreign key (producto_venta_id)
references producto_venta (id)
Hibernate:
alter table detalle_venta
add constraint FK834rjd9cwo349m5xdf1knb101
foreign key (venta_id)
references ventas (id)
Hibernate:
alter table producto_cliente
add constraint FKkuo1pbhmh978y2g75tp510eq9
foreign key (cliente_id)
references clientes (id)
Hibernate:
table Smart Insert 20 ... 638
```

```
Servers Terminal Console x Data Source Explorer Properties Problems Git Staging
crud_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20
Hibernate:
    alter table categorias
    add constraint UKqcoq8b7hps1hioi9onqwjdt6y unique (nombre)
Hibernate:
    alter table clientes
    drop index UKk6e2jvvgm9hmica5xb6iss227
Hibernate:
    alter table clientes
    add constraint UKk6e2jvvgm9hmica5xb6iss227 unique (ine)
Hibernate:
    alter table detalle_venta
    add constraint FKil3siitipuouo4bwko73kf0210
    foreign key (producto_venta_id)
    references producto_venta (id)
Hibernate:
    alter table detalle_venta
    add constraint FK834rjd9cwo349m5xd1knbl01
    foreign key (venta_id)
    references ventas (id)
Hibernate:
    alter table producto_cliente
    add constraint FKkuo1pbhmh978y2g75tp5l0eq9
    foreign key (cliente_id)
    references clientes (id)
Hibernate:
    alter table producto_cliente
    add constraint FKl79de9h09qratkvfk1k6owvxx
    foreign key (producto_id)
    references productos (id)
Hibernate:
    alter table producto_venta
    add constraint FKrcr9xkxe56cyujmdyqx1hrbxx
    foreign key (categoria_id)
    references categorias (id)
Hibernate:
    alter table productos
    add constraint FK2fwql0nwymfv7fumctxt9vpqb
    foreign key (categoria_id)
    references categorias (id)
Hibernate:
    alter table ventas
    add constraint FK4dgjhcc12uu08swmx1xb4ipb5
    foreign key (cliente_id)
    references clientes (id)
2025-05-30T18:22:08.234-06:00 INFO 30168 --- [crud_proyecto] [main] j.LocalContainerEntityManagerFactoryBean : Initia
2025-05-30T18:22:08.642-06:00 WARN 30168 --- [crud_proyecto] [main] JpaBaseConfiguration$JpaWebConfiguration : spring
2025-05-30T18:22:08.689-06:00 INFO 30168 --- [crud_proyecto] [main] o.s.v.b.OptionalValidatorFactoryBean : Failed
2025-05-30T18:22:09.203-06:00 INFO 30168 --- [crud_proyecto] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
2025-05-30T18:22:09.209-06:00 INFO 30168 --- [crud_proyecto] [main] com.coderhouse.CrudProyectoApplication : Starter
itable | Smart Insert | 20 ... 638 |
```

El desarrollo de este proyecto se realizó mediante una **metodología técnico-aplicada**, centrada en el aprendizaje práctico del estudiante a través de la implementación de una solución funcional. Este enfoque no solo facilita la asimilación de conocimientos teóricos, sino que también permite aplicar diversas áreas del conocimiento —como programación orientada a objetos, bases de datos relacionales y arquitecturas REST— dentro de un entorno controlado y realista, simulando escenarios del mundo profesional.

## Requerimientos Técnicos Iniciales

Como primer paso, se definieron los requerimientos técnicos esenciales del sistema. Se optó por el uso del lenguaje de programación **Java**, junto con la API **JPA (Java Persistence API)**. Esta API proporciona un marco estandarizado para la gestión de datos relacionales mediante

técnicas de mapeo objeto-relacional (ORM), facilitando la traducción entre las **tablas de una base de datos** y las **clases de Java**, denominadas **entidades**.

A través de JPA (implementada con Hibernate en este proyecto), se logró una gestión eficiente de la persistencia de datos, reduciendo la necesidad de escribir consultas SQL manuales y permitiendo la manipulación de datos desde objetos Java.

## Arquitectura y Herramientas

El entorno de desarrollo empleado fue **Eclipse IDE**, utilizando el **framework Spring Boot**, el cual facilitó la organización del código en capas:

- **Modelos o Entidades** (@Entity)
- **Repositorios** (@Repository, usando JpaRepository)
- **Servicios** (@Service)
- **Controladores REST** (@RestController)
- **DTOs (Data Transfer Objects)**
- **Configuraciones de conexión** a base de datos.

Para la persistencia, se utilizó el sistema gestor de bases de datos **MySQL**, el cual se conectó a través del archivo de configuración application.properties, donde se definieron parámetros clave como el dialect, el modo ddl-auto, y las credenciales de conexión.

## Diseño de la Base de Datos

La base de datos se denominó **factu**, y fue diseñada siguiendo los principios de normalización para garantizar la integridad, coherencia y eficiencia del sistema. Aunque inicialmente se menciona como un modelo no relacional, el diseño es **relacional**, ya que establece vínculos claros entre entidades a través de claves foráneas.

El modelo de datos se compone de las siguientes **tablas relacionales**:

- **Cliente**: contiene la información del cliente (ID, nombre, apellido, correo electrónico, número de identificación oficial).
- **Producto**: almacena los datos de cada producto (ID, nombre, precio, stock).
- **Categoría**: define la clasificación de los productos (ID, nombre de categoría).
- **Venta**: representa las transacciones de compra realizadas, incluyendo la fecha, el total, los puntos generados, y las relaciones con cliente, producto y categoría.

Estas tablas están relacionadas mediante anotaciones JPA como @OneToMany, @ManyToOne y @JoinColumn, gestionadas por Hibernate para facilitar la creación automática de relaciones en la base de datos.

## Fase 2: Diseño del Backend (API REST Con Spring Boot).

Se diseñó un modelo relacional con las siguientes entidades:

- **Cliente:** nombre, apellido, correo, INE, puntos acumulados.
- **Producto:** nombre, stock, precio.
- **Categoría:** nombre de categoría.
- **Venta:** fecha, total, cliente asociado.
- **ProductoVenta:** tabla intermedia entre producto y venta, que incluye cantidad y subtotal.

Se establecieron las relaciones entre entidades:

- @OneToMany entre Cliente y Venta.
- @ManyToOne entre Venta y Cliente.
- @ManyToOne entre ProductoVenta y Producto, así como ProductoVenta y Venta.
- @ManyToOne entre Producto y Categoría.
- Un Producto puede tener una Categoría
- Una Venta tiene múltiples DetalleVenta
- Cada DetalleVenta tiene un Producto y cantidad

### Uso de relaciones JPA(Java Persistence API) en los Modelos implementados.

En una aplicación CRUD con Spring Boot que maneja entidades como **Producto**, **Cliente**, **Venta**, **DetalleVenta** y **Categoría**, es fundamental usar **relaciones JPA** para mapear correctamente las dependencias entre tablas y facilitar operaciones con la base de datos.

- @OneToMany y @ManyToOne: Producto-Categoría

**Relación:** Muchos productos pueden pertenecer a una sola categoría.

Código Ejemplo en Producto-Venta:

```
Venta.java | ProductoVen... | Producto.java | DetalleVent...
1 package com.coderhouse.models;
2
3 import io.swagger.v3.oas.annotations.media.Schema;
4
5 @Getter
6 @Setter
7 @ToString
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @Entity
11 @Table(name = "producto_venta")
12 @Schema(description = "Modelo de Producto-Venta", title = "Modelo de Producto Venta")
13 public class ProductoVenta {
14
15     @Schema(description = "ID del detalle de venta", requiredMode= Schema.RequiredMode.REQUIRED)
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Long id;
19
20     @Schema(description = "Nombre del producto", requiredMode= Schema.RequiredMode.REQUIRED)
21     private String titulo;
22
23     @Schema(description = "Descripcioon", requiredMode= Schema.RequiredMode.REQUIRED)
24     private String descripcion;
25
26     @Schema(description = "Precio del producto", requiredMode= Schema.RequiredMode.REQUIRED)
27     private double precio;
28
29     @Schema(description = "Stock existente", requiredMode= Schema.RequiredMode.REQUIRED)
30     private int stock;
31
32     @ManyToOne(fetch = FetchType.EAGER)
33     @JoinColumn(name = "categoria_id")
34     private Categoria categoria;
35 }
36
37
38
39
40
41
42
43
44
45
```

#### Explicación:

- @ManyToOne: En la clase Producto, indica que muchos productos pueden estar asociados a una sola categoría.

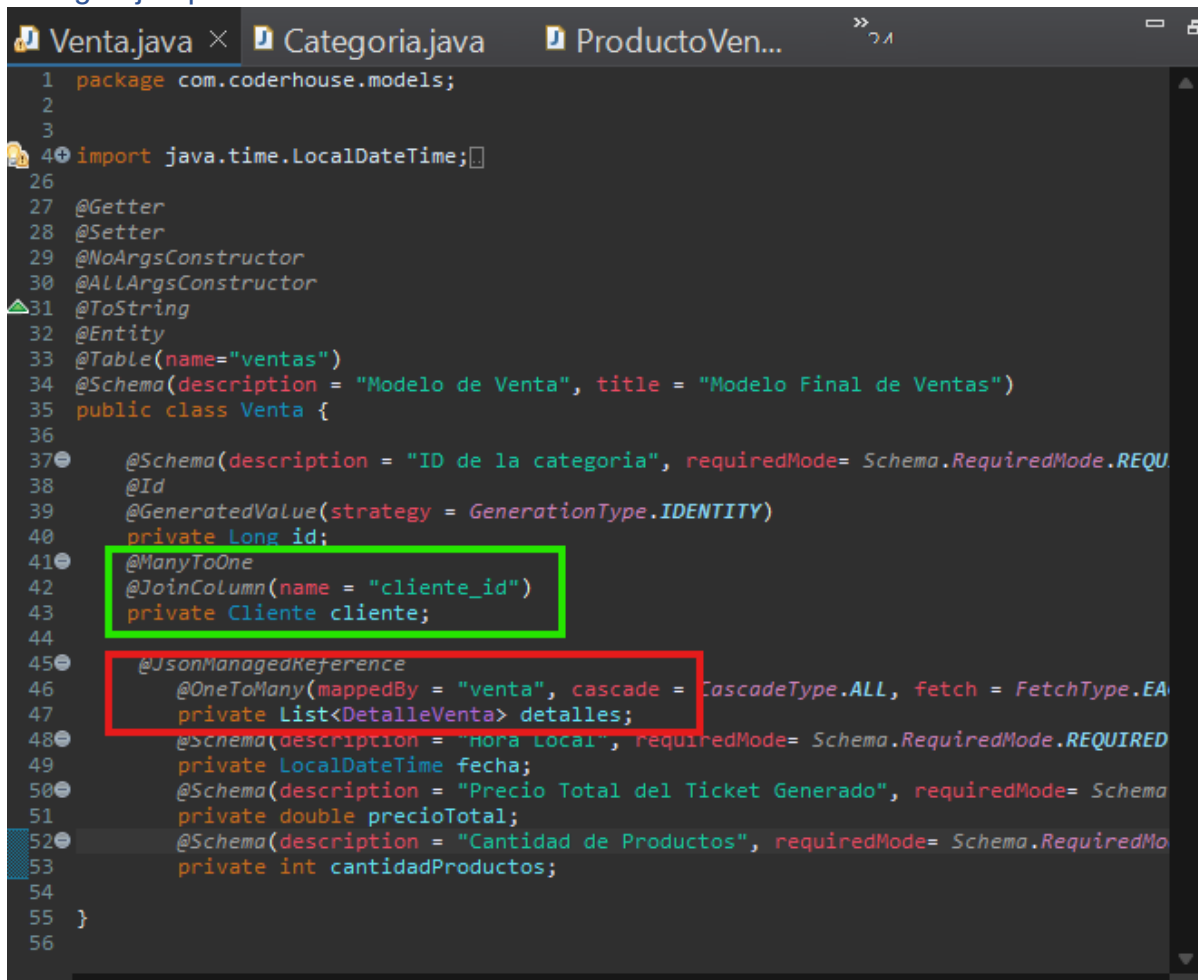
En el apartado de Categoría:

```
Categoria.java × ProductoVen... Producto.java » 2.1
1 package com.coderhouse.models;
2
3 import java.util.ArrayList;
22
23 @Getter
24 @Setter
25 @ToString
26 @NoArgsConstructor
27 @AllArgsConstructor
28 @Table(name = "categorias")
29 @Entity
30 @Schema(description = "Modelo de Categoria", title = "Modelo Final Categorias")
31 public class Categoria {
32
33     @Schema(description = "ID de la categoria", requiredMode= Schema.RequiredMode.REQUIRED)
34     @Id
35     @GeneratedValue(strategy = GenerationType.IDENTITY)
36     private Long id;
37
38     @Schema(description = "Nombre de la categoria", requiredMode= Schema.RequiredMode.REQUIRED)
39     @Column(unique = true, nullable = false)
40     private String nombre;
41
42     @OneToMany(mappedBy = "categoria", fetch = FetchType.EAGER)
43     @JsonIgnore
44     private List<Producto> productos = new ArrayList<>();
45
46 }
47
```

- @OneToMany: En la clase Categoría, indica que una categoría puede tener muchos productos asociados.
- mappedBy: Se refiere al nombre del atributo que hace la relación en la otra entidad ("categoria" en este caso).

➤ @OneToMany y @ManyToOne: Venta – DetalleVenta

Código ejemplo:



```
1 package com.coderhouse.models;
2
3
4 import java.time.LocalDateTime;
5
6 @Getter
7 @Setter
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @ToString
11 @Entity
12 @Table(name="ventas")
13 @Schema(description = "Modelo de Venta", title = "Modelo Final de Ventas")
14 public class Venta {
15
16     @Schema(description = "ID de la categoria", requiredMode= Schema.RequiredMode.REQUIRED)
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @ManyToOne
22     @JoinColumn(name = "cliente_id")
23     private Cliente cliente;
24
25     @JsonManagedReference
26     @OneToMany(mappedBy = "venta", cascade = CascadeType.ALL, fetch = FetchType.EAGER)
27     private List<DetalleVenta> detalles;
28
29     @Schema(description = "hora Local", requiredMode= Schema.RequiredMode.REQUIRED)
30     private LocalDateTime fecha;
31
32     @Schema(description = "Precio Total del Ticket Generado", requiredMode= Schema.RequiredMode.REQUIRED)
33     private double precioTotal;
34
35     @Schema(description = "Cantidad de Productos", requiredMode= Schema.RequiredMode.REQUIRED)
36     private int cantidadProductos;
37
38 }
39
```

Aqui podemos observar que hay dos relaciones

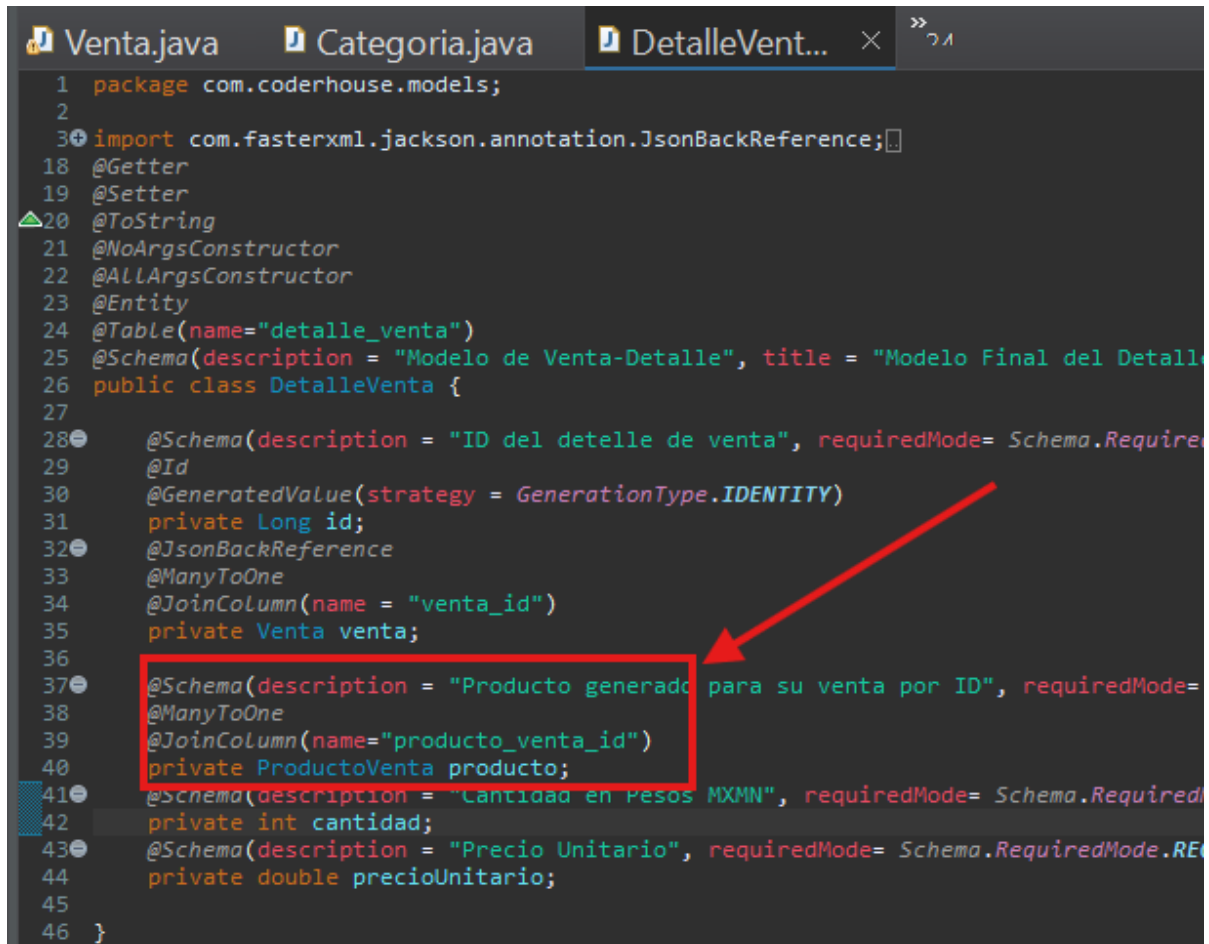
Explicación:

- En la tabla cliente que se hace una unión que asocia la venta con un cliente, a su misma vez la tabla ventas, muchas ventas registradas están relacionadas a un solo registro en la tabla clientes
- El cascade = CascadeType.ALL permite que, al guardar una venta, también se guarden automáticamente sus detalles.

Para la segunda relación:

Relación: Una venta contiene varios detalles de venta(Esta marcada en el recuadro rojo).

En el modelo DetalleVenta:



```
1 package com.coderhouse.models;
2
3 import com.fasterxml.jackson.annotation.JsonBackReference;
4
5 @Getter
6 @Setter
7 @ToString
8 @NoArgsConstructor
9 @AllArgsConstructor
10 @Entity
11 @Table(name="detalle_venta")
12 @Schema(description = "Modelo de Venta-Detalle", title = "Modelo Final del Detalle")
13 public class DetalleVenta {
14
15     @Schema(description = "ID del detalle de venta", requiredMode= Schema.RequiredMode.REQUIRED)
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Long id;
19
20     @JsonBackReference
21     @ManyToOne
22     @JoinColumn(name = "venta_id")
23     private Venta venta;
24
25     @Schema(description = "Producto generado para su venta por ID", requiredMode= Schema.RequiredMode.REQUIRED)
26     @ManyToOne
27     @JoinColumn(name="producto_venta_id")
28     private ProductoVenta producto;
29
30     @Schema(description = "Cantidad en Pesos MXMN", requiredMode= Schema.RequiredMode.REQUIRED)
31     private int cantidad;
32
33     @Schema(description = "Precio Unitario", requiredMode= Schema.RequiredMode.REQUIRED)
34     private double precioUnitario;
35
36 }
```

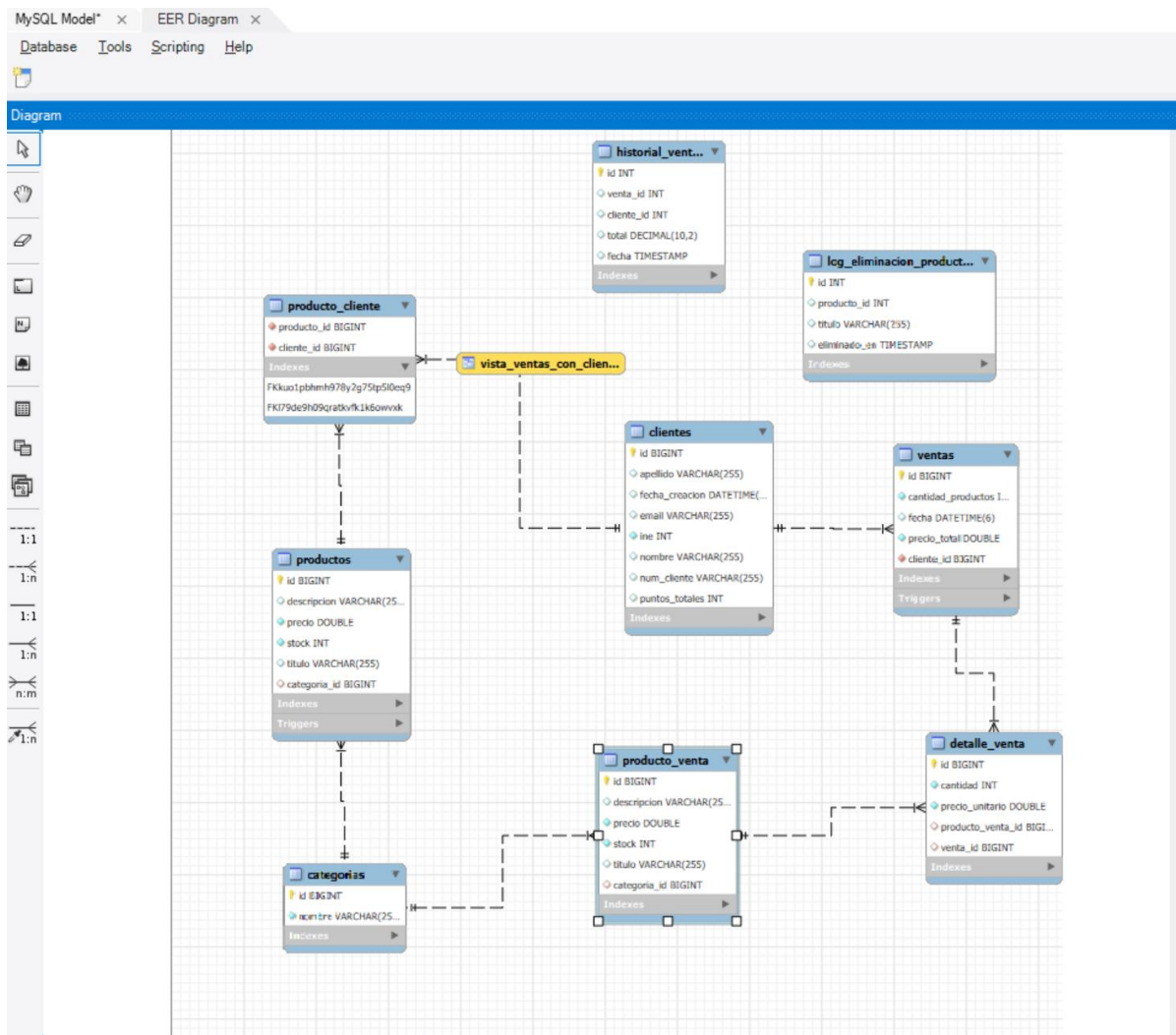
Explicación:

- Esto permite registrar **varios productos en una misma venta**.

### Beneficios de usar relaciones JPA

- Automatiza la creación de claves foráneas.
- Facilita la navegación entre entidades (por ejemplo: producto.getCategoria().getNombre()).
- Permite guardar o eliminar en cascada registros relacionados.
- Reduce la escritura de SQL manual.

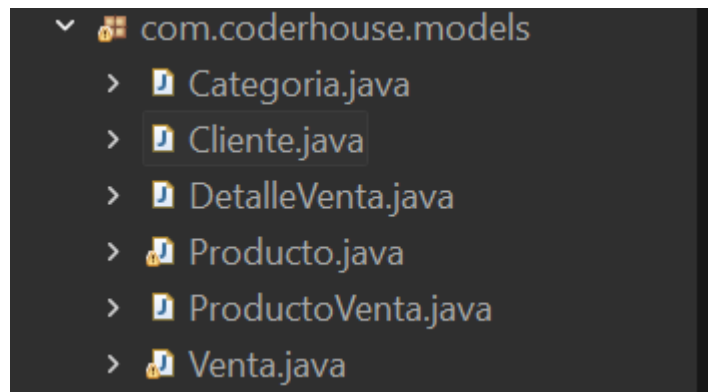
MODELO RELACIONAL:



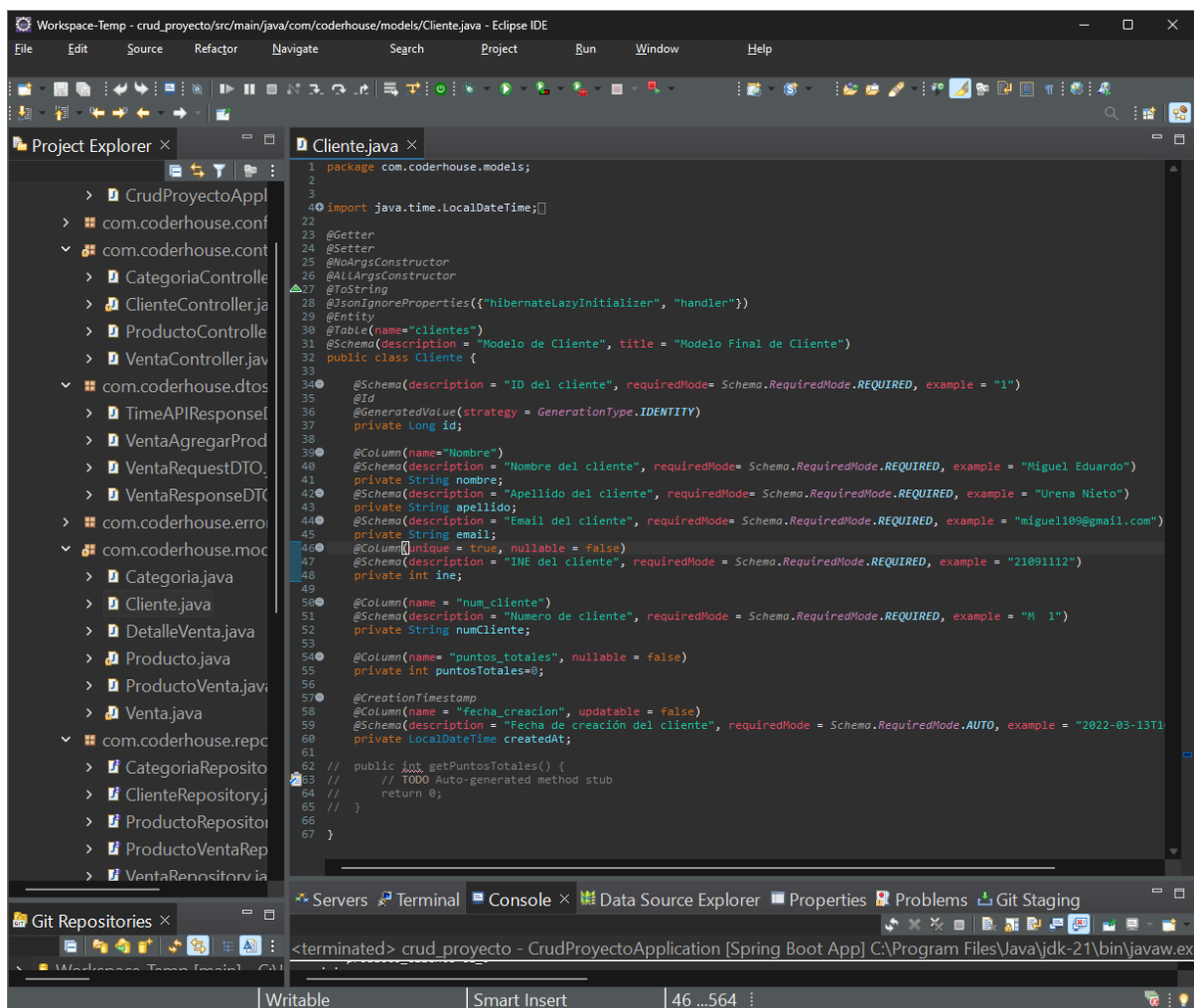
### Fase 3: Implementación.

Se estructuró el proyecto bajo la arquitectura MVC con capas bien definidas:

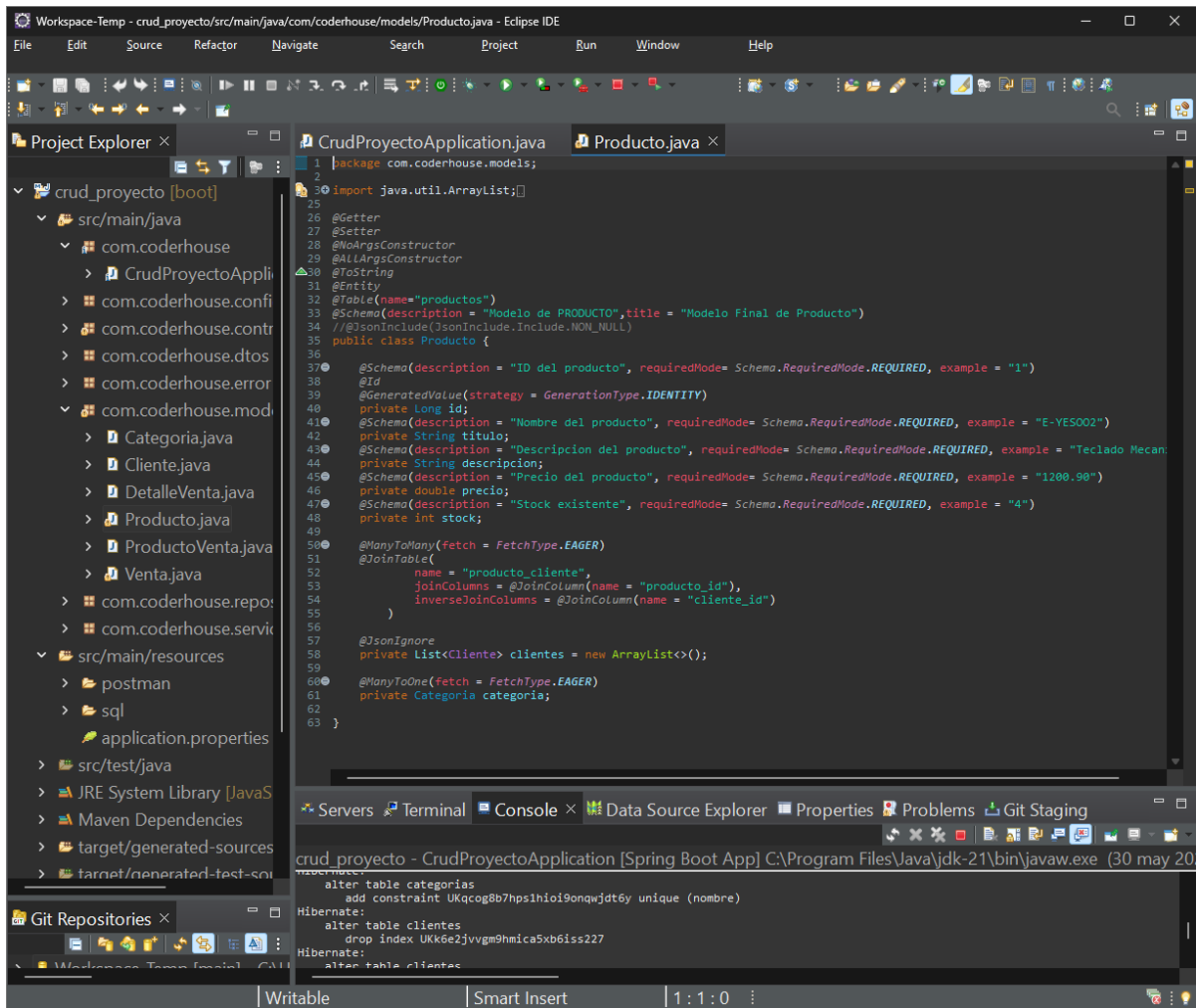
1. **Modelos:** Entidades JPA con anotaciones de relaciones (@Entity, @Id, @ManyToOne, @OneToMany, @ManyToMany, etc.).
- Para la capa modelos que en este caso es la carpeta “Models” donde desplegaremos nuestros actores principales que son “Cliente”, “Producto”, “Categoría”, “Venta”, “ProductoVenta”, “DetalleVenta”. Gracias a la herramienta JPA se automatiza la creación de las tablas de estas mismas entidades sin la necesidad de crear una base de datos robusta como la que conocemos, con esta herramienta podremos hacer las relaciones ya explicadas anteriormente mandándole a la entidad cual va a ser su relación gracias a este mapeo de datos.
- Las entidades son clases anotadas con @Entity, y representan directamente una tabla en la base de datos.



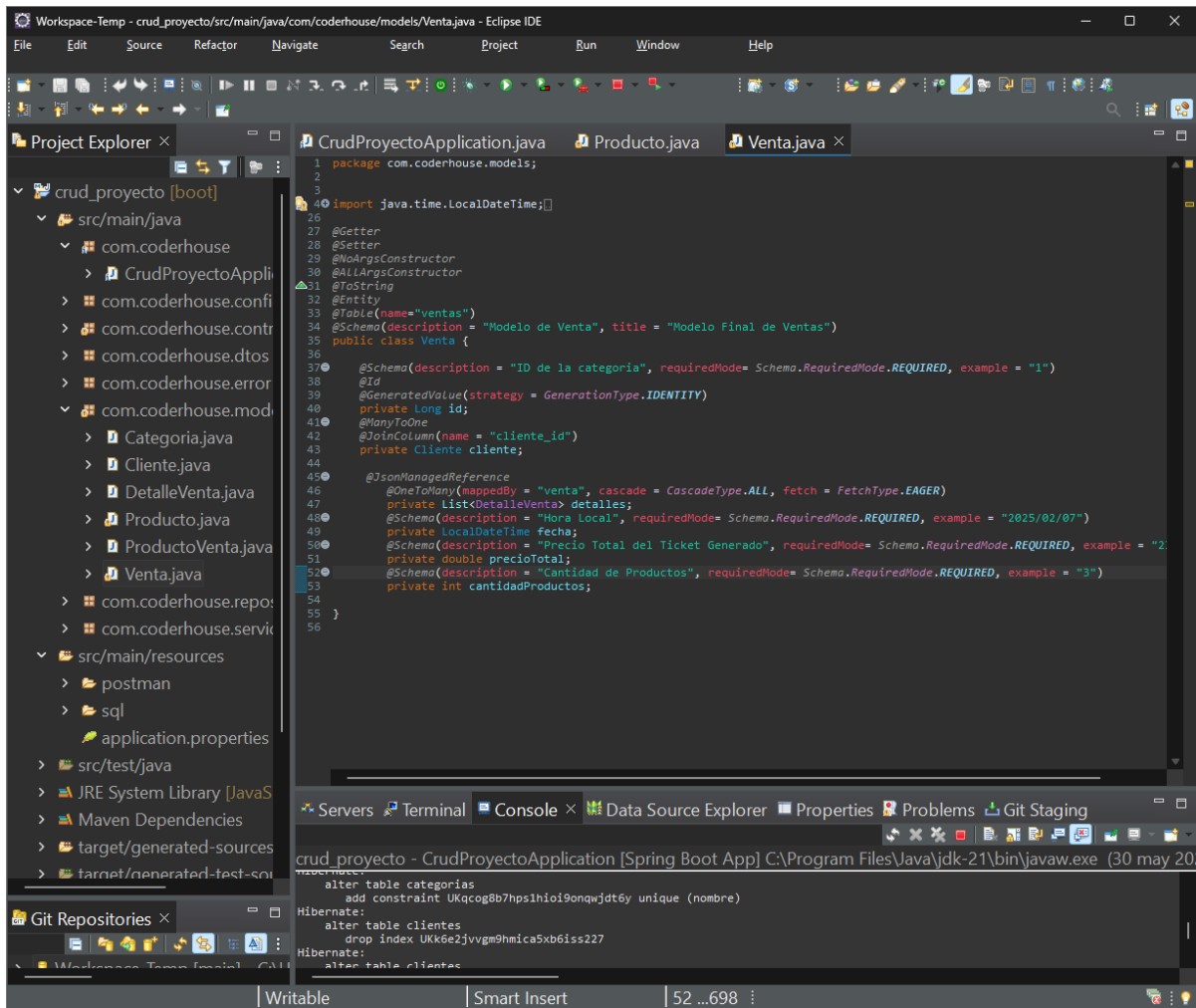
✓ Tabla cliente generada en una clase de Java.



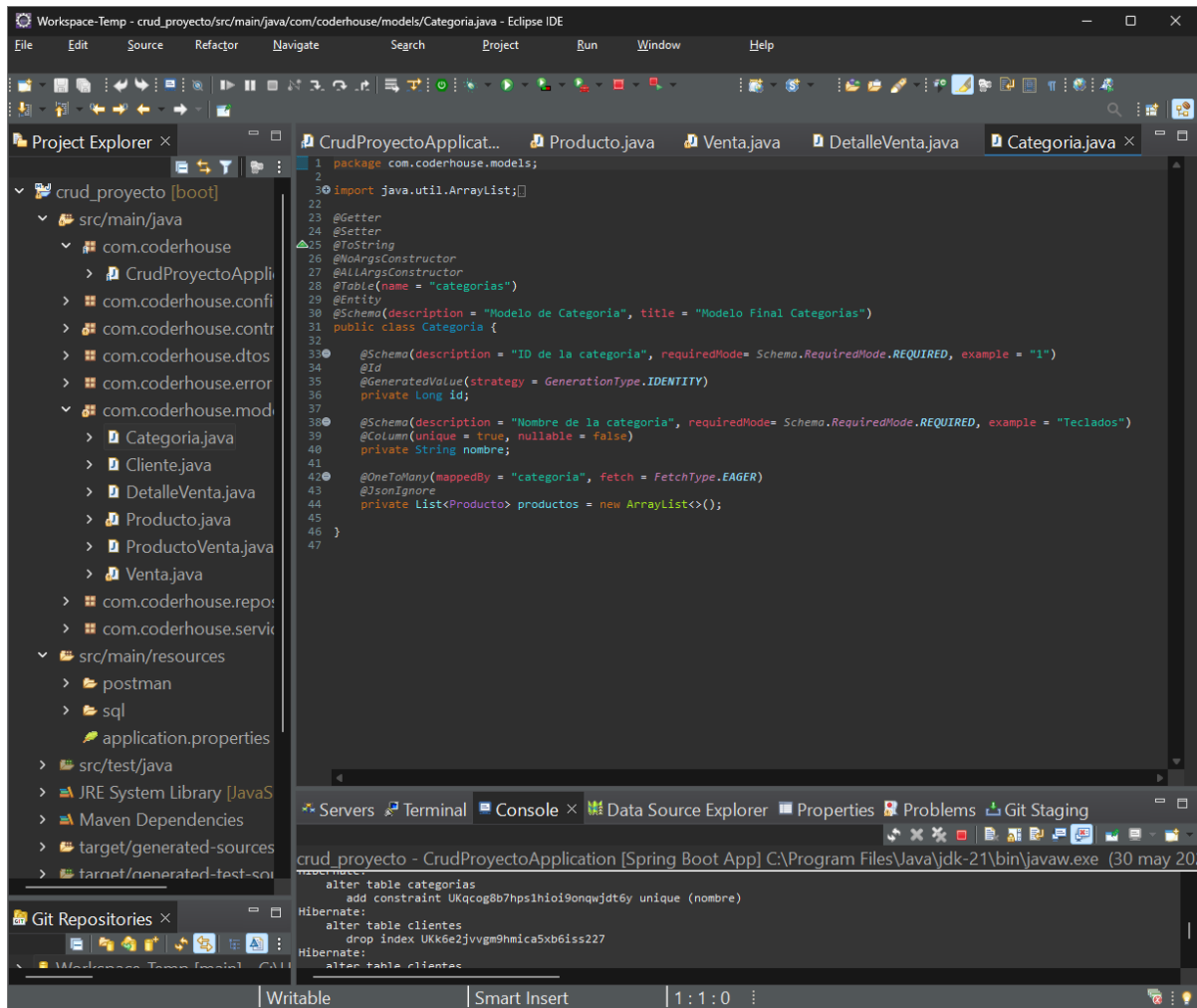
➤ Tabla “productos” mediante una clase en Java.



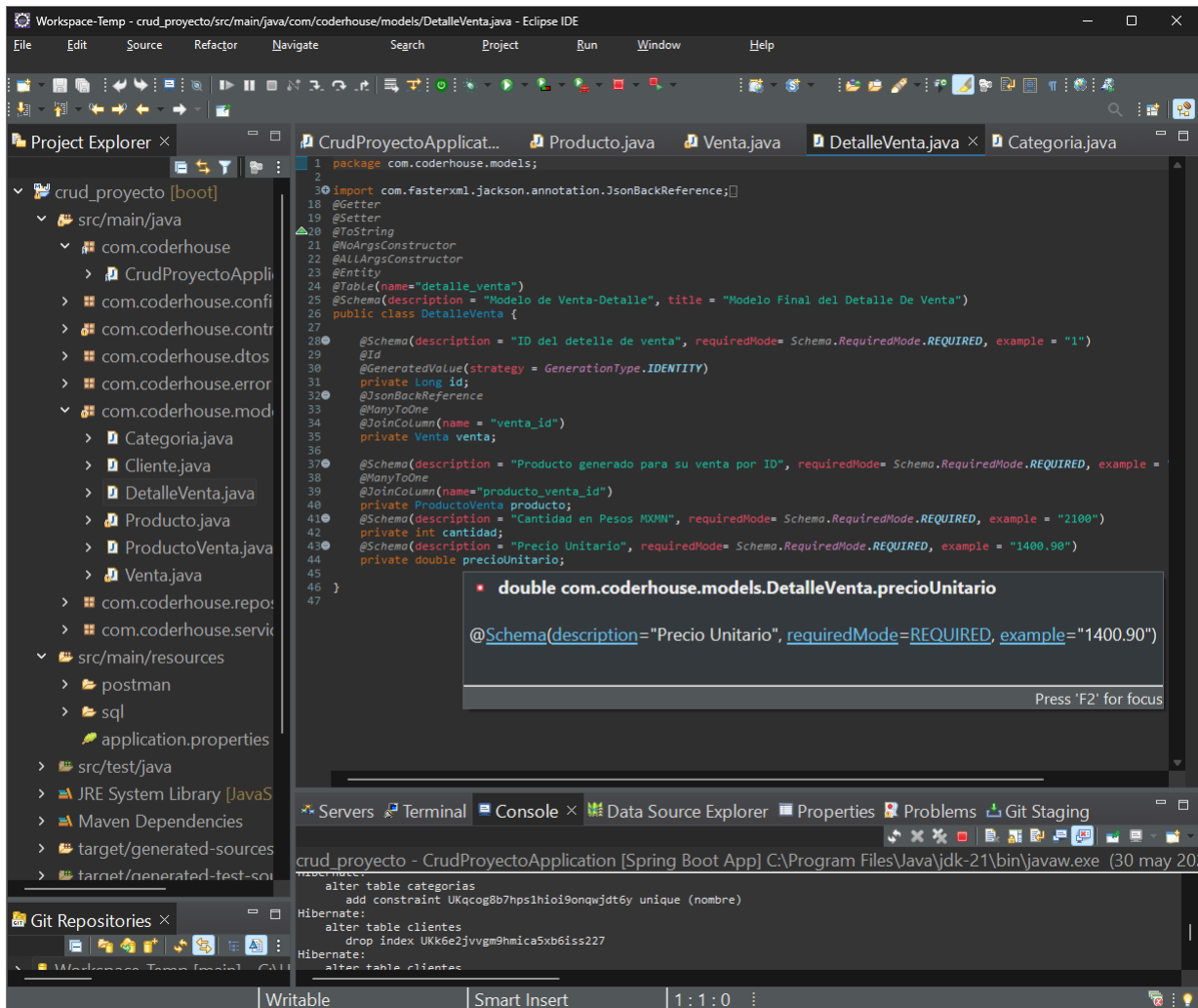
- Tabla ventas generada en una clase en Java.



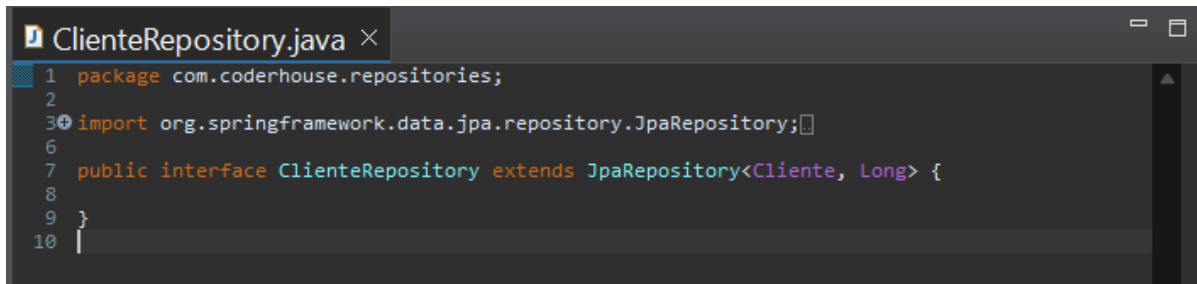
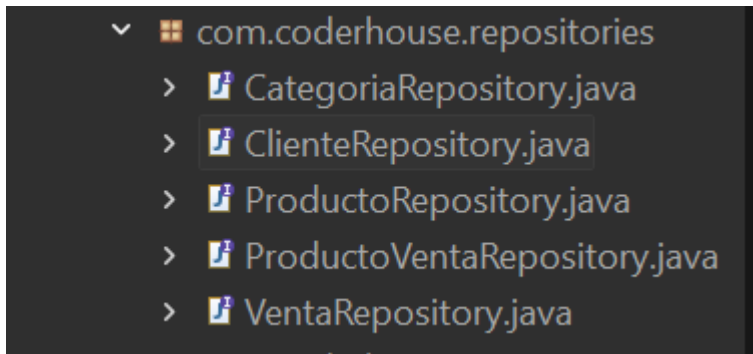
- Tabla Categorías generada en una clase en Java.



- Tabla DetalleVenta generada en una clase en Java.



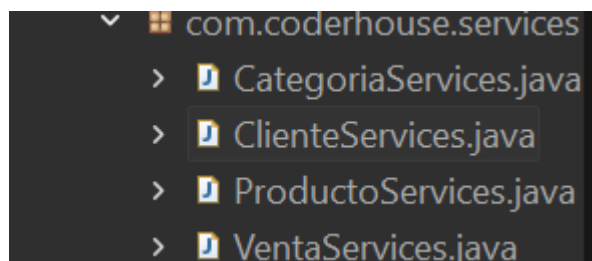
- Hibernate convierte esta clase en una tabla.
  - Cada instancia representa un registro.
  - Se usan anotaciones como @Id, @OneToMany, @JoinColumn para definir relaciones
2. **Repositorios:** Extienden JpaRepository para facilitar operaciones CRUD, permitiendo acceso automático a operaciones básicas como findAll(), findById(), save(), deleteById().
- En nuestra carpeta “repositories” tenemos los repositorios de todos nuestros modelos.



- Como podemos observar estas interfaces funcionan como la tercera capa que tiene la responsabilidad de conectarse a una base de datos que en este caso es “factu” recordemos que los datos en este caso del cliente nos retornan la clase “Cliente” de nuestro modelo gracias al “extends” que extiende JpaRepository.  
NOTA: SERA EL MISMO PROCEDIMIENTO PARA LAS DEMAS CLASES QUE SOLO CAMBIA EL NOMBRE.

3. **Servicios:** Contienen la lógica de negocio.

- Creamos el service que es la segunda capa o capa lógica que es la que se comunica con la tercer capa o capa de datos, para procesar la información recibida y devolver al Controller.
- Aquí se encuentra la lógica de negocio. Los servicios se encargan de:
- Procesar datos antes o después de guardarlos.
- Validar reglas de negocio.
- Comunicar el controlador con el repositorio.
- Convertir DTOs a entidades y viceversa.
- Distribución de nuestros servicios para la logica funcional de cada actor:



1. Empezaremos con lógica del cliente:

```
*ClienteServices.java ×
1 package com.coderhouse.services;
2 import java.util.List;
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5 import com.coderhouse.models.Cliente;
6 import com.coderhouse.repositories.ClienteRepository;
7 import jakarta.transaction.Transactional;
8
9 @Service
10 public class ClienteServices {
11
12     @Autowired
13     private ClienteRepository clienteRepository;
14     public List<Cliente> getAllClientes() {
15         return clienteRepository.findAll();
16     }
17     public Cliente getClienteById(Long id) {
18         return clienteRepository.findById(id)
19             .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado"));
20     }
21     @Transactional
22     public Cliente newCliente(Cliente clienteInfo) {
23         return clienteRepository.save(clienteInfo);
24     }
25
26     @Transactional
27     public Cliente updateClienteById(Long id, Cliente clienteInfo) {
28         Cliente cliente = clienteRepository.findById(id)
29             .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado"));
30
31         if(clienteInfo.getNombre() != null && !clienteInfo.getNombre().isEmpty()) {
32             cliente.setNombre(clienteInfo.getNombre());
33         }
34
35         if(clienteInfo.getApellido() != null && !clienteInfo.getApellido().isEmpty()) {
36             cliente.setApellido(clienteInfo.getApellido());
37         }
38
39         if(clienteInfo.getEmail() != null && !clienteInfo.getEmail().isEmpty()) {
40             cliente.setEmail(clienteInfo.getEmail());
41         }
42
43         return clienteRepository.save(cliente);
44     }
45
46     public void deleteClienteById(Long id) {
47         if(!clienteRepository.existsById(id)) {
48             throw new IllegalArgumentException("Cliente no encontrado");
49         }
50         clienteRepository.delete(id);
51     }
52 }
```

```
*ClienteServices.java ×
1 package com.coderhouse.services;
2 import java.util.List;
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5 import com.coderhouse.models.Cliente;
6 import com.coderhouse.repositories.ClienteRepository;
7 import jakarta.transaction.Transactional;
8
```

- Package: Indica el paquete Java donde se encuentra esta clase. En este caso, pertenece al paquete services, encargado de la lógica de negocio.

- List: para manejar listas de clientes.
- @Autowired: para la inyección automática de dependencias.
- @Service: marca esta clase como servicio de Spring, habilitándola para ser utilizada e inyectada en otras capas.
- Cliente: la entidad del cliente.
- ClienteRepository: interfaz que permite acceso a la base de datos.
- @Transactional: garantiza que las operaciones de base de datos sean atómicas y seguras.

Se realiza la inyección:

@Autowired

```
private ClienteRepository clienteRepository;
```

Se inyecta automáticamente el repositorio ClienteRepository para poder acceder a los métodos CRUD de JPA sin instanciarlo manualmente.

### Método 1: getAllClients()

```
public List<Cliente> getAllClients(){
    return clienteRepository.findAll();
}
```

- Devuelve una lista con **todos los clientes** almacenados en la base de datos.
- Usa findAll(), un método ya implementado por JpaRepository.

### Método 2: getClienteById(Long id)

```
public Cliente getClienteById(Long id) {
    return clienteRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado"));
}
```

Busca un cliente por su **ID**. Si no lo encuentra, lanza una excepción con el mensaje "Cliente no encontrado". Esto es útil para evitar que el programa continúe con datos inexistentes.

### Método 3: newCliente(Cliente clienteInfo)

```
@Transactional
public Cliente newCliente(Cliente clienteInfo) {
    return clienteRepository.save(clienteInfo);
}
```

```
}
```

Crea un **nuevo cliente** en la base de datos. Usa `@Transactional` para garantizar que, si ocurre un error en medio del proceso, la operación se **revierte**.

`save()` guarda el nuevo cliente.

#### Método 4: `updateClienteById(Long id, Cliente clienteInfo)`

```
@Transactional
public Cliente updateClienteById(Long id, Cliente clienteInfo) {
    Cliente cliente = clienteRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Cliente no encontrado"));
```

Este método **actualiza los datos de un cliente existente**. Primero, busca el cliente por ID. Si no existe, lanza una excepción.

```
    if(clienteInfo.getNombre() != null && !clienteInfo.getNombre().isEmpty()) {
        cliente.setNombre(clienteInfo.getNombre());
    }
```

Verifica si el nuevo nombre es **no nulo y no vacío**, y si es así, lo actualiza. Lo mismo se hace para apellido y email en los siguientes bloques.

```
    return clienteRepository.save(cliente);
}
```

Guarda los cambios en la base de datos. Aunque ya exista el cliente, `save()` también actualiza si el objeto tiene un ID existente.

#### Método 5: `deleteClienteById(Long id)`

```
public void deleteClienteById(Long id) {
    if (!clienteRepository.existsById(id)) {
        throw new IllegalArgumentException("Cliente no encontrado");
    }
    clienteRepository.deleteById(id);
}
```

Elimina un cliente por su ID. Primero verifica si el cliente **existe**, y si no, lanza una excepción. Luego, si sí existe, lo elimina de la base de datos usando deleteById().

2. Empezamos con la logica de ProductoServices :

```
ClienteServices.java  ProductoServices.java ×
1 package com.coderhouse.services;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import com.coderhouse.error.AgregarCategoriaException;
9 import com.coderhouse.models.Categoria;
10 import com.coderhouse.models.Producto;
11 import com.coderhouse.repositories.CategoriaRepository;
12 import com.coderhouse.repositories.ProductoRepository;
13
14 import jakarta.transaction.Transactional;
```

- Importaciones necesarias:
- List: para retornar colecciones de productos.
- @Autowired: para la inyección automática de dependencias.
- @Service: marca esta clase como componente de servicio, reconocible por Spring.
- Entidades: Producto, Categoria.
- Repositorios: permiten comunicación con la base de datos.
- AgregarCategoriaException: una excepción personalizada usada para errores específicos.
- @Transactional: asegura consistencia y atomicidad en operaciones que modifican datos.

```
ClienteServices.java  ProductoServices.java ×
1 package com.coderhouse.services;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import com.coderhouse.error.AgregarCategoriaException;
9 import com.coderhouse.models.Categoria;
10 import com.coderhouse.models.Producto;
11 import com.coderhouse.repositories.CategoriaRepository;
12 import com.coderhouse.repositories.ProductoRepository;
13
14 import jakarta.transaction.Transactional;
15
16 @Service
```

Inyeccion de clases:

```
16 @Service
17 public class ProductoServices {
18
19     @Autowired
20     private ProductoRepository productoRepository;
21     @Autowired
22     private CategoriaRepository categoriaRepository;
23 }
```

- Inyeccion del services: Esta clase se declara como **servicio**, lo que significa que es responsable de aplicar las reglas de negocio sobre los datos de productos.
- Inyección de dos repositorios: productoRepository: permite acceder a las operaciones CRUD sobre productos.
- categoriaRepository: se utiliza cuando se necesita relacionar productos con categorías.

Método 1: getAllProductos()

```
public List<Producto> getAllProductos(){
    return productoRepository.findAll();
}
```

- Devuelve una lista de todos los productos registrados.
- Utiliza el método findAll() proporcionado por JpaRepository.

Método 2: getProductoById(Long id)

```
public Producto getProductoById(Long id) {
    return productoRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Producto no encontrado"));
}
```

Busca un producto por su ID. Si no lo encuentra, lanza una excepción indicando que no existe.

Método 3: saveProducto(Producto producto)

```
@Transactional
public Producto saveProducto(Producto producto){
    return productoRepository.save(producto);
}
```

- Guarda un nuevo producto o actualiza uno existente.
- Este método es genérico y se puede usar para ambas operaciones, gracias a que save() detecta si el objeto tiene ID o no.
- La anotación @Transactional asegura que la operación es segura y atómica.

Método 4: createProducto(Producto producto)

```
@Transactional
public Producto createProducto(Producto producto) {
    return productoRepository.save(producto);
}
```

Hace lo mismo que saveProducto(), pero está dedicado explícitamente a la creación de productos.

Se puede mantener por razones semánticas o para separar lógicamente las operaciones.

Método 5: updateProducto(Long id, Producto productoDetails)

```
@Transactional
public Producto updateProducto(Long id, Producto productoDetails){
    Producto producto = productoRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("No se encuentra producto con ID: " + id));
```

Primero, busca el producto por ID. Si no existe, lanza una excepción.

Luego, actualiza los campos uno por uno con los datos nuevos que vengan en productoDetails.

```
    producto.setTitulo(productoDetails.getTitulo());
    producto.setDescripcion(productoDetails.getDescripcion());
    producto.setPrecio(productoDetails.getPrecio());
    producto.setStock(productoDetails.getStock());
    producto.setClientes(productoDetails.getClientes());
    producto.setCategoria(productoDetails.getCategoria());
```

Cada uno de los atributos del producto es sobrescrito con los nuevos valores.

Esto incluye la relación con clientes y categoria, lo cual es importante si un producto puede tener asociados varios clientes o categorías.

```
    return productoRepository.save(producto);
}
```

Finalmente, guarda el producto ya modificado.

Método 6: deleteProductoById(Long id)

```
public void deleteProductoById(Long id){
    if(productoRepository.existsById(id)){
        productoRepository.deleteById(id);
    } else {
```

```

        throw new IllegalArgumentException("No se encuentra producto con ID: " + id);
    }
}

```

- Elimina un producto por su ID.
- Primero verifica si existe el producto.
- Si existe, lo elimina; si no, lanza una excepción personalizada.
- Esto ayuda a evitar errores silenciosos y mejorar la trazabilidad.

Método 7: addCategoryToProduct(Long productid, Long categoriald)

```

@Transactional
public Producto addCategoryToProduct(Long productid, Long categoriald) throws
AgregarCategoriaException {

```

- Este método asigna una categoría a un producto existente, estableciendo una relación entre ambas entidades.  
Utiliza una excepción personalizada para mejorar el manejo de errores.

```

    Categoria categoria = categoriaRepository.findById(categoriald)
        .orElseThrow(() -> new AgregarCategoriaException("No se encuentra categoria con
ID: " + categoriald));

```

```

    Producto producto = productoRepository.findById(productid)
        .orElseThrow(() -> new AgregarCategoriaException("No se encuentra producto con
ID: " + productid));

```

Verifica que tanto la categoría como el producto existan. Si alguno no existe, lanza una excepción específica.

```

        producto.setCategoria(categoria);

        return productoRepository.save(producto);
    }

```

Establece la categoría en el producto y guarda los cambios en la base de datos.

```

D *ProductoServices.java x
12 public class ProductoServices {
13     @Autowired
14     private ProductoRepository productoRepository;
15     @Autowired
16     private CategoriaRepository categoriaRepository;
17
18     public List<Producto> getAllProductos(){
19         return productoRepository.findAll();
20     }
21
22     public Producto getProductoById(Long id) {
23         return productoRepository.findById(id)
24             .orElseThrow(() -> new IllegalArgumentException("Producto no encontrado"));
25     }
26     @Transactional
27     public Producto saveProducto(Producto producto){
28         return productoRepository.save(producto);
29     }
30     @Transactional
31     public Producto createProducto(Producto producto) {
32         return productoRepository.save(producto);
33     }
34     @Transactional
35     public Producto updateProducto(Long id, Producto productoDetails){
36         Producto producto = productoRepository.findById(id)
37             .orElseThrow(() -> new IllegalArgumentException("No se encuentra producto con ID: " + id));
38
39         producto.setTitulo(productoDetails.getTitulo());
40         producto.setDescripcion(productoDetails.getDescripcion());
41         producto.setPrecio(productoDetails.getPrecio());
42         producto.setStock(productoDetails.getStock());
43         producto.setClientes(productoDetails.getClientes());
44         producto.setCategoria(productoDetails.getCategoria());
45
46         return productoRepository.save(producto);
47     }
48     public void deleteProductoById(Long id){
49         if(productoRepository.existsById(id)){
50             productoRepository.deleteById(id);
51         } else {
52             throw new IllegalArgumentException("No se encuentra producto con ID: " + id);
53         }
54     }
55     @Transactional
56     public Producto addCategoryToProduct(Long productoId, Long categoriaId) throws AgregarCategoriaException {
57         Categoria categoria = categoriaRepository.findById(categoriaId)
58             .orElseThrow(() -> new AgregarCategoriaException("No se encuentra categoria con ID: " + categoriaId));
59         Producto producto = productoRepository.findById(productoId)
60             .orElseThrow(() -> new AgregarCategoriaException("No se encuentra producto con ID: " + productoId));
61         producto.setCategoria(categoria);
62         return productoRepository.save(producto);
63     }
64 }

```

3. Logica para CategoriaServices: Este servicio maneja la lógica relacionada con las categorías de productos.

```
1 package com.coderhouse.services;
2
3 import java.util.List;
12 @Service
13 public class CategoriaServices {
14     @Autowired
15     private CategoriaRepository categoriaRepository;
16
17     public List<Categoria> getAllCategorias(){
18         return categoriaRepository.findAll();
19     }
20
21     public Categoria getCategoriaById(Long id){
22         return categoriaRepository.findById(id)
23             .orElseThrow(() -> new IllegalArgumentException("No se encuentra categoría con"));
24     }
25
26     public boolean existCategoriaById(Long id) {
27         return categoriaRepository.existsById(id);
28     }
29
30     @Transactional
31     public Categoria createCategoria(Categoria categoria) {
32         return categoriaRepository.save(categoria);
33     }
34
35     @Transactional
36     public Categoria updateCategoria(Long id, Categoria categoriaDetails) {
37         Categoria categoria = categoriaRepository.findById(id)
38             .orElseThrow(() -> new IllegalArgumentException("Categoria no encontrada"));
39         categoria.setNombre(categoriaDetails.getNombre());
40
41         return categoriaRepository.save(categoria);
42     }
43
44     public void deleteCategoriaById(Long id) {
45         categoriaRepository.deleteById(id);
46     }
47 }
48
```

### 1. getAllCategorias()

- Obtiene una lista con **todas las categorías** de la base de datos.

### 2. getCategoriaById(id)

- Busca una categoría por su ID.
- Si no existe, lanza una excepción.

### 3. existCategoriaById(id)

- Devuelve true si una categoría con ese ID existe; false si no.

### 4. createCategoria(categoria)

- Guarda una nueva categoría en la base de datos.

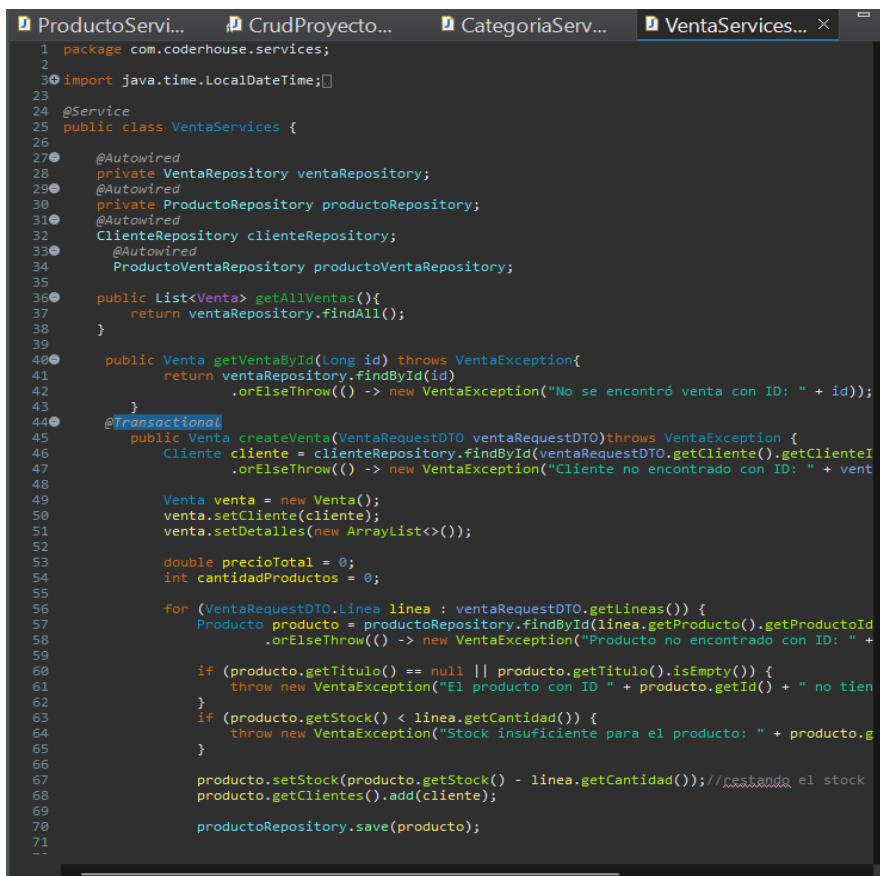
## 5. updateCategoria(id, categoriaDetails)

- Busca una categoría por ID.
- Si existe, **actualiza su nombre** y guarda los cambios.

## 6. deleteCategoriaById(id)

- Elimina una categoría por su ID directamente.

4. Logica Para VentasServices : Este servicio gestiona todo lo relacionado con las **ventas**.



```
1 package com.coderhouse.services;
2
3 import java.time.LocalDateTime;
4
5 @Service
6 public class VentaServices {
7
8     @Autowired
9     private VentaRepository ventaRepository;
10
11     @Autowired
12     private ProductoRepository productoRepository;
13
14     @Autowired
15     private ClienteRepository clienteRepository;
16
17     @Autowired
18     private ProductoVentaRepository productoVentaRepository;
19
20     public List<Venta> getAllVentas(){
21         return ventaRepository.findAll();
22     }
23
24     public Venta getVentaById(Long id) throws VentaException{
25         return ventaRepository.findById(id)
26             .orElseThrow(() -> new VentaException("No se encontró venta con ID: " + id));
27     }
28
29     @Transactional
30     public Venta createVenta(VentaRequestDTO ventaRequestDTO) throws VentaException {
31         Cliente cliente = clienteRepository.findById(ventaRequestDTO.getCliente().getId())
32             .orElseThrow(() -> new VentaException("Cliente no encontrado con ID: " + ventaRequestDTO.getCliente().getId()));
33
34         Venta venta = new Venta();
35         venta.setCliente(cliente);
36         venta.setDetalles(new ArrayList<>());
37
38         double precioTotal = 0;
39         int cantidadProductos = 0;
40
41         for (VentaRequestDTO.Linea linea : ventaRequestDTO.getLineas()) {
42             Producto producto = productoRepository.findById(linea.getProducto().getId())
43                 .orElseThrow(() -> new VentaException("Producto no encontrado con ID: " + linea.getProducto().getId()));
44
45             if (producto.getTitulo() == null || producto.getTitulo().isEmpty()) {
46                 throw new VentaException("El producto con ID " + producto.getId() + " no tiene título");
47             }
48             if (producto.getStock() < linea.getCantidad()) {
49                 throw new VentaException("Stock insuficiente para el producto: " + producto.getId());
50             }
51
52             producto.setStock(producto.getStock() - linea.getCantidad()); // restamos el stock
53             producto.getClientes().add(cliente);
54
55             productoRepository.save(producto);
56         }
57     }
58 }
```

```
Producto Servi...  Crud Proyecto...  Categoria Servi...  Venta Services... x
69
70     productoRepository.save(producto);
71
72
73     ProductoVenta productoVenta = new ProductoVenta();
74     productoVenta.setTitulo(producto.getTitulo());
75     productoVenta.setDescripcion(producto.getDescripcion());
76     productoVenta.setPrecio(producto.getPrecio());
77     productoVenta.setStock(producto.getStock());
78     productoVenta.setCategoria(producto.getCategoria());
79
80     productoVentaRepository.save(productoVenta);
81
82     DetalleVenta detalle = new DetalleVenta();
83     detalle.setVenta(venta);
84     detalle.setProducto(productoVenta);
85     detalle.setCantidad(linea.getCantidad());
86     detalle.setPrecioUnitario(producto.getPrecio());
87
88     venta.getDetalles().add(detalle);
89
90     precioTotal += producto.getPrecio() * linea.getCantidad();
91     cantidadProductos += linea.getCantidad();
92 }
93
94 venta.setPrecioTotal(precioTotal);
95 venta.setCantidadProductos(cantidadProductos);
96 venta.setFecha(LocalDate.now());
97
98 //lógica de puntos por monto de compra
99 int puntosGanados;
100 if (precioTotal > 1500) {
101     puntosGanados = 15;
102 } else if (precioTotal >= 800) {
103     puntosGanados = 10;
104 } else {
105     puntosGanados = 5;
106 }
107
108 //Sumar los puntos al cliente
109 cliente.setPuntosTotales(cliente.getPuntosTotales() + puntosGanados);
110 clienteRepository.save(cliente); // guardar los nuevos puntos en BD
111
112 // Asocio al cliente actualizado antes de guardar la venta
113 venta.setCliente(clienteRepository.findById(cliente.getId()).orElseThrow());
114
115 return ventaRepository.save(venta);
116
117 }
118
119
120
121 @Transactional
122
```

Servers Terminal Console x Data S... Proper... Proble... Git Sta...  
crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\

```
120
121 @Transactional
122 public Venta updateVenta(Long id, Venta ventaDetails) {
123     Venta venta = ventaRepository.findById(id)
124         .orElseThrow(() -> new IllegalArgumentException("Venta no encontrada con su ID: " +
125             id));
126     venta.setCliente(ventaDetails.getCliente());
127     return ventaRepository.save(venta);
128 }
129
130 public void deleteVenta(Long id) {
131     if (ventaRepository.existsById(id)) {
132         ventaRepository.deleteById(id);
133     } else {
134         throw new IllegalArgumentException("Venta no encontrado CON SU ID" + id);
135     }
136 }
137 }
```

### 1. getAllVentas()

- Devuelve todas las ventas registradas.

### 2. getVentaById(id)

- Busca una venta por ID.
- Si no existe, lanza una excepción personalizada.

### 3. createVenta(ventaRequestDTO)

#### *Paso 1: Buscar cliente*

- Se verifica que el cliente existe por su ID.

#### *Paso 2: Procesar cada producto del carrito*

Por cada línea del pedido:

- Se busca el producto por su ID.
- Se valida que tenga título y suficiente stock.
- Se **descuenta el stock**.
- Se **guarda una copia del producto** en ProductoVenta (histórico).
- Se crea un DetalleVenta con cantidad y precio.

#### *Paso 3: Calcular totales*

- Suma el precio total y la cantidad total de productos.

#### *Paso 4: Calcular puntos*

- Si gastó:
  - Más de 1500 → gana 15 puntos.
  - Entre 800 y 1500 → 10 puntos.
  - Menos de 800 → 5 puntos.

#### *Paso 5: Guardar puntos al cliente*

- Se actualizan los puntos del cliente.

#### *Paso 6: Guardar la venta*

- Se guarda la venta con cliente, fecha, total y detalles.

#### 4. updateVenta(id, ventaDetails)

- Actualiza el cliente de una venta ya registrada.

#### 5. deleteVenta(id)

- Elimina una venta por ID si existe.

4. **Controladores:** Manejo de peticiones REST, expone los endpoints REST para que el frontend pueda consumirlos mediante @RestController.

→ Cliente Controller: El ClienteController es una clase que forma parte de la capa de presentación (controladores) de una API REST desarrollada con Spring Boot. Su principal objetivo es **gestionar las operaciones CRUD (Crear, Leer, Actualizar y Eliminar)** para la entidad Cliente. Esta clase se encuentra anotada con @RestController y expone rutas a través de la anotación @RequestMapping("/api/clientes"), lo que permite consumir sus funcionalidades mediante peticiones HTTP.

### 1. Funcionalidades implementadas:

#### 2. GET /api/clientes

Lista todos los clientes almacenados en la base de datos.

Utiliza el servicio clienteServices.getAllClients() y devuelve una lista en formato JSON. Maneja errores como IllegalArgumentException y errores generales del servidor.

#### 3. GET /api/clientes/{id}

Devuelve un cliente específico por su ID.

En caso de que el cliente no exista, responde con 404 Not Found.

#### 4. POST /api/clientes

Permite crear un nuevo cliente.

El cliente es recibido mediante el cuerpo de la solicitud (@RequestBody) y se guarda usando clienteServices.newCliente(). Si ocurre un error de validación o algún fallo interno, se devuelven los códigos HTTP correspondientes.

#### 5. PUT /api/clientes/{id}

Actualiza los datos de un cliente existente.

Requiere el ID del cliente a modificar y un objeto Cliente con los nuevos datos.

#### 6. DELETE /api/clientes/{id}

Elimina un cliente por su ID.

Si el cliente tiene ventas asociadas, se lanza una excepción de tipo

DataIntegrityViolationException y se devuelve un mensaje de conflicto (409 Conflict), indicando que no se puede eliminar.

```
ClienteController.java x
1 package com.coderhouse.controllers;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.dao.DataIntegrityViolationException;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.HttpStatusCode;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.web.ErrorResponse;
11 import org.springframework.web.bind.annotation.DeleteMapping;
12 import org.springframework.web.bind.annotation.GetMapping;
13 import org.springframework.web.bind.annotation.PathVariable;
14 import org.springframework.web.bind.annotation.PostMapping;
15 import org.springframework.web.bind.annotation.PutMapping;
16 import org.springframework.web.bind.annotation.RequestBody;
17 import org.springframework.web.bind.annotation.RequestMapping;
18 import org.springframework.web.bind.annotation.RestController;
19
20
21 import com.coderhouse.models.Cliente;
22 import com.coderhouse.models.Venta;
23 import com.coderhouse.services.ClienteServices;
24
25 import io.swagger.v3.oas.annotations.Operation;
26 import io.swagger.v3.oas.annotations.media.Content;
27 import io.swagger.v3.oas.annotations.media.Schema;
28 import io.swagger.v3.oas.annotations.responses.ApiResponse;
29 import io.swagger.v3.oas.annotations.responses.ApiResponses;
30 import io.swagger.v3.oas.annotations.tags.Tag;
31
32 @RestController
33 @RequestMapping("/api/clientes")
34 @Tag(name = "Cliente", description = "Gestion O Manejo de Clientes")
35 public class ClienteController {
36
37
38     @Autowired
39     private ClienteServices clienteServices;
40
41     @Operation(summary = "Listar Clientes")
42     @ApiResponses(value = {
43         @ApiResponse(responseCode = "200", description = "Listado de Clientes Correctamente",
44             content = {
45                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
46             }
47         ),
48         @ApiResponse(responseCode = "404", description = "Error al listar los clientes",
49             content = @Content ),
50         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
51             content = @Content (mediaType = "aplication/json",
52                 schema = @Schema(implementation = ErrorResponse.class)))
53     })
54     @GetMapping
55     public ResponseEntity <List<Cliente>> getAllClientes(){
56         try {
57
58         }
59     }
60 }
```

```

ClienteController.java ×
52     })
53     @GetMapping
54     public ResponseEntity <List<Cliente>> getAllClientes(){
55         try {
56             List<Cliente> clientes = clienteServices.getAllClientes();
57             return ResponseEntity.ok(clientes);
58         } catch (IllegalArgumentException e){
59             return ResponseEntity.notFound().build();
60         } catch (Exception e){
61             e.printStackTrace();
62             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
63         }
64     }
65
66
67     @Operation(summary = "Listar Clientes por su ID")
68     @ApiResponse(value = {
69         @ApiResponse(responseCode = "200", description = "Clientes listados Correctamente",
70             content = {
71                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
72             }
73         ),
74         @ApiResponse(responseCode = "404", description = "Error al listar los Clientes",
75             content = @Content ),
76         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
77             content = @Content (mediaType = "application/json",
78                 schema = @Schema(implementation = ErrorResponse.class)))
79     })
80     @GetMapping("/{id}")
81     public ResponseEntity<Cliente> getClienteById(@PathVariable Long id){
82         try {
83             Cliente cliente = clienteServices.getClienteById(id);
84             return ResponseEntity.ok(cliente);
85         } catch (IllegalArgumentException e){
86             return ResponseEntity.notFound().build();
87         } catch (Exception e) {
88             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
89         }
90     }
91     @Operation(summary = "Creacion de un Cliente")
92     @ApiResponse(value = {
93         @ApiResponse(responseCode = "201", description = "Cliente Creado Correctamente",
94             content = {
95                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
96             }
97         ),
98         @ApiResponse(responseCode = "404", description = "Error al crear al cliente",
99             content = @Content ),
100         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
101             content = @Content (mediaType = "application/json",
102                 schema = @Schema(implementation = ErrorResponse.class)))
103     })
104     @PostMapping
105     public ResponseEntity<Cliente> newCliente(@RequestBody Cliente cliente){
106         try {

```

```
ClienteController.java x
100         schema = @Schema(implementation = ErrorResponse.class)))
101     })
102     @PostMapping
103     public ResponseEntity<Cliente> newCliente(@RequestBody Cliente cliente){
104         try {
105             Cliente clienteNuevo = clienteServices.newCliente(cliente);
106             return ResponseEntity.ok(clienteNuevo);
107         } catch (IllegalArgumentException e) {
108             return ResponseEntity.notFound().build();
109         } catch (Exception e) {
110             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
111         }
112     }
113     @Operation(summary = "Actualizacion de un Cliente por su ID")
114     @ApiResponse(value = {
115         @ApiResponse(responseCode = "200", description = "Cliente actualizado Correctamente",
116             content = {
117                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
118             }),
119         @ApiResponse(responseCode = "404", description = "Error al actualizar un cliente",
120             content = @Content ),
121         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
122             content = @Content (mediaType = "application/json",
123                 schema = @Schema(implementation = ErrorResponse.class)))
124     })
125     @PutMapping("/{id}")
126     public ResponseEntity<Cliente> updateClienteById(@PathVariable Long id, @RequestBody Cliente clienteModificado) {
127         try {
128             Cliente updateCliente = clienteServices.updateClienteById(id, clienteModificado);
129             return ResponseEntity.ok(updateCliente);
130         } catch (IllegalArgumentException e) {
131             return ResponseEntity.notFound().build();
132         } catch (Exception e) {
133             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
134         }
135     }
136
137
138     @Operation(summary = "Eliminar un Cliente por SU ID")
139     @ApiResponse(value = {
140         @ApiResponse(responseCode = "204", description = "Cliente eliminado Correctamente por su ID",
141             content = {
142                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
143             }),
144         @ApiResponse(responseCode = "404", description = "Error AL eliminar un Cliente",
145             content = @Content ),
146         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
147             content = @Content (mediaType = "application/json",
148                 schema = @Schema(implementation = ErrorResponse.class)))
149     })
150     @DeleteMapping("/{id}")
151     public ResponseEntity<?> deleteClienteById(@PathVariable Long id) {
152         try {
153             clienteServices.deleteClienteById(id);
154             return ResponseEntity.ok().body("Cliente eliminado correctamente...! "); //204
155         } catch (DataIntegrityViolationException e) {
156             return ResponseEntity.status(HttpStatus.CONFLICT).
157                 body("No se puede eliminar el cliente por que tiene ventas registradas...!");
158         } catch (IllegalArgumentException e) {
159             return ResponseEntity.notFound().build(); //400
160         } catch (Exception e) {
161             e.printStackTrace();
162             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
163                 .body("Error Interno: " + e.getMessage()); //500
164         }
165     }
166
167 }
```

Servers Terminal Console Data Source Explorer Properties Problems Git Staging

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 2024)

```
150     @DeleteMapping("/{id}")
151     public ResponseEntity<?> deleteClienteById(@PathVariable Long id) {
152         try {
153             clienteServices.deleteClienteById(id);
154             return ResponseEntity.ok().body("Cliente eliminado correctamente...! "); //204
155         } catch (DataIntegrityViolationException e) {
156             return ResponseEntity.status(HttpStatus.CONFLICT).
157                 body("No se puede eliminar el cliente por que tiene ventas registradas...!");
158         } catch (IllegalArgumentException e) {
159             return ResponseEntity.notFound().build(); //400
160         } catch (Exception e) {
161             e.printStackTrace();
162             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
163                 .body("Error Interno: " + e.getMessage()); //500
164         }
165     }
166
167 }
```

Servers Terminal Console Data Source Explorer Properties Problems Git Staging

NOTA: La documentación está en la misma clase con La tecnología Swagger que sirve para la documentación de API REST.

→ @ProductoController:

```
ClienteController.java ProductoController.java x
1 package com.coderhouse.controllers;
2
3 import java.util.List;
4
5 @RestController
6 @RequestMapping("/api/productos")
7 @Tag(name = "Productos", description = "Manejador De Productos")
8 public class ProductoController {
9
10     @Autowired
11     private ProductoServices productoService;
12     @SuppressWarnings("unused")
13     @Autowired
14     private CategoriaServices categoriaService;
15
16     @Operation(summary = "Listar Productos")
17     @ApiResponses(value = {
18         @ApiResponse(responseCode = "200", description = "Producto listados Correctamente",
19             content = {
20                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
21             }
22         ),
23         @ApiResponse(responseCode = "404", description = "Error al listar los producto",
24             content = @Content ),
25         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
26             content = @Content (mediaType = "application/json",
27                 schema = @Schema(implementation = ErrorResponse.class)))
28     })
29     @GetMapping
30     public ResponseEntity<List<Producto>> getAllProductos(){
31         try {
32             List<Producto> productos = productoService.getAllProductos();
33             return ResponseEntity.ok(productos);
34         } catch (IllegalArgumentException e) {
35             return ResponseEntity.notFound().build();
36         } catch (Exception e) {
37             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
38         }
39     }
40
41     @Operation(summary = "Listar de un producto por SU ID")
42     @ApiResponses(value = {
43         @ApiResponse(responseCode = "200", description = "Producto por su ID listado Correctamente",
44             content = {
45                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
46             }
47         ),
48         @ApiResponse(responseCode = "404", description = "Error al listar producto",
49             content = @Content ),
50         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
51             content = @Content (mediaType = "application/json",
52                 schema = @Schema(implementation = ErrorResponse.class)))
53     })
54     @GetMapping("/{id}")
55     public ResponseEntity<Producto> getProductoById(@PathVariable Long id){
56
57     }
```

```
ClienteController.java ProductoController.java ×
79 @GetMapping("/{id}")
80 public ResponseEntity<Producto> getProductoById(@PathVariable Long id){
81     try {
82         Producto producto = productoService.getProductoById(id);
83         return ResponseEntity.ok(producto);
84     } catch (IllegalArgumentException e) {
85         return ResponseEntity.notFound().build();
86     } catch (Exception e) {
87         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
88     }
89 }
90
91 @Operation(summary = "Creacion de un producto")
92 @ApiResponses(value = {
93     @ApiResponse(responseCode = "201", description = "Producto Creado Correctamente",
94         content = {
95             @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
96         }),
97     @ApiResponse(responseCode = "404", description = "Error al crear el producto",
98         content = @Content ),
99     @ApiResponse(responseCode = "500", description = "Error interno del servidor",
100         content = @Content (mediaType = "application/json",
101             schema = @Schema(implementation = ErrorResponse.class)))
102 })
103 @PostMapping
104 public ResponseEntity<Producto> createProducto(@RequestBody Producto producto) {
105     try {
106         Producto createProducto = productoService.saveProducto(producto);
107         return ResponseEntity.ok(createProducto);
108     } catch (IllegalArgumentException e) {
109         e.printStackTrace();
110         return ResponseEntity.notFound().build();
111     } catch (Exception e) {
112         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
113     }
114 }
115
116 @Operation(summary = "Actualizacion de producto por ID")
117 @ApiResponses(value = {
118     @ApiResponse(responseCode = "200", description = "Producto corregido Correctamente",
119         content = {
120             @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
121         }),
122     @ApiResponse(responseCode = "404", description = "Error al actualizar el producto",
123         content = @Content ),
124     @ApiResponse(responseCode = "500", description = "Error interno del servidor",
125         content = @Content (mediaType = "application/json",
126             schema = @Schema(implementation = ErrorResponse.class)))
127 })
128 @PutMapping("/{id}")
129 public ResponseEntity<Producto> updateProducto(@PathVariable Long id, @RequestBody Producto productoDetails){
130     try {
131         ...
132     }
133 }
```

Servers Terminal Console × Data Source Explorer Properties Problems Git Staging

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20

```
ClienteController.java ProductoController.java x
127    })
128    @PostMapping("/{id}")
129    public ResponseEntity<Producto> updateProducto(@PathVariable Long id,@RequestBody Producto productoDetails){
130        try {
131
132            Producto updateProducto = productoService.updateProducto(id, productoDetails);
133            return ResponseEntity.ok(updateProducto);
134        } catch (IllegalArgumentException e) {
135            return ResponseEntity.notFound().build();
136        } catch (Exception e) {
137            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
138        }
139    }
140
141    @Operation(summary = "Eliminar un producto por SU ID")
142    @ApiResponses(value = {
143        @ApiResponse(responseCode = "204", description = "Producto eliminado Correctamente por su ID",
144            content = {
145                @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
146            }),
147        @ApiResponse(responseCode = "404", description = "Error AL eliminar producto",
148            content = @Content ),
149        @ApiResponse(responseCode = "500", description = "Error interno del servidor",
150            content = @Content (mediaType = "application/json",
151                schema = @Schema(implementation = ErrorResponse.class)))
152    })
153    @DeleteMapping("/{id}")
154    public ResponseEntity<Void> deleteProductoById(@PathVariable Long id){
155        try {
156            productoService.deleteProductoById(id);
157            return ResponseEntity.noContent().build();
158        } catch (IllegalArgumentException e) {
159            return ResponseEntity.notFound().build();
160        } catch (Exception e) {
161            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
162        }
163    }
164    @Operation(summary = "Asignar una categoria A Un producto")
165    @ApiResponses(value = {
166        @ApiResponse(responseCode = "200", description = "Categoria asignada Correctamente",
167            content = {
168                @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
169            }),
170        @ApiResponse(responseCode = "404", description = "Error al asignar la categoria",
171            content = @Content(mediaType = "application/json",
172                schema = @Schema(implementation = ErrorResponse.class)) ),
173        @ApiResponse(responseCode = "500", description = "Error interno del servidor",
174            content = @Content (mediaType = "application/json",
175                schema = @Schema(implementation = ErrorResponse.class)))
176    })
177    @PostMapping("/asignar-categoria")
178    public ResponseEntity<Producto> addCategoryToProduct(@RequestBody VentaAgregarProductosDTO agregarCatetoriaProductoDTC
179    ---
180    try {
181
182        Producto updateProducto = productoService.addCategoryToProduct(
183            agregarCatetoriaProductoDTO.getProductoId(),
184            agregarCatetoriaProductoDTO.getCategoriaId()
185        );
186        return ResponseEntity.ok(updateProducto);
187    } catch (AgregarCategoriaException e) {
188        return ResponseEntity.notFound().build();
189    } catch (Exception e) {
190        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
191    }
192    }
```

El ProductoController administra todas las operaciones relacionadas con productos. Utiliza una estructura RESTful para manejar las siguientes acciones desde la ruta /api/productos:

- Consultar productos disponibles.
- Buscar productos por ID.
- Registrar nuevos productos.
- Editar productos existentes.
- Eliminar productos, teniendo en cuenta posibles restricciones como asociaciones con ventas.

Al igual que los otros controladores, incluye documentación con Swagger para facilitar pruebas y validación desde herramientas web.

→ @CategoriaController:

El CategoriaController sigue la misma estructura y principios que ClienteController, pero se encarga de gestionar las operaciones CRUD relacionadas con la entidad Categoria. Expone rutas como /api/categorias y permite:

- Listar todas las categorías.
- Obtener una categoría por ID.
- Crear una nueva categoría.
- Actualizar una categoría existente.
- Eliminar una categoría por su ID.

También está documentado con Swagger, lo cual facilita la exploración de los endpoints y el entendimiento del flujo de datos.

```
ClienteController.java ProductoController.java CategoriaController.java ×
1 package com.coderhouse.controllers;
2
3 import java.util.List;
4
29
30 @RestController
31 @RequestMapping("/api/categoria")
32 @Tag(name = "Categoria", description = "Categoria Manejador Sistemática")
33 public class CategoriaController {
34     @Autowired
35     private CategoriaServices categoriaService;
36
37     @Operation(summary = "Listar Categorías")
38     @ApiResponses(value = {
39         @ApiResponse(responseCode = "200", description = "Listar Categorías Correctamente",
40             content = {
41                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
42             }
43         ),
44         @ApiResponse(responseCode = "404", description = "Error al listar las categorías",
45             content = @Content ),
46         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
47             content = @Content (mediaType = "application/json",
48                 schema = @Schema(implementation = ErrorResponse.class)))
49     })
50     @GetMapping
51     public List<Categoria> getAllCategorias() {
52         return categoriaService.getAllCategorias();
53     }
54     @Operation(summary = "Listar Categorías por su ID")
55     @ApiResponses(value = {
56         @ApiResponse(responseCode = "200", description = "Categorías listadas Correctamente",
57             content = {
58                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
59             }
60         ),
61         @ApiResponse(responseCode = "404", description = "Error al listar las categorías",
62             content = @Content ),
63         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
64             content = @Content (mediaType = "application/json",
65                 schema = @Schema(implementation = ErrorResponse.class)))
66     })
67     @GetMapping("/{id}")
68     public ResponseEntity<Categoria> getCategoriaById(@PathVariable Long id) {
69         try {
70             Categoria categoria = categoriaService.getCategoriaById(id);
71             return ResponseEntity.ok(categoria);
72         } catch (IllegalArgumentException e) {
73             return ResponseEntity.notFound().build();
74         } catch (Exception e) {
75             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
76         }
77     }
78     @Operation(summary = "Creación de una Categoría")
79     @ApiResponses(value = {
80         @ApiResponse(responseCode = "201", description = "Categoría Creada Correctamente",
81             content = {
82                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
83             }
84         ),
85         @ApiResponse(responseCode = "404", description = "Error al crear la categoría",
86             content = @Content ),
87         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
88             content = @Content (mediaType = "application/json",
89                 schema = @Schema(implementation = ErrorResponse.class)))
90     })
91     @PostMapping
92     public ResponseEntity<Categoria> createCategoria(@RequestBody Categoria categoria) {
93         try {
94             Categoria nuevaCategoria = categoriaService.createCategoria(categoria);
95             return ResponseEntity.status(HttpStatus.CREATED).body(nuevaCategoria);
96         } catch (IllegalArgumentException e) {
97             return ResponseEntity.badRequest().build();
98         } catch (Exception e) {
99             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
100         }
101     }
102 }
```

Servers Terminal Console × Data Source Explorer Properties Problems Git Staging

crud proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20

```
ClienteController.java ProductoController.java CategoriaController.java x
74     }
75     }
76     @Operation(summary = "Creacion de una Categoria")
77     @ApiResponse(value = {
78         @ApiResponse(responseCode = "201", description = "Categoria Creada Correctamente",
79             content = {
80                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
81             }),
82         @ApiResponse(responseCode = "404", description = "Error al crear la categoria",
83             content = @Content ),
84         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
85             content = @Content (mediaType = "application/json",
86                 schema = @Schema(implementation = ErrorResponse.class)))
87     })
88     @PostMapping
89     public ResponseEntity<Categoria> createCategoria(@RequestBody Categoria categoria) {
90         try {
91             Categoria createdCategoria = categoriaService.createCategoria(categoria);
92             return ResponseEntity.status(HttpStatus.CREATED).body(createdCategoria);
93         } catch (IllegalArgumentException e) {
94             return ResponseEntity.notFound().build();
95         } catch (Exception e) {
96             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
97         }
98     }
99
100     @Operation(summary = "Actualizacion de categoria por su ID")
101     @ApiResponse(value = {
102         @ApiResponse(responseCode = "200", description = "Producto actualizada Correctamente",
103             content = {
104                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
105             }),
106         @ApiResponse(responseCode = "404", description = "Error al actualizar la categoria",
107             content = @Content ),
108         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
109             content = @Content (mediaType = "application/json",
110                 schema = @Schema(implementation = ErrorResponse.class)))
111     })
112     @PutMapping("/{id}")
113     public ResponseEntity<Categoria> updateCategoria(@PathVariable Long id, @RequestBody Categoria categoriaDetails) {
114         try {
115             Categoria updateCategoria = categoriaService.updateCategoria(id, categoriaDetails);
116             return ResponseEntity.status(HttpStatus.CREATED).body(updateCategoria);
117         } catch (IllegalArgumentException e) {
118             return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
119         } catch (Exception e) {
120             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
121         }
122     }
123
124
125     @Operation(summary = "Eliminar una Categoria por SU ID")
126     @ApiResponse(value = {
127         @ApiResponse(responseCode = "200", description = "Categoria eliminada Correctamente",
128             content = {
129                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
130             }),
131         @ApiResponse(responseCode = "404", description = "Error al eliminar la categoria",
132             content = @Content ),
133         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
134             content = @Content (mediaType = "application/json",
135                 schema = @Schema(implementation = ErrorResponse.class)))
136     })
137     @DeleteMapping("/{id}")
138     public ResponseEntity<Categoria> deleteCategoria(@PathVariable Long id) {
139         try {
140             categoriaService.deleteCategoria(id);
141             return ResponseEntity.noContent().build();
142         } catch (IllegalArgumentException e) {
143             return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
144         } catch (Exception e) {
145             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
146         }
147     }
148 }
```

Servers Terminal Console x Data Source Explorer Properties Problems Git Staging

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 2024)

```

125 @Operation(summary = "Eliminar una Categoría por SU ID")
126 @ApiResponses(value = {
127     @ApiResponse(responseCode = "204", description = "Categoría eliminado Correctamente por su ID",
128         content = {
129             @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
130         }),
131     @ApiResponse(responseCode = "404", description = "Error AL eliminar Categoría",
132         content = @Content ),
133     @ApiResponse(responseCode = "500", description = "Error interno del servidor",
134         content = @Content(mediaType = "application/json",
135             schema = @Schema(implementation = ErrorResponse.class)))
136 })
137 @DeleteMapping("/{id}")
138 public ResponseEntity<?> deleteCategoriaById(@PathVariable Long id) {
139     try {
140         categoriaService.deleteCategoriaById(id);
141         return ResponseEntity.ok("Categoría eliminada correctamente...!");
142     } catch (DataIntegrityViolationException e) {
143         return ResponseEntity.status(HttpStatus.CONFLICT).
144             body("No se puede eliminar la categoría por que esta asignada a uno o mas producto");
145         //if(categoriaService.existCategoriaById(id)) {
146         //    categoriaService.deleteCategoriaById(id);
147         //    return ResponseEntity.status(HttpStatus.NO_CONTENT).build();
148         //} else {
149         //    return ResponseEntity.notFound().build();
150         //}
151     } catch (Exception e) {
152         e.printStackTrace();
153         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error interno: " + e.getMessage());
154     }
155 }
156 }

```

Servers Terminal Console Data Source Explorer Properties Problems Git Staging  
 crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 202

→ @VentaController:

```
ClienteController.ja... ProductoController.j... CategoriaController... VentaController.java x
1 package com.coderhouse.controllers;
2
3 import java.util.List;
4
5 @Tag(name = "Venta", description = "Manejo de Ventas")
6 @RestController
7 @RequestMapping("/api/ventas")
8 public class VentaController {
9
10     @Autowired
11     private VentaServices ventaService;
12
13     @Operation(summary = "Obtener Lista de Ventas")
14     @ApiResponses(value = {
15         @ApiResponse(responseCode = "200", description = "Lista de ventas obtenida correctamente",
16             content = {
17                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
18             })
19     },
20     @ApiResponse(responseCode = "404", description = "Error al intentar obtener las ventas",
21         content = @Content ),
22     @ApiResponse(responseCode = "500", description = "Error interno del servidor",
23         content = @Content )
24     })
25     @GetMapping
26     public ResponseEntity<List<Venta>> getAllVentas() {
27         try {
28             List<Venta> ventas = ventaService.getAllVentas();
29             return ResponseEntity.ok(ventas);
30         } catch (Exception e) {
31             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
32         }
33     }
34
35     @Operation(summary = "Obtener una Venta por su ID")
36     @ApiResponses(value = {
37         @ApiResponse(responseCode = "205", description = "Venta Obtenida Correctamente",
38             content = {
39                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
40             })
41     },
42     @ApiResponse(responseCode = "404", description = "Error al obtener la venta",
43         content = @Content ),
44     @ApiResponse(responseCode = "500", description = "Error interno del servidor",
45         content = @Content )
46     })
47     @GetMapping("/{id}")
48     public ResponseEntity<Venta> getVentaById(@PathVariable Long id) {
49         try {
50             Venta venta = ventaService.getVentaById(id);
51             return ResponseEntity.ok(venta);
52         } catch (IllegalArgumentException e) {
53             return ResponseEntity.notFound().build();
54         } catch (Exception e) {
55             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
56         }
57     }
58 }
```

Servers Terminal Console Data Source Explorer Properties Problems Git Staging

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 202

```
ClienteController.ja... ProductoController.j... CategoriaController... VentaController.java x
66         content = @Content )
67     })
68     @GetMapping("/{id}")
69     public ResponseEntity <Venta> getVentaById(@PathVariable Long id) {
70         try {
71             Venta venta = ventaService.getVentaById(id);
72             return ResponseEntity.ok(venta);
73         } catch (IllegalArgumentException e) {
74             return ResponseEntity.notFound().build();
75         } catch (Exception e) {
76             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
77         }
78     }
79
80     @Operation(summary = "Creacion de venta correctamente")
81     @ApiResponses(value = {
82         @ApiResponse(responseCode = "201", description = "Venta Creada Correctamente",
83             content = {
84                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
85             }
86         ),
87         @ApiResponse(responseCode = "404", description = "Error al crear la venta",
88             content = @Content ),
89         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
90             content = @Content )
91     })
92     @PostMapping
93     public ResponseEntity <?> createVenta(@RequestBody VentaRequestDTO ventaRequestDTO) {
94         try {
95             Venta venta = ventaService.createVenta(ventaRequestDTO);
96             return ResponseEntity.ok(venta); // ok se crea la venta
97         } catch (IllegalArgumentException e) {
98             return ResponseEntity.notFound().build(); // 404
99         } catch (Exception e) {
100             e.printStackTrace();
101             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error interno: " + e.getMessage());
102         }
103     }
104
105     @Operation(summary = "Eliminar UNA Venta por su ID")
106     @ApiResponses(value = {
107         @ApiResponse(responseCode = "204", description = "Venta Eliminada Correctamente",
108             content = {
109                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
110             }
111         ),
112         @ApiResponse(responseCode = "404", description = "Error al eliminar la venta",
113             content = @Content ),
114         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
115             content = @Content )
116     })
117     @DeleteMapping("/{id}")
118     public ResponseEntity<Void> deleteVenta(@PathVariable Long id) {
119         try {
120             ventaService.deleteVenta(id);
121         } catch (Exception e) {
122             e.printStackTrace();
123             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error interno: " + e.getMessage());
124         }
125     }
126 }
```

Servers Terminal Console x Data Source Explorer Properties Problems Git Staging

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 2024)

Writable | Smart Insert | 1...0

```
115     @DeleteMapping("/{id}")
116     public ResponseEntity<Void> deleteVenta(@PathVariable Long id) {
117         try {
118             ventaService.deleteVenta(id);
119             return ResponseEntity.noContent().build();
120         } catch (IllegalArgumentException e) {
121             return ResponseEntity.notFound().build();
122         } catch (Exception e) {
123             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
124         }
125     }
126
127     @Operation(summary = "Actualizacion de venta correctamente por su ID")
128     @ApiResponse(value = {
129         @ApiResponse(responseCode = "200", description = "Venta Actualizada Correctamente",
130             content = {
131                 @Content(mediaType = "application/json", schema = @Schema(implementation = Venta.class))
132             }),
133         @ApiResponse(responseCode = "404", description = "Error al actualizar la venta",
134             content = @Content ),
135         @ApiResponse(responseCode = "500", description = "Error interno del servidor",
136             content = @Content )
137     })
138     @PutMapping("/{id}")
139     public ResponseEntity<Venta> updateVenta(@PathVariable Long id, @RequestBody Venta ventaDetails) {
140         try {
141             Venta updatedVenta = ventaService.updateVenta(id, ventaDetails);
142             return ResponseEntity.ok(updatedVenta);
143         } catch (IllegalArgumentException e) {
144             return ResponseEntity.notFound().build();
145         } catch (Exception e) {
146             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
147         }
148     }
149 }
```

crud\_proyecto - CrudProyectoApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (30 may 20

El VentaController se especializa en el manejo de las operaciones relacionadas con las ventas realizadas por los clientes. Desde la ruta /api/ventas, se permite:

- Listar todas las ventas registradas.
- Consultar detalles de una venta por su ID.
- Registrar una nueva venta (generalmente con detalles de productos y cliente involucrado).
- Actualizar los datos de una venta.
- Eliminar una venta si no existe conflicto de integridad.

Incluye manejo de errores, validaciones y documentación con Swagger, al igual que los demás controladores, lo cual garantiza una estructura estandarizada y mantenible.

## ¿Cómo funcionan los métodos HTTP en Postman y en los controladores Spring Boot?

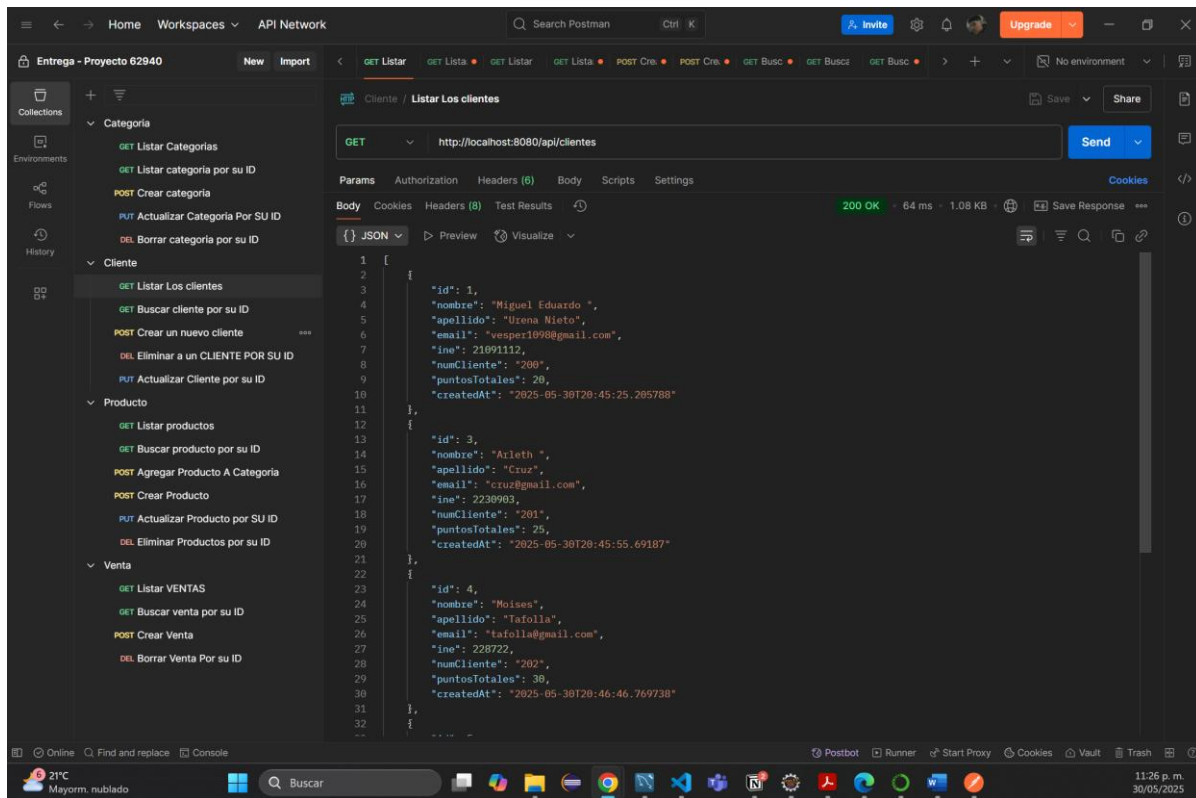
En una API REST, se utilizan distintos métodos HTTP para realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar). Postman es una herramienta que permite enviar fácilmente peticiones HTTP a tu API para probar que funcionan correctamente.

### ◇ @GetMapping → Método GET (Consultar datos)

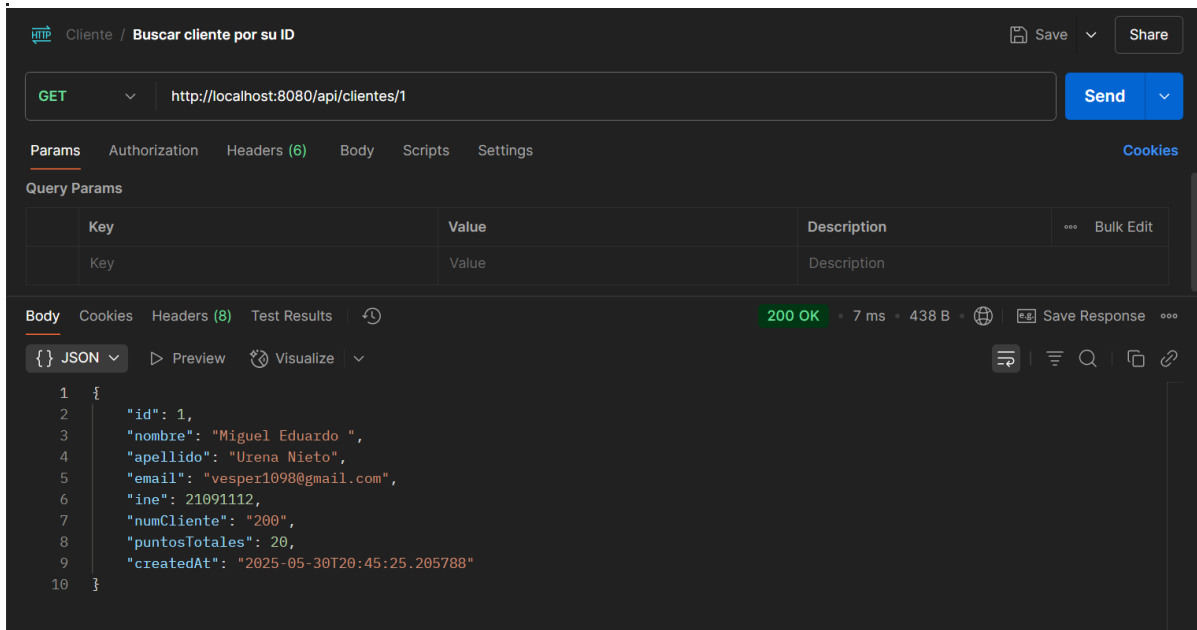
**Postman:**

- En Postman seleccionas el método GET y escribes la URL de tu endpoint (por ejemplo: <http://localhost:8080/api/clientes>).
- Puedes hacer dos tipos de consultas:
  - **Todos los registros:** GET /api/clientes

## LISTADO DE CLIENTES QUE RESPONDE AL METODO GET



- **Por ID:** GET /api/clientes/{id}



### Spring Boot:

- `getAllClientes()` devuelve todos los clientes.
- `getClienteById(Long id)` devuelve un cliente específico usando su ID.

**Respuesta esperada:** Código 200 OK con la lista o el cliente en formato JSON.

### ◇ @PostMapping → Método POST (Crear un nuevo registro)

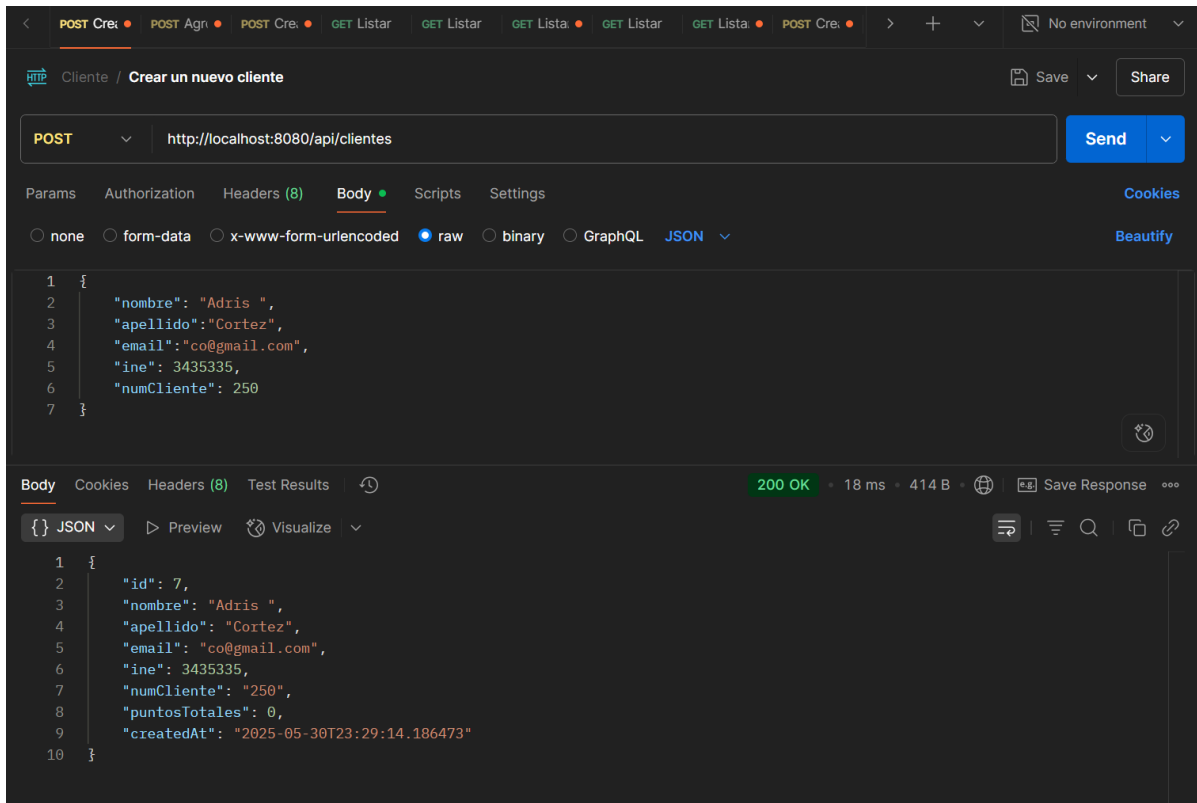
#### Postman:

- En Postman eliges POST y colocas la URL <http://localhost:8080/api/clientes>.
- En el body seleccionas raw → JSON y colocas el objeto a crear:

```

{
  "nombre": "Adris ",
  "apellido": "Cortez",
  "email": "co@gmail.com",
  "ine": 3435335,
  "numCliente": 250
}

```



### Spring Boot:

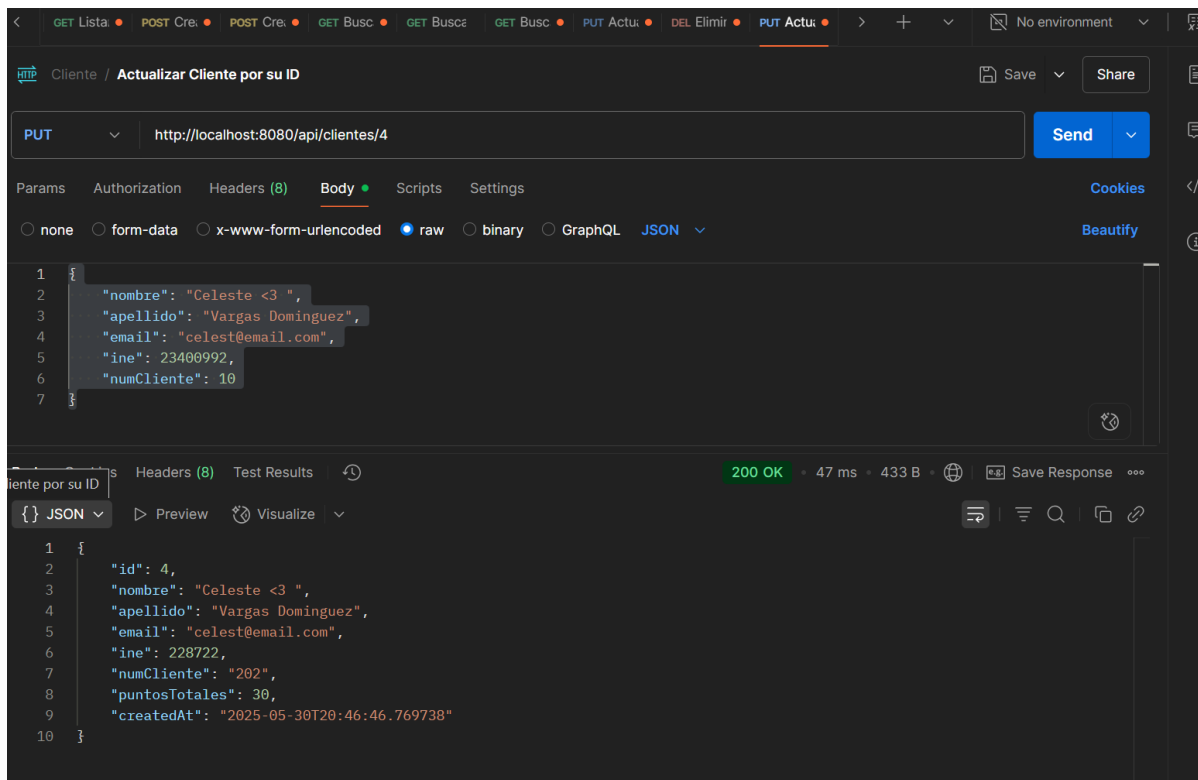
- `newCliente(@RequestBody Cliente cliente)` recibe el JSON del cliente en el cuerpo de la petición y lo guarda.

**Respuesta esperada:** Código 201 Created (aunque tu método actualmente devuelve 200 OK) y el objeto creado.

### ◇ `@PutMapping("/{id}")` → Método PUT (Actualizar un registro)

#### Postman:

- En Postman eliges PUT y colocas una URL como <http://localhost:8080/api/clientes/3> (donde 3 es el ID del cliente).
- En el body envías el nuevo objeto actualizado en formato JSON.



## Spring Boot:

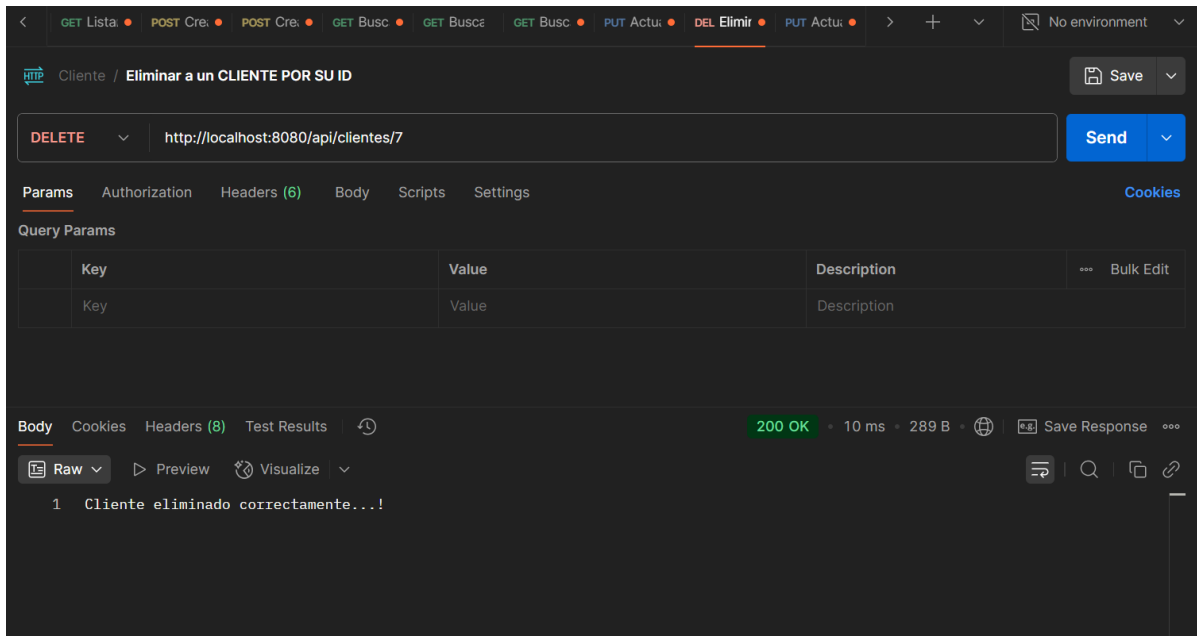
- updateClienteById(Long id, Cliente clienteModificado) busca al cliente por ID y reemplaza sus datos con los nuevos.

**Respuesta esperada:** Código 200 OK y el cliente actualizado. Si no se encuentra, se devuelve 404 Not Found.

◇ **@DeleteMapping("/{id}")** → **Método DELETE (Eliminar un registro)**

## Postman:

- En Postman seleccionas DELETE y colocas la URL con el ID del cliente: `http://localhost:8080/api/clientes/7`.



### Spring Boot:

- `deleteClienteById(Long id)` elimina el cliente si no tiene relaciones con ventas.
- Si tiene ventas, devuelve un 409 Conflict con un mensaje personalizado.
- Si ocurre otra excepción, responde con 500 Internal Server Error.

**Respuesta esperada:** Código 200 OK o 204 No Content si se elimina correctamente, o 409 Conflict si tiene relaciones.

### ClienteController.java

Este controlador gestiona las operaciones CRUD para la entidad Cliente. Se documenta con Swagger para facilitar su visualización e integración. Incluye:

- **GET /api/clientes:** lista todos los clientes.
- **GET /api/clientes/{id}:** obtiene un cliente por su ID.
- **POST /api/clientes:** crea un nuevo cliente recibiendo un objeto JSON.
- **PUT /api/clientes/{id}:** actualiza los datos de un cliente ya existente.
- **DELETE /api/clientes/{id}:** elimina un cliente. Si tiene ventas, no se elimina. Cada método está probado con Postman, enviando peticiones HTTP y observando las respuestas en formato JSON.

### ProductoController.java

Este controlador maneja productos. Permite listar productos, obtenerlos por ID, crearlos, actualizarlos y eliminarlos. También está documentado con Swagger. Las pruebas se hicieron

con Postman utilizando los métodos HTTP GET, POST, PUT y DELETE con JSON en el cuerpo.

### CategoriaController.java

Permite la gestión de categorías. Contiene rutas similares al resto: listar todas, buscar por ID, crear, actualizar y eliminar una categoría. Se documenta con Swagger y se prueba en Postman enviando los datos adecuados en JSON.

### VentaController.java

Controlador enfocado en la entidad Venta. Administra la creación y consulta de ventas, posiblemente incluyendo lógica adicional para registrar detalles de productos vendidos. Cada operación es accesible vía HTTP y fue validada en Postman con los formatos y rutas adecuados.

5. **DTOs:** Se usaron para encapsular y transportar datos entre capas, separando el modelo de dominio de la vista.



```
1 package com.coderhouse.dtos;
2 import lombok.AllArgsConstructor;
3
4 @Data
5 @NoArgsConstructor
6 @AllArgsConstructor
7 public class VentaRequestDTO {
8
9     private ClienteRequest cliente;
10    private List<Linea> lineas;
11
12    @Data
13    @NoArgsConstructor
14    @AllArgsConstructor
15    public static class ClienteRequest {
16        private Long clienteId;
17    }
18
19    @Data
20    @NoArgsConstructor
21    @AllArgsConstructor
22    public static class Linea {
23        private int cantidad;
24        private ProductoRequest producto;
25    }
26
27    @Data
28    @NoArgsConstructor
29    @AllArgsConstructor
30    public static class ProductoRequest {
31        private Long productoId;
32    }
33 }
34
35
36
37
```

6. **Errors:** Manejo de errores a través de clases que se mandan a llamar en situaciones muy específicas

```
AgregarCategoriaException.java x CompraException.java VentaException.java
1 package com.coderhouse.error;
2
3
4 import java.io.Serializable;
12
13 @SuppressWarnings("serial")
14 @Getter
15 @Setter
16 @NoArgsConstructor
17 @ToString
18 @RegisterReflection
19 public class AgregarCategoriaException extends Exception implements Serializable {
20
21     public AgregarCategoriaException(String message) {
22         super(message);
23     }
24 }
```

- Se simplifican en clases para poder atrapar un error mandando a llamar el objeto de esta clase en la lógica funcional de los servicios que a su vez implementa la herramienta "Serializable"

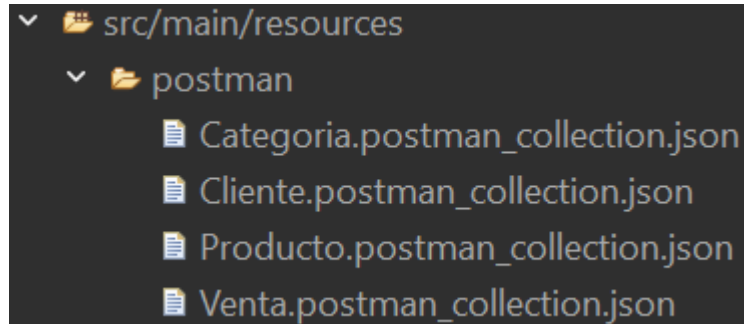
## 7. Config: Despliegue de API En Swagger.

```
configS.java x
1 package com.coderhouse.config;
2 import java.util.List;
12
13 @Configuration
14 public class configS {
15
16     @Bean
17     OpenAPI customOpenAPI() {
18         return new OpenAPI()
19             .info(new Info()
20                 .title("API REST FULL - CRUD DE CLIENTES-PRODUCTOS-VENTAS - CODERHOUSE")
21                 .version("1.0.0")
22                 .description("UN CRUD SENCILLO DE CLIENTE, PRODUCTO Y VENTA")
23                 .contact(new Contact()
24                     .name("Miguel Eduardo Urena Nieto")
25                     .email("vesper1098@gmail.com")
26                     .url("https://coderhouse.com.ar"))
27                 .license(new License()
28                     .name("Licencia")
29                     .url("https://github.com/MigueV10"))
30             )
31             .servers(List.of(new Server()
32                 .url("http://localhost:8080")
33                 .description("Servidor Local")));
34     }
35 }
36
37 }
38
39 }
```

- 8. Carpeta resources: Carpeta secundaria a la del proyecto que guarda las sig configuraciones.

- Postman: Nos permite consumir nuestras APIS a traves de una interfaz muy intuitiva.

Aqui guardaremos las colecciones que creamos para cada actor mostrado con la extensión.  
json



Por ejemplo:

- Nos manda un script en formato JSON, donde hacemos todas las peticiones en postman para cliente, producto, categorias, ventas.

Cliente.postman\_collection.json

```
CrudProyecto...  CategoriaSe...  VentaServic...  Cliente.pos... x 1
1 {
2   "info": {
3     "_postman_id": "50a70ca9-b5c9-4c73-b05f-96ba5063f7ee",
4     "name": "Cliente",
5     "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
6     "_exporter_id": "40991915"
7   },
8   "item": [
9     {
10      "name": "Listar los clientes",
11      "request": {
12        "auth": {
13          "type": "noauth"
14        },
15        "method": "GET",
16        "header": [],
17        "url": {
18          "raw": "http://localhost:8080/api/clientes",
19          "protocol": "http",
20          "host": [
21            "localhost"
22          ],
23          "port": "8080",
24          "path": [
25            "api",
26            "clientes"
27          ]
28        }
29      },
30      "response": []
31    },
32    {
33      "name": "Buscar cliente por su ID",
34      "request": {
35        "auth": {
36          "type": "noauth"
37        },
38        "method": "GET",
39        "header": [],
40        "url": {
41          "raw": "http://localhost:8080/api/clientes/1",
42          "protocol": "http",
43          "host": [
44            "localhost"
45          ],
46          "port": "8080",
47          "path": [
48            "api",
49            "clientes",
50            "1"
51          ]
52        }
53      }
54    }
55  ]
56 }
```

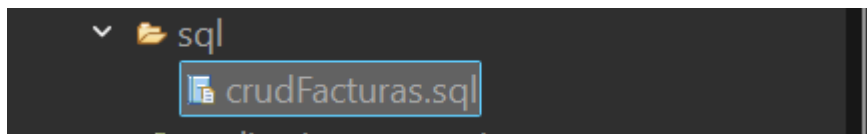
```

54     "response": []
55 },
56 {
57     "name": "Crear un nuevo cliente",
58     "request": {
59         "auth": {
60             "type": "noauth"
61         },
62         "method": "POST",
63         "header": [],
64         "body": {
65             "mode": "raw",
66             "raw": "{\r\n    \"nombre\": \"Brisa\", \r\n    \"apellido\": \"Gamacca\", \r\n"
67             "options": {
68                 "raw": {
69                     "language": "json"
70                 }
71             }
72         },
73         "url": {
74             "raw": "http://localhost:8080/api/clientes",
75             "protocol": "http",
76             "host": [
77                 "localhost"
78             ],
79             "port": "8080",
80             "path": [
81                 "api",
82                 "clientes"
83             ]
84         }
85     },
86     "response": []
87 },
88 {
89     "name": "Eliminar a un CLIENTE POR SU ID",
90     "request": {
91         "auth": {
92             "type": "noauth"
93         },
94         "method": "DELETE",
95         "header": [],
96         "url": {
97             "raw": "http://localhost:8080/api/clientes/1",
98             "protocol": "http",
99             "host": [
100                 "localhost"
101             ],
102             "port": "8080",
103             "path": [
104                 "api",

```

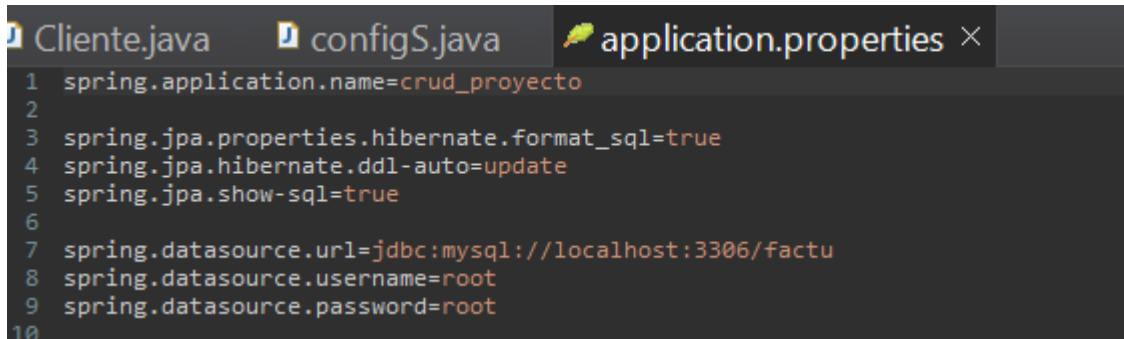
```
CrudProyecto...  CategoriaSe...  VentaServic...  Cliente.pos...  x  » 1  [
94         "method": "DELETE",
95         "header": [],
96         "url": {
97             "raw": "http://localhost:8080/api/clientes/1",
98             "protocol": "http",
99             "host": [
100                 "localhost"
101             ],
102             "port": "8080",
103             "path": [
104                 "api",
105                 "clientes",
106                 "1"
107             ]
108         },
109     },
110     "response": []
111 },
112 {
113     "name": "Actualizar Cliente por su ID",
114     "request": {
115         "auth": {
116             "type": "noauth"
117         },
118         "method": "PUT",
119         "header": [],
120         "body": {
121             "mode": "raw",
122             "raw": "{\r\n  \"nombre\": \"Nuevo \",\r\n  \"apellido\": \"Cliente\",\r\n",
123             "options": {
124                 "raw": {
125                     "language": "json"
126                 }
127             }
128         },
129         "url": {
130             "raw": "http://localhost:8080/api/clientes/1",
131             "protocol": "http",
132             "host": [
133                 "localhost"
134             ],
135             "port": "8080",
136             "path": [
137                 "api",
138                 "clientes",
139                 "1"
140             ]
141         }
142     },
143     "response": []
144 }
145 ]
---
```

- Sql: Guarda el script SQL denominado “factu” que se conecta con este proyecto.



- Application.properties: Esta es una configuración bastante poderosa que es la que se encarga de hacer la conexión de nuestra base de datos con spring que es una de las configuraciones que tiene SpringBoot
- Hibernate: Es la configuración que se encarga de la generar los campos y las tablas a la hora de correr nuestro proyecto en spring.

- JPA: Incluye la extensión JPA, para la generación de los modelos y entienda que se están haciendo de manera correcta todas las tablas.
- Se despliega en el puerto 3306 con localhost.
- Nos pide ingresar con root en el username y password

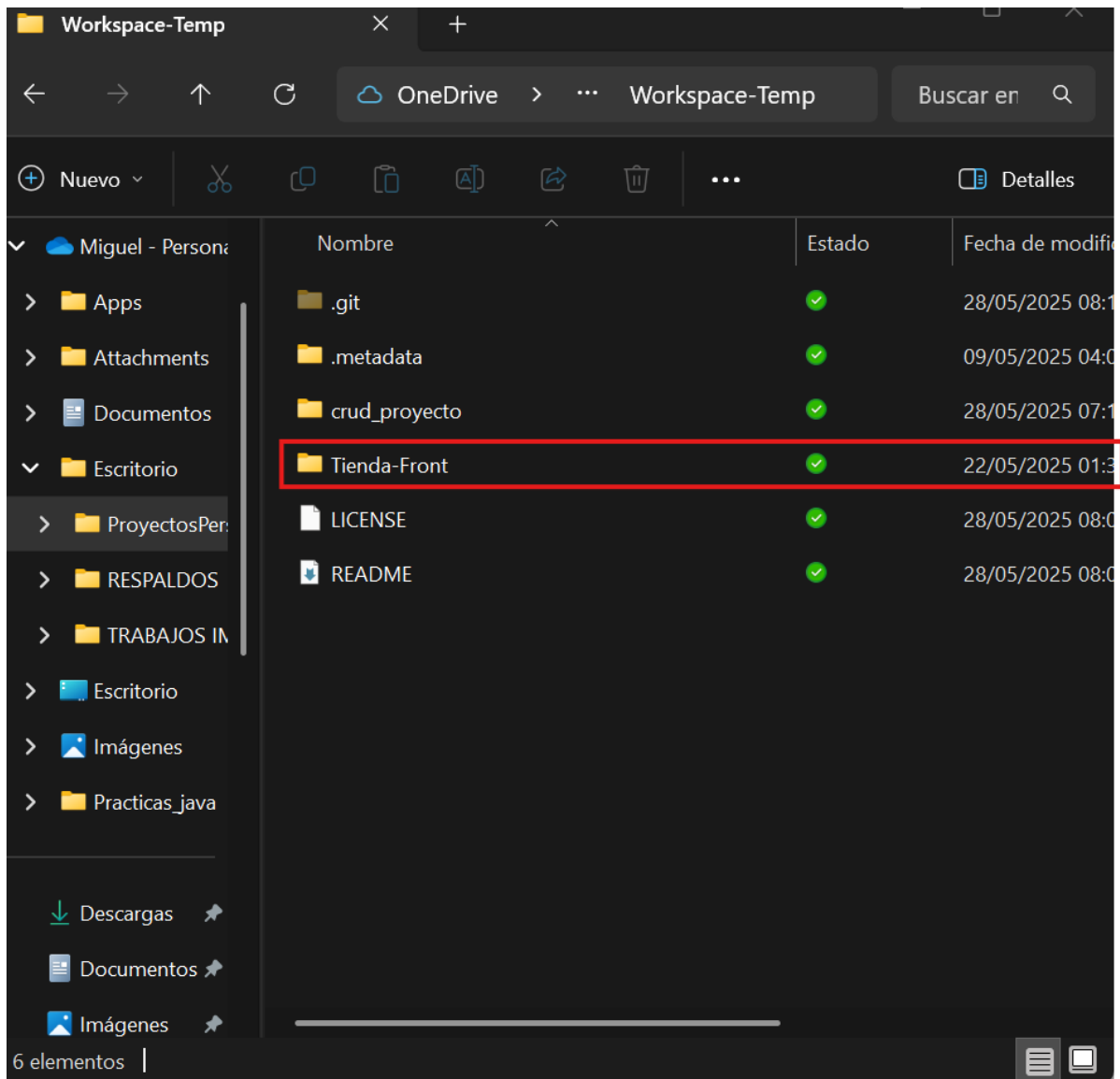


```
1 spring.application.name=crud_proyecto
2
3 spring.jpa.properties.hibernate.format_sql=true
4 spring.jpa.hibernate.ddl-auto=update
5 spring.jpa.show-sql=true
6
7 spring.datasource.url=jdbc:mysql://localhost:3306/factu
8 spring.datasource.username=root
9 spring.datasource.password=root
10
```

#### 9. Integración de la Interfaz Web (HTML + CSS + JS)

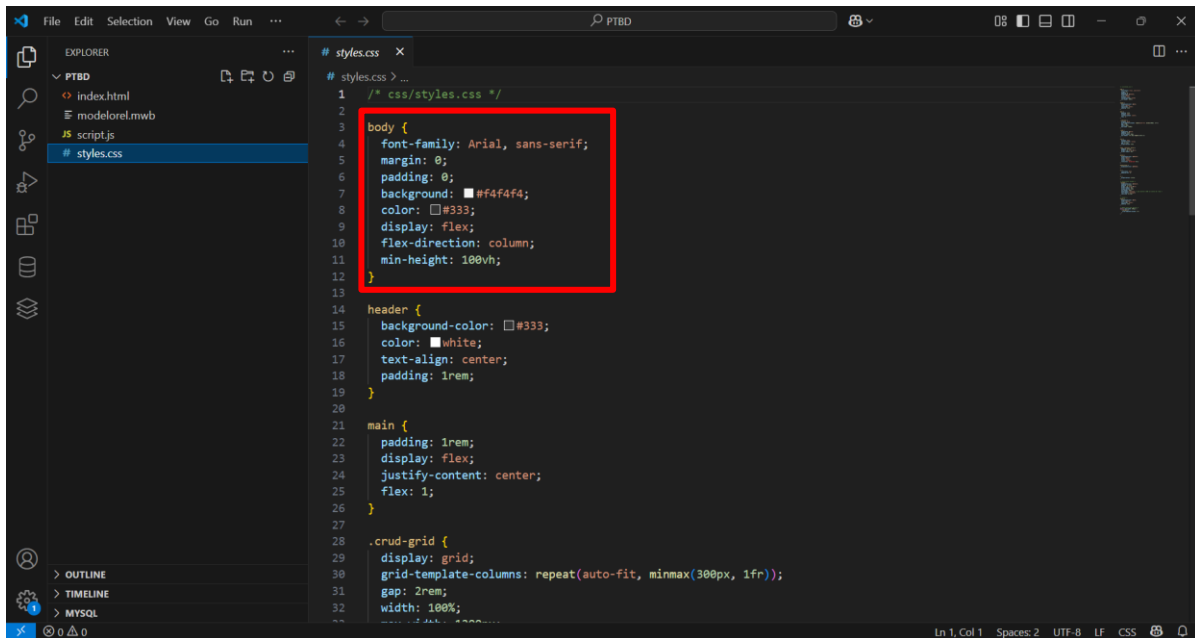
#### Estructura modular:

Dentro de la carpeta para la página web tenemos código en .html, .css y .js.



## CSS

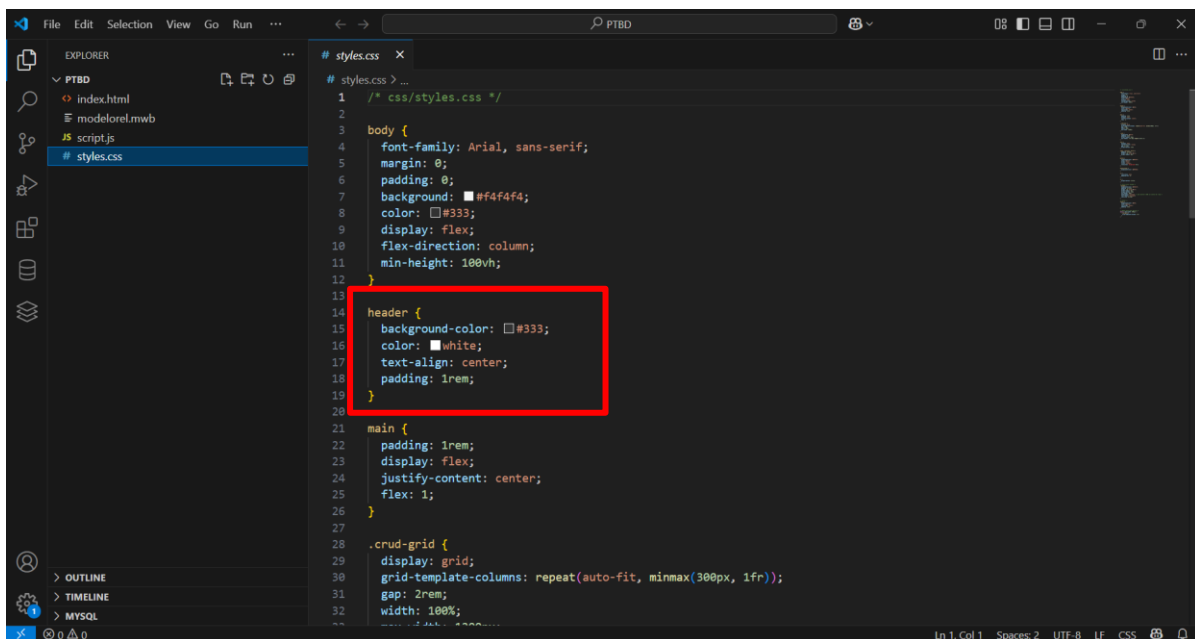
En el archivo styles.css se define el estilo visual de una página web basada en una interfaz CRUD. A continuación, se detalla cada bloque de código y su función.



```
# styles.css
1 /* css/styles.css */
2
3 body {
4   font-family: Arial, sans-serif;
5   margin: 0;
6   padding: 0;
7   background-color: #f4f4f4;
8   color: #333;
9   display: flex;
10  flex-direction: column;
11  min-height: 100vh;
12 }
13
14 header {
15   background-color: #333;
16   color: white;
17   text-align: center;
18   padding: 1rem;
19 }
20
21 main {
22   padding: 1rem;
23   display: flex;
24   justify-content: center;
25   flex: 1;
26 }
27
28 .crud-grid {
29   display: grid;
30   grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
31   gap: 2rem;
32   width: 100%;
```

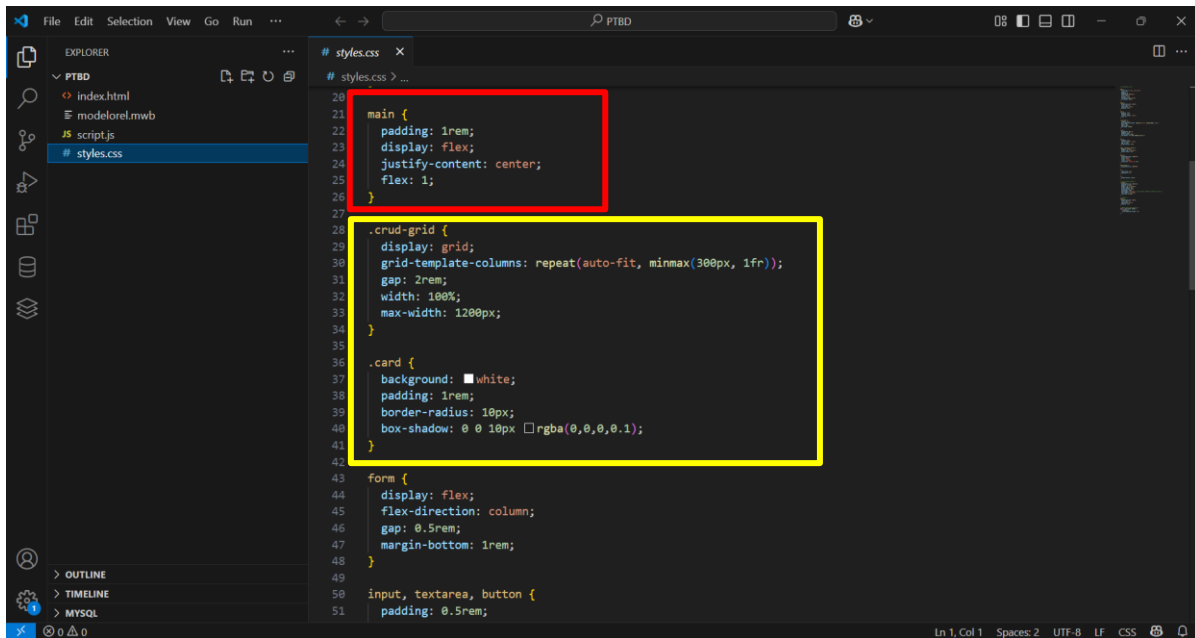
Dentro de este bloque body se establece:

- Una tipografía clara y legible “Arial”.
- Márgenes y relleno eliminados para ocupar toda la pantalla.
- Fondo gris claro para visualizar de mejor manera la página.
- Texto de color oscuro para hacerlo mayormente visible.
- Diseño tipo columna que se extiende a lo largo de toda la pantalla.



```
# styles.css
1 /* css/styles.css */
2
3 body {
4   font-family: Arial, sans-serif;
5   margin: 0;
6   padding: 0;
7   background-color: #f4f4f4;
8   color: #333;
9   display: flex;
10  flex-direction: column;
11  min-height: 100vh;
12 }
13
14 header {
15   background-color: #333;
16   color: white;
17   text-align: center;
18   padding: 1rem;
19 }
20
21 main {
22   padding: 1rem;
23   display: flex;
24   justify-content: center;
25   flex: 1;
26 }
27
28 .crud-grid {
29   display: grid;
30   grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
31   gap: 2rem;
32   width: 100%;
```

En esta parte se define un encabezado oscuro, centrada y con padding. Nos sirve como barra superior de título para nuestra página web.



### Dentro del main.

En esta parte se estiliza el cuerpo del contenido:

- Con espacio interior (padding).
- Centrado horizontalmente.
- Expande el área para ocupar el resto de la pantalla.

### Dentro de .crud-grid.

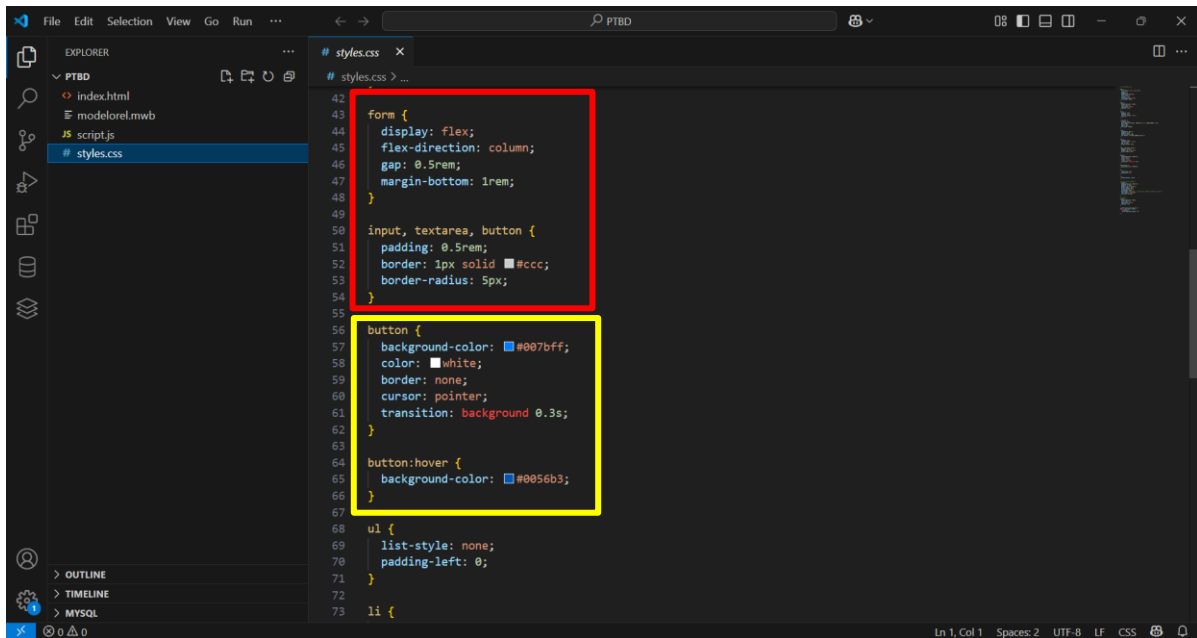
Se organizan los elementos (como formularios y tarjetas) en una cuadrícula adaptable, que contiene:

- Columnas de mínimo 300px.
- Ajusta automáticamente el número de columnas según el ancho de pantalla.
- Tiene separaciones (gap) entre los elementos.

### Dentro de .card.

Se estiliza cada sección como una tarjeta visual.

- Fondo blanco.
- Bordes redondeados y sombra suaves.

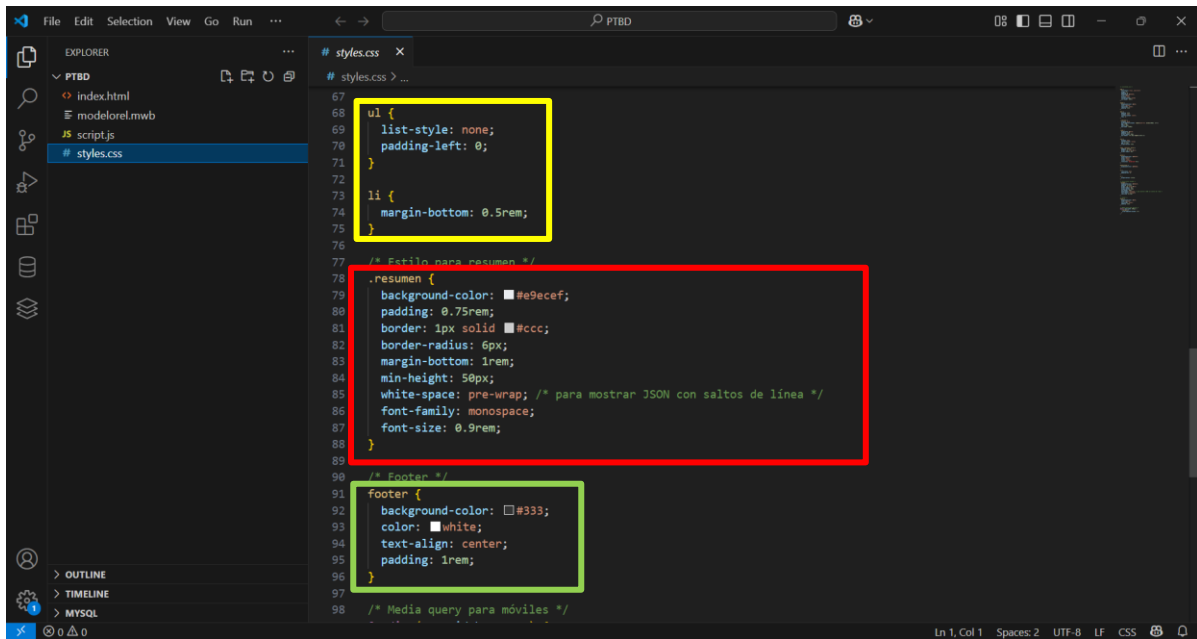


Dentro de form e input, textarea y button, se definen:

- Formularios verticales.
- Campos con relleno, borde claro y bordes redondeados.

Dentro de button, se definen:

- Fondos azul brillante.
- Cambio de color al pasar el mouse para dar retroalimentación visual.



Dentro de ul, li.

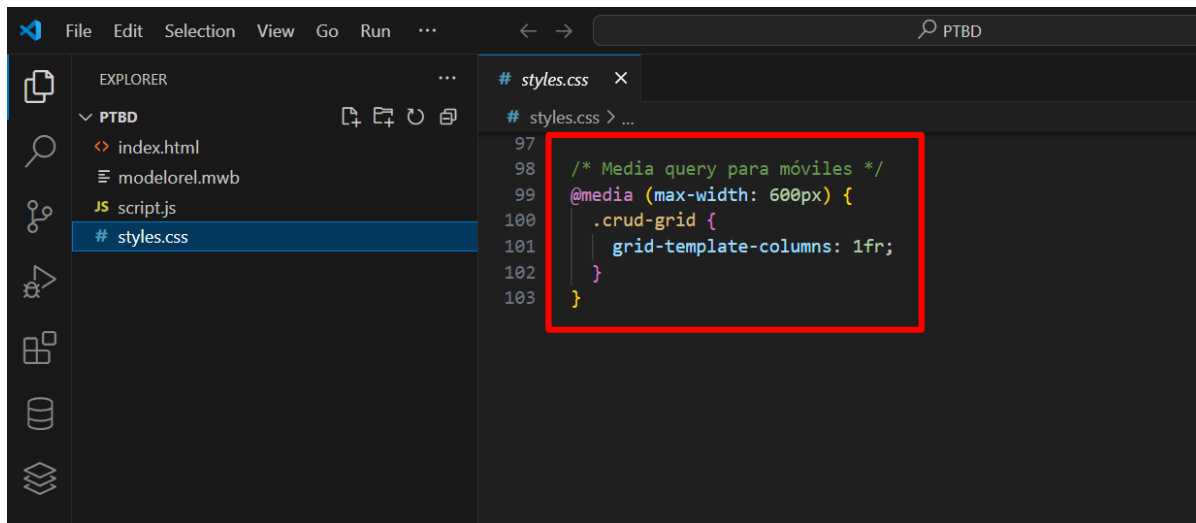
Se eliminan viñetas y espacios entre en las listas para mostrarlas limpias y alineadas.

En la sección de `.resumen`.

Se estiliza una caja para mostrar un resumen en JSON, útil para mostrar datos guardados o procesados.

Dentro de footer.

Se define el estilo del footer igual que el header.



@media.

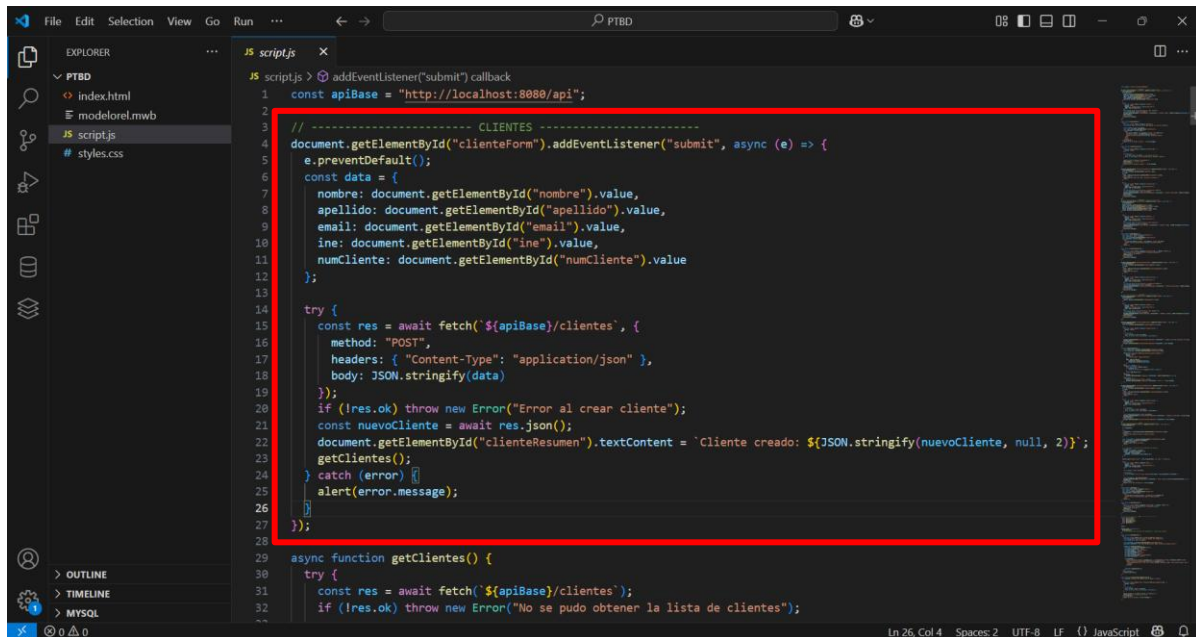
Este bloque se activa en pantallas pequeñas y cambia el diseño de una cuadrícula a una sola columna, facilitando la lectura y navegación en móviles.

## JS

Este archivo nos ayuda a conectar la aplicación web con una API REST.

```
JS script.js > ...  
1  const apiBase = "http://localhost:8080/api";
```

Aquí se define la URL base para todas las peticiones a la API, sin tener que repetirla en cada fetch().



Este fragmento del javascript permite registrar un nuevo cliente mediante un formulario web.

Al presionar el botón “enviar”, se realiza lo siguiente:

1. Prevención del envío normal del formulario con `e.preventDefault()`.
2. Se recolectan los datos ingresados por el usuario (nombre, apellido, email, ine y numCliente).
3. Se construye un objeto data con esa información.
4. Se envía una solicitud HTTP tipo post al backend en la URL `http://localhost:8080/api/clientes`, utilizando `fetch()` para guardar ese cliente en la base de datos.
5. Si la operación es exitosa, se muestra un mensaje en pantalla con los datos del cliente recién creado y se recarga la lista de clientes (`getClientes()`).
6. Si ocurre un error (por ejemplo, conexión fallida o validación del servidor), se muestra una alerta con el mensaje correspondiente.

JavaScript (vanilla):

- Uso de fetch() para consumir la API REST
- Validación de formularios
- Actualización dinámica del DOM con los datos

Ejemplo:

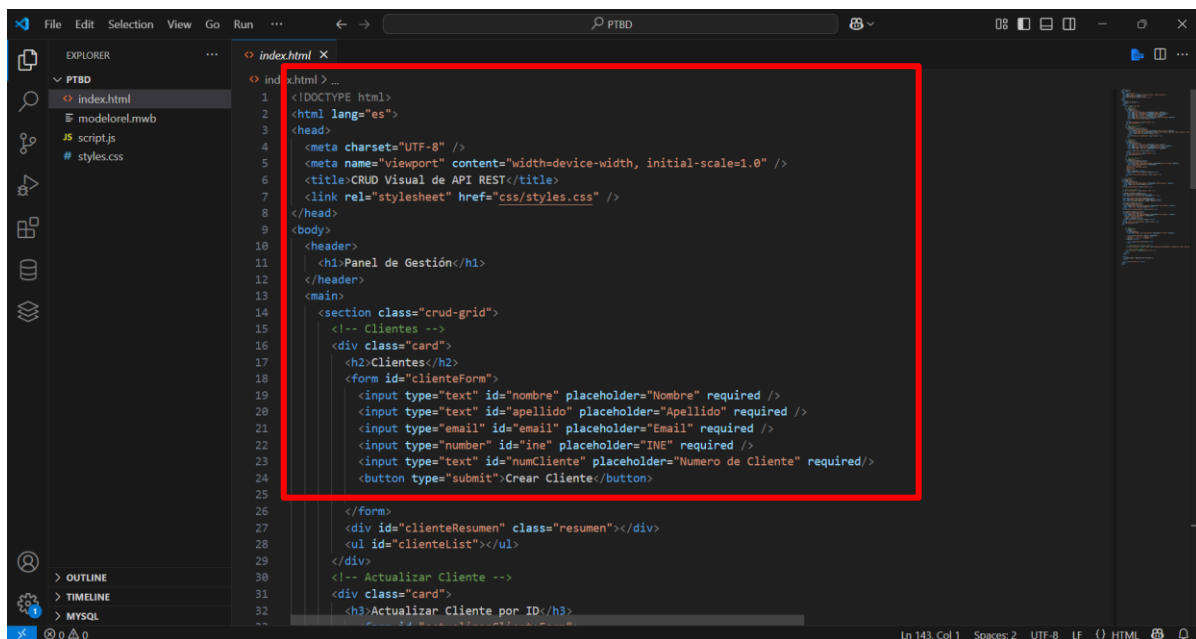
```
fetch('http://localhost:8080/productos')
  .then(res => res.json())
  .then(data => mostrarProductos(data))
```

Responsividad: uso de CSS Flexbox o Grid

## HTML

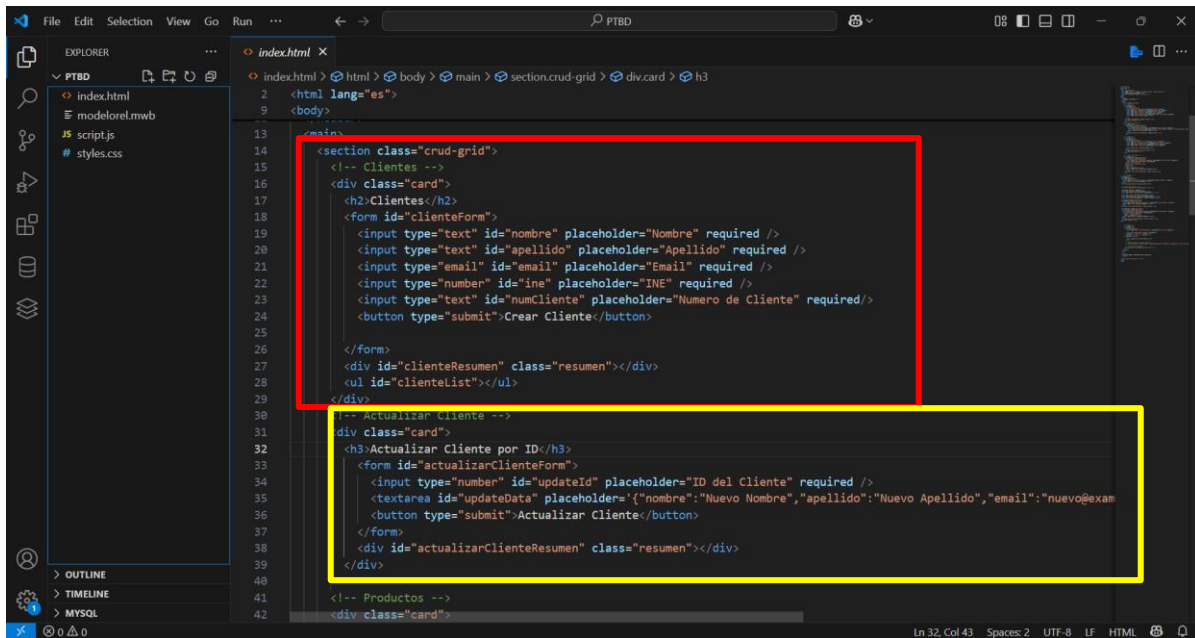
Es el lenguaje base que se usa para crear la estructura de la pagina web.

Se define la versión del navegador a utilizar y posteriormente se define el idioma principal del contenido.



se define el titulo de cabecera del sitio “Panel de Gestión”.

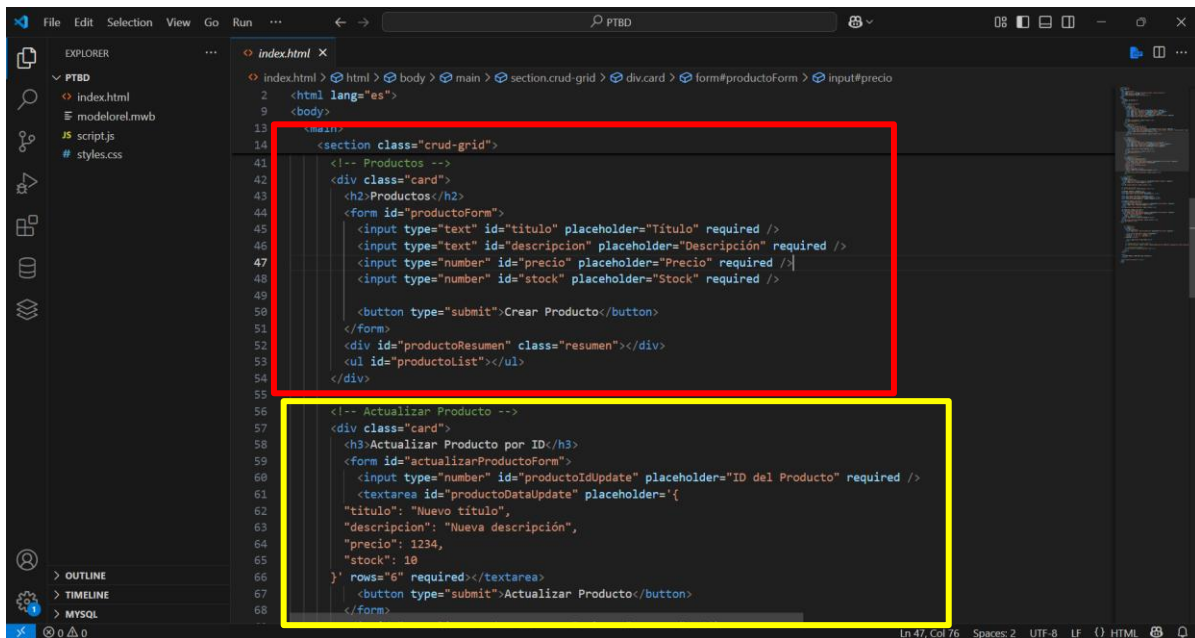
Dentro del main, todo se divide en tarjetas (.card) que representan módulos: Clientes, Productos, Categoría y Ventas. Cada uno tiene su propio formulario y acciones (crear, actualizar, eliminar, etc.).



```
1 <html lang="es">
2 <body>
3
4 <!-- Clientes -->
5 <div class="card">
6 <h2>Clientes</h2>
7 <form id="clienteForm">
8   <input type="text" id="nombre" placeholder="Nombre" required />
9   <input type="text" id="apellido" placeholder="Apellido" required />
10  <input type="email" id="email" placeholder="Email" required />
11  <input type="number" id="line" placeholder="INE" required />
12  <input type="text" id="numCliente" placeholder="Numero de Cliente" required />
13  <button type="submit">Crear Cliente</button>
14 </form>
15 <div id="clienteResumen" class="resumen"></div>
16 <ul id="clienteList"></ul>
17 </div>
18
19 <!-- Actualizar Cliente -->
20 <div class="card">
21 <h3>Actualizar Cliente por ID</h3>
22 <form id="actualizarClienteForm">
23   <input type="number" id="updateId" placeholder="ID del Cliente" required />
24   <textarea id="updateData" placeholder="{ 'nombre': 'Nuevo Nombre', 'apellido': 'Nuevo Apellido', 'email': 'nuevo@exam' }" required />
25   <button type="submit">Actualizar Cliente</button>
26 </form>
27 <div id="actualizarClienteResumen" class="resumen"></div>
28 </div>
29
30 <!-- Productos -->
31 <div class="card">
```

Dentro del form clientes se recogen los datos del cliente para enviarlos al backend por medio del API REST.

El form actualizarCliente, nos permite modificar los datos de un cliente ya existente, usando su ID.



```
1 <html lang="es">
2 <body>
3
4 <!-- Productos -->
5 <div class="card">
6 <h2>Productos</h2>
7 <form id="productoForm">
8   <input type="text" id="titulo" placeholder="Titulo" required />
9   <input type="text" id="descripcion" placeholder="Descripción" required />
10  <input type="number" id="precio" placeholder="Precio" required />
11  <input type="number" id="stock" placeholder="Stock" required />
12  <button type="submit">Crear Producto</button>
13 </form>
14 <div id="productoResumen" class="resumen"></div>
15 <ul id="productoList"></ul>
16 </div>
17
18 <!-- Actualizar Producto -->
19 <div class="card">
20 <h3>Actualizar Producto por ID</h3>
21 <form id="actualizarProductoForm">
22   <input type="number" id="productoIdUpdate" placeholder="ID del Producto" required />
23   <textarea id="productoDataUpdate" placeholder="{ 'titulo': 'Nuevo titulo', 'descripcion': 'Nueva descripción', 'precio': 1234, 'stock': 10 }" rows="6" required />
24   <button type="submit">Actualizar Producto</button>
25 </form>
26 </div>
```

Los form de productos, funcionan de la misma manera que los formularios para clientes, solo que para productos que incluyen (titulo, descripción, precio, stock).

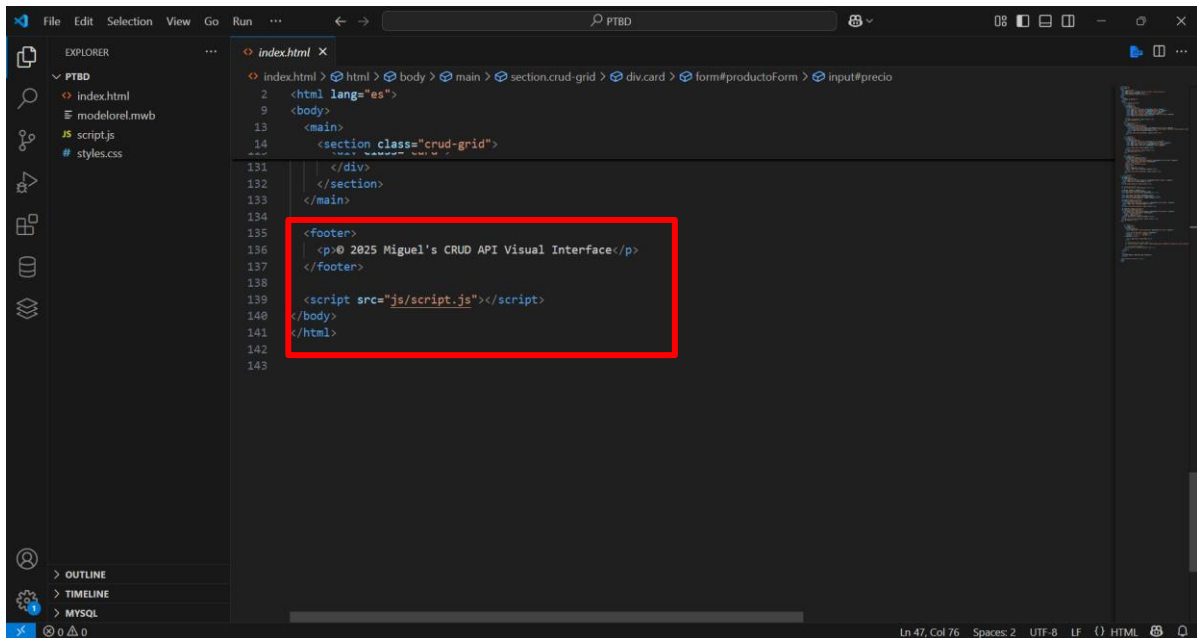
```
index.html X
index.html > html > body > main > section.crud-grid > div.card > form#productoForm > input#precio
2 <html lang="es">
9 <body>
13 <main>
14 <section class="crud-grid">
71 |
72 | <!-- Categorías -->
73 | <div class="card">
74 | <h2>Categorías</h2>
75 | <form id="categoriaForm">
76 |   <input type="text" id="nombreCategoria" placeholder="Nombre Categoría" required />
77 |   <button type="submit">Crear Categoría</button>
78 | </form>
79 | <div id="categoriaResumen" class="resumen"></div>
80 |
81 | <!-- Aquí va la lista -->
82 | <ul id="listaCategorias" style="padding: 1rem;"></ul>
83 |
84 | <h3>Asignar Producto a Categoría</h3>
85 | <label for="productoId">ID del Producto:</label>
86 | <input type="number" id="productoId" placeholder="Ej. 2"><br>
87 |
88 | <label for="categoriaId">ID de la Categoría:</label>
89 | <input type="number" id="categoriaId" placeholder="Ej. 2"><br>
90 | <button onclick="asignarCategoria()">Asignar Categoría</button>
91 |
92 | <h3>Eliminar Categoría por ID</h3>
93 | <form id="eliminarCategoriaForm">
94 |   <input type="number" id="categoriaIdDelete" placeholder="ID de Categoría" required />
95 |   <button type="submit">Eliminar Categoría</button>
96 | </form>
97 | <div id="eliminarCategoriaResumen" class="resumen"></div>
98 |
```

El form de categoría, permite crear categorías, luego podemos asignarlas a productos o modificar su ID.

```
index.html X
index.html > html > body > main > section.crud-grid > div.card > form#productoForm > input#precio
2 <html lang="es">
9 <body>
13 <main>
14 <section class="crud-grid">
109 </div>
110 |
111 | <!-- Ventas -->
112 | <div class="card">
113 | <h2>Ventas</h2>
114 | <form id="ventaForm">
115 |   <input type="number" id="clienteVentaId" placeholder="ID Cliente" required />
116 |
117 |   <textarea id="lineasVenta" rows="5" placeholder="[
118 |     { "producto": { "id": 1 }, "cantidad": 2 },
119 |     { "producto": { "id": 2 }, "cantidad": 1 }
120 |   ]" required></textarea>
121 |
122 |   <button type="submit">Crear Venta</button>
123 | </form>
124 |
125 | <!-- Resumen bonito de la venta creada -->
126 | <pre id="ventaResumen" class="resumen" style="background-color: #f4f4f4; padding: 1em; border-radius: 8px;"></pre>
127 |
128 | <!-- Lista de ventas creadas -->
129 | <ul id="ventalist" style="margin-top: 1em;"></ul>
130 | </div>
131 | </section>
132 | </main>
133 |
134 | <footer>
135 | <p>© 2025 Miguel's CRUD API Visual Interface</p>
136 |
```

Aquí se crean ventas indicando que cliente compro que producto y en que cantidad.

También hay elementos como `<textarea>`, `<ul>`, `<div class = "resumen">` y `<pre>` que muestran información de respuestas o listas generadas dinámicamente.



```
index.html X
  2 <html lang="es">
  9 <body>
 13 <main>
 14 <section class="crud-grid">
 15 <div class="card">
 16 <div class="form">
 17 <input type="text" value="" />
 18 <input type="button" value="Crear" />
 19 </div>
 20 <div class="table">
 21 <table>
 22 <thead>
 23 <tr>
 24 <th>ID</th>
 25 <th>Nombre</th>
 26 <th>Precio</th>
 27 </tr>
 28 </thead>
 29 <tbody>
 30 <tr>
 31 <td>1</td>
 32 <td>Producto 1</td>
 33 <td>1000</td>
 34 </tr>
 35 </tbody>
 36 </table>
 37 </div>
 38 </div>
 39 </section>
 40 </main>
 41 </body>
 42 </html>
 43
```

```
135 <footer>
136 <p>© 2025 Miguel's CRUD API Visual Interface</p>
137 </footer>
138
139 <script src="js/script.js"></script>
140 </body>
141 </html>
142
143
```

El footer pone la información en el pie de pagina.

Dentro del `<script src="js/script.js"></script>` se conecta el archivo javascript que da vida al formulario. Se encarga de enviar datos, recibir respuestas y manipular el DOM.

El archivo HTML esta organizado de forma clara y modular. Utiliza etiquetas semánticas para estructurar visualmente la interfaz. Cada modulo funcional esta contenida en una tarjeta que incluye formularios para crear, actualizar y eliminar registros. Además se incorporan archivos externos de estilos .css y scripts .js para separar la lógica de presentación y comportamiento, promoviendo así una estructura mantenible.

## Visualización Pagina web

Se muestra la visualización de página web, se muestran todos los formularios.

The screenshot displays the 'Panel de Gestión' (Management Panel) web application. It features a dark blue header with the title 'Panel de Gestión'. The main content area is divided into six white panels with blue borders, each containing a form for data management. The forms are: 'Clientes' (Clients) with fields for Nombre, Apellido, Email, RNE, and Numero de Cliente; 'Actualizar Cliente por ID' with a text area for JSON data; 'Productos' (Products) with fields for Titulo, Descripción, Precio, and Stock; 'Actualizar Producto por ID' with a text area for JSON data; 'Categorías' (Categories) with a field for Nombre Categoría; and 'Ventas' (Sales) with a field for ID Cliente and a text area for JSON data. Each form has a blue 'Crear' (Create) button and a grey 'Actualizar' (Update) button. The browser's address bar shows the file path: C:/Users/vespe/OneDrive/Escritorio/Proyectos/Personales/Curso-Coder-Java/Workspace-Temp/Tienda-Front/index.html.

This screenshot shows a different view of the 'Panel de Gestión' web application, focusing on the 'Categorías' (Categories) and 'Ventas' (Sales) sections. The 'Actualizar Producto por ID' form is still visible on the left. The 'Categorías' section includes a 'Listado de Categorías' (Categories List) and an 'Asignar Producto a Categoría' (Assign Product to Category) form with fields for ID del Producto and ID de la Categoría. Below this is an 'Eliminar Categoría por ID' (Delete Category by ID) form and an 'Actualizar Categoría por ID' (Update Category by ID) form. The 'Ventas' section includes an 'Actualizar Venta por ID' (Update Sale by ID) form with fields for ID de Venta and a text area for JSON data. The browser's address bar shows the same file path: C:/Users/vespe/OneDrive/Escritorio/Proyectos/Personales/Curso-Coder-Java/Workspace-Temp/Tienda-Front/index.html.

#### Fase 4: Reglas de negocios.

Posteriormente se definieron las reglas del negocio fundamentales del sistema.

##### 1. Sistemas de Puntos para clientes:

Estas reglas establecen que, por cada compra registrada, deben disminuir las existencias del producto adquirido y otorgarse puntos al cliente conforme al valor total de la compra.

Las reglas se definieron como sigue:

- Si el costo total de la compra es superior a \$1500, se otorgan 15 puntos.
- Si el costo total se encuentra entre \$800 y \$1500, se otorgan 10 puntos.
- Si el costo total es menor a \$800, se otorgan 10 puntos.

Estos puntos deben sumarse al total acumulado del cliente (puntosGanados) fortaleciendo así un sistema de fidelización.

##### 2. Control de Inventario

- Al realizar una venta, se descuenta del stock del producto la cantidad vendida.
- Si no hay suficiente stock, la venta debe rechazarse (esto debe validarse en el servicio).

##### 3. Relación Producto-Categoría:

- La asignación de categorías se hace de forma separada mediante un endpoint especial, como está establecido en **ProductoController**.

##### 4. Persistencia desacoplada con DTOS:

- Uso de clases **VentaRequestDTO**, **VentaAgregarProductosDTO** para manejar la entrada/salida de datos, separando el modelo de negocio del modelo de presentación.

##### 5. Relación Cliente-Venta debe ser valida

- No se debe permitir crear ventas para clientes inexistentes.

#### Validaciones y Triggers (MySQL)

##### 1. Triggers definidos en la BD:

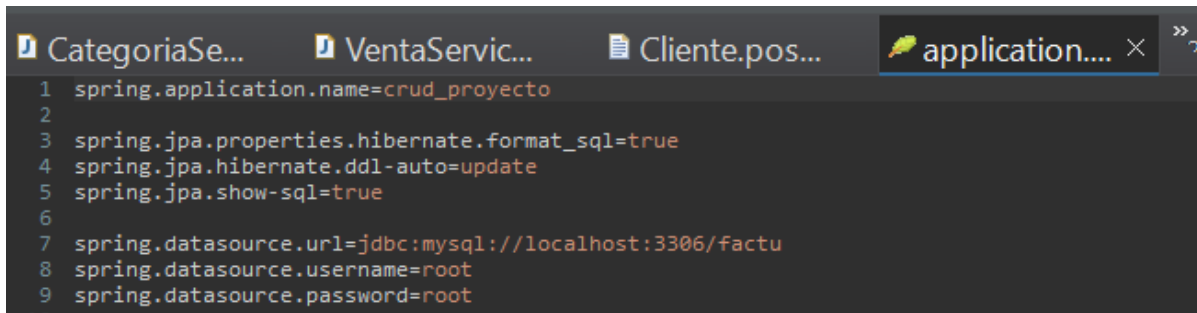
- a. Uno para mostrar historial de ventas.
- b. Otro para recuperar registros eliminados.

##### 2. Validaciones funcionales:

- a. Que se guarden correctamente las relaciones entre producto y venta.
- b. Que se actualice el stock al realizar la venta.
- c. Que se sumen puntos correctamente.

#### Fase 5:

Con las estructuras lógicas y reglas de negocio ya establecidas, se integró el driver JDBC para conexión con MySQL, y se configuró el acceso a la base de datos desde el archivo `application.properties`.



```
1 spring.application.name=crud_proyecto
2
3 spring.jpa.properties.hibernate.format_sql=true
4 spring.jpa.hibernate.ddl-auto=update
5 spring.jpa.show-sql=true
6
7 spring.datasource.url=jdbc:mysql://localhost:3306/factu
8 spring.datasource.username=root
9 spring.datasource.password=root
```

## Fase 6:

Las fases de programación en java consistieron en crear clases específicas para cada tipo de operaciones CRUD (create, read, update, delete). Además, se integraron interfaces gráficas con menús o botones que facilitan al usuario la interacción con la base de datos. Cada clase se encargo de una tarea puntual: insertar registros, eliminar productos, modificar datos y realizar consultas, todo ellos respaldado con una clase base que maneja la conexión a la base de datos.

Un elemento esencial del proyecto fue la implementación de triggers directamente en MySQL. Se crearon dos triggers principales: uno para mostrar el historial de las ventas y otro para la lectura de los datos que han sido eliminados. Estos triggers aseguran que la lógica de negocio se mantenga intacta incluso si las operaciones se ejecutan de forma directa sobre la base de datos, sin pasar por el sitio java.

Finalmente se desarrolló una etapa de validación exhaustiva. Esta incluyo pruebas funcionalidades de cada clase y triggers, captura de evidencias gráficas que demuestren el éxito de la conexión, la ejecución correcta de las operaciones y la verificación de las reglas de negocio implementadas.

Este enfoque metodológico no solo facilitó la comprensión profunda de los procesos de integración entre Java y MySQL, sino que también aseguro la eficiencia y viabilidad del sistema desarrollado.

## Triggers.

Los triggers o disparadores en SQL constituyen herramientas fundamentales para la automatización y control de bases de datos relacionales, sirviendo como mecanismos que ejecuten código automáticamente en respuestas a eventos específicos. Nos ayudan principalmente en la capacidad de responder de manera inmediata a operaciones de manipulación de datos como INSERT, UPDATE y DELETE, ejecutándose antes o después de estos eventos para garantizar que se cumplan las políticas establecidas para la base de datos.

Dentro de nuestro proyecto los triggers nos permiten implementar lógica condicional avanzada que puede tomar diferentes acciones basándose en los valores específicos de los datos, el contexto de la operación y las condiciones del sistema.

## Estructura de los Triggers dentro de la Base de Datos.

El sistema utiliza una base de datos llamada **factu** que contiene múltiples tablas interrelacionadas para gestionar clientes, productos, ventas y categorías. La arquitectura

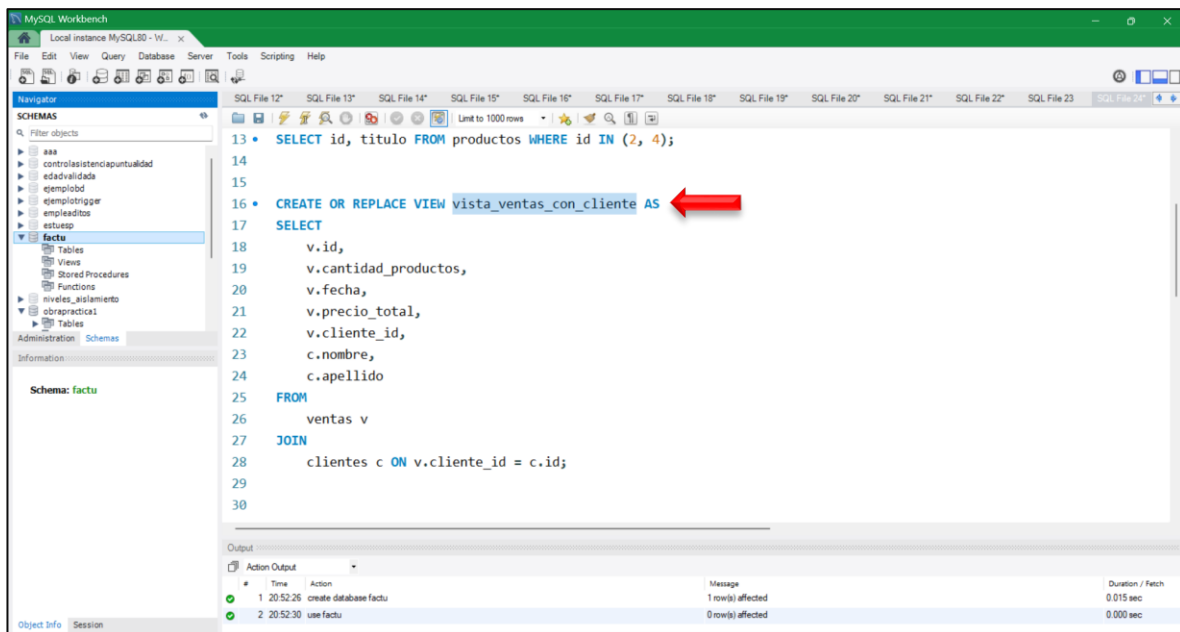
sigue un soporte para operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y mecanismos de auditoría integrados.

#### Las tablas principales:

- **Clientes:** almacena información de los clientes del sistema.
- **Productos:** registra el catálogo de artículos disponibles.
- **Ventas:** gestiona las transacciones comerciales realizadas.
- **Categorías:** organiza los productos en grupos jerárquicos.
- **producto\_cliente** y **producto\_venta:** tablas de relación many-to-many.
- **detalle\_venta:** contiene el desglose específico de cada transacción.

#### Vista de ventas consolidada.

- **vista\_ventas\_con\_cliente**



Se visualizan los datos que arroja el realizar la consulta de vista\_ventas\_con\_cliente.

|                                                      |                                           |  |  |  |  |  |  |
|------------------------------------------------------|-------------------------------------------|--|--|--|--|--|--|
| crudFacturas* x SQL File 4* empleaditos* SQL File 5* |                                           |  |  |  |  |  |  |
| Limit to 1000 rows                                   |                                           |  |  |  |  |  |  |
| 1                                                    | • create database factu;                  |  |  |  |  |  |  |
| 2                                                    | • use factu;                              |  |  |  |  |  |  |
| 3                                                    |                                           |  |  |  |  |  |  |
| 4                                                    | • select * from clientes;                 |  |  |  |  |  |  |
| 5                                                    | • select * from productos;                |  |  |  |  |  |  |
| 6                                                    | • select * from ventas;                   |  |  |  |  |  |  |
| 7                                                    | • SELECT * FROM vista_ventas_con_cliente; |  |  |  |  |  |  |
| 8                                                    | • select * from categorias;               |  |  |  |  |  |  |

| Result Grid Filter Rows: Export: Wrap Cell Content: IA |    |                    |                            |              |            |                |             |
|--------------------------------------------------------|----|--------------------|----------------------------|--------------|------------|----------------|-------------|
|                                                        | id | cantidad_productos | fecha                      | precio_total | cliente_id | nombre         | apellido    |
| ▶                                                      | 1  | 1                  | 2025-05-30 20:56:19.387568 | 1500         | 1          | Miguel Eduardo | Urena Nieto |
|                                                        | 2  | 2                  | 2025-05-30 20:57:32.733968 | 2000         | 3          | Arleth         | Cruz        |
|                                                        | 3  | 3                  | 2025-05-30 20:58:28.293481 | 2700         | 4          | Moises         | Tafolla     |
|                                                        | 4  | 1                  | 2025-05-30 20:59:31.737085 | 900          | 5          | Nuevo Cliente  | Prueba      |
|                                                        | 5  | 1                  | 2025-05-30 21:03:22.686220 | 600          | 6          | Jesus          | Salinas     |

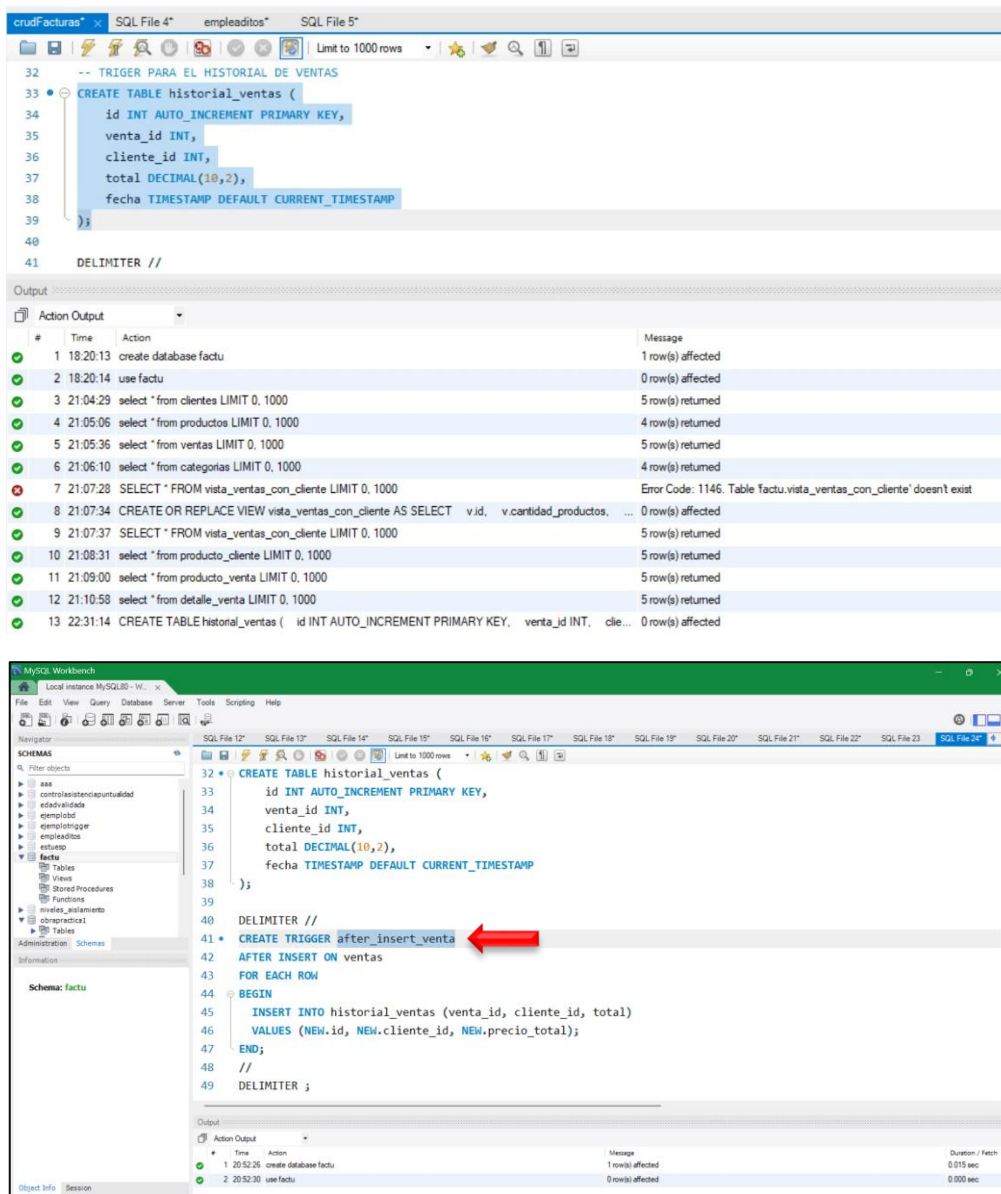
En esta vista se combina información de las tablas **VENTAS** y **CLIENTES**, para proporcionar un reporte unificado de transacciones comerciales. Los campos seleccionados incluyen identificador único de venta, cantidad de productos involucrados, fecha de operaciones, monto total y datos del cliente asociado.

La cláusula **JOIN** realiza una combinación interna usando el campo **cliente\_id** como clave de relación.

#### Automatización mediante Triggers.

- **after\_insert\_venta**

Creación del triggers **historial\_ventas**.



Este disparador se activa posteriormente a cada inserción en la tabla **VENTAS**, registrando automáticamente un historial en la tabla **historial\_ventas**.

Consulta del trigger.

71 • `SELECT * FROM historial_ventas;`

| Result Grid    Filter Rows: <input type="text"/> Edit:    |          |            |         |                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------|---------|---------------------|
| id                                                                                                                                                                                                                                                                                                                                                                                              | venta_id | cliente_id | total   | fecha               |
| 1                                                                                                                                                                                                                                                                                                                                                                                               | 6        | 1          | 1000.00 | 2025-05-30 22:34:14 |
| 2                                                                                                                                                                                                                                                                                                                                                                                               | 7        | 3          | 1200.00 | 2025-05-30 22:34:24 |
| 3                                                                                                                                                                                                                                                                                                                                                                                               | 8        | 5          | 2400.00 | 2025-05-30 22:34:35 |
| 4                                                                                                                                                                                                                                                                                                                                                                                               | 9        | 4          | 3000.00 | 2025-05-30 22:34:51 |
| NULL                                                                                                                                                                                                                                                                                                                                                                                            | NULL     | NULL       | NULL    | NULL                |

Ultimas ventas registradas:

|                          |          |
|--------------------------|----------|
| Venta #6 - Cliente ID: 1 | Eliminar |
| Venta #7 - Cliente ID: 3 | Eliminar |
| Venta #8 - Cliente ID: 5 | Eliminar |
| Venta #9 - Cliente ID: 4 | Eliminar |

Dentro de la consulta del trigger de historial\_ventas se muestran los datos de las ventas que se han realizado id del numero de venta, IdVenta, IdCliente, total de la venta que el cliente realizo y la fecha de la venta que se genera de manera automática.

La estructura de la tabla historial incluye:

- identificador único autoincremental.
- Referencia de la venta original,
- Identificador del cliente asociado.
- Monto total de la transacción.
- Marca temporal de registro.

- **after\_delete\_producto**

## Creación de trigger para la eliminación de productos.

```
52 • -- TRIGGER PARA VER LA ELIMINACION DE UN PRODUCTO
53 CREATE TABLE log_eliminacion_productos (
54     id INT AUTO_INCREMENT PRIMARY KEY,
55     producto_id INT,
56     titulo VARCHAR(255),
57     eliminado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
58 );
59
```

Output

| #  | Time     | Action                                                                                    | Message                                                               |
|----|----------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 5  | 21:05:36 | select * from ventas LIMIT 0, 1000                                                        | 5 row(s) returned                                                     |
| 6  | 21:06:10 | select * from categorias LIMIT 0, 1000                                                    | 4 row(s) returned                                                     |
| 7  | 21:07:28 | SELECT * FROM vista_ventas_con_cliente LIMIT 0, 1000                                      | Error Code: 1146. Table factu.vista_ventas_con_cliente' doesn't exist |
| 8  | 21:07:34 | CREATE OR REPLACE VIEW vista_ventas_con_cliente AS SELECT v.id, v.cantidad_productos, ... | 0 row(s) affected                                                     |
| 9  | 21:07:37 | SELECT * FROM vista_ventas_con_cliente LIMIT 0, 1000                                      | 5 row(s) returned                                                     |
| 10 | 21:08:31 | select * from producto_cliente LIMIT 0, 1000                                              | 5 row(s) returned                                                     |
| 11 | 21:09:00 | select * from producto_venta LIMIT 0, 1000                                                | 5 row(s) returned                                                     |
| 12 | 21:10:58 | select * from detalle_venta LIMIT 0, 1000                                                 | 5 row(s) returned                                                     |
| 13 | 22:31:14 | CREATE TABLE historial_ventas ( id INT AUTO_INCREMENT PRIMARY KEY, venta_id INT, cli...   | 0 row(s) affected                                                     |
| 14 | 22:31:55 | CREATE TRIGGER after_insert_venta AFTER INSERT ON ventas FOR EACH ROW BEGIN INSERT ...    | 0 row(s) affected                                                     |
| 15 | 22:32:23 | CREATE TABLE log_eliminacion_productos ( id INT AUTO_INCREMENT PRIMARY KEY, product...    | 0 row(s) affected                                                     |

```
50
51 • -- TRIGGER PARA VER LA ELIMINACION DE UN PRODUCTO
52 CREATE TABLE log_eliminacion_productos (
53     id INT AUTO_INCREMENT PRIMARY KEY,
54     producto_id INT,
55     titulo VARCHAR(255),
56     eliminado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
57 );
58
59 DELIMITER //
60 • CREATE TRIGGER after_delete_producto
61 AFTER DELETE ON productos
62 FOR EACH ROW
63 BEGIN
64     INSERT INTO log_eliminacion_productos (producto_id, titulo)
65     VALUES (OLD.id, OLD.titulo);
66 END;
67 //
```

Output

| # | Time     | Action                | Message           | Duration / Fetch |
|---|----------|-----------------------|-------------------|------------------|
| 1 | 20:52:26 | create database factu | 1 row(s) affected | 0.015 sec        |
| 2 | 20:52:30 | use factu             | 0 row(s) affected | 0.000 sec        |

Este mecanismo se auditoría captura todas las eliminaciones de productos preservando el identificador y título de artículos eliminado en la tabla **log\_eliminacion\_productos**.

La tabla de log contiene:

- Clave primaria autoincremental.
- Referencia al producto eliminado.
- Nombre del producto al momento de eliminación.
- Fecha/hora automática de registro.

Estas consultas permiten verificar el correcto funcionamiento de los triggers y realizar análisis de las operaciones del sistema.

Consulta de los productos existentes:

|                                      |          |
|--------------------------------------|----------|
| iPhone 15 - \$1500 - Stock: 2        | Eliminar |
| Samsung S25 - \$1000 - Stock: 3      | Eliminar |
| Redmi YS - \$900 - Stock: 16         | Eliminar |
| Huawei Y7 - \$600 - Stock: 4         | Eliminar |
| Acatel One - \$800 - Stock: 10       | Eliminar |
| Otros Celulares - \$1200 - Stock: 20 | Eliminar |

En este caso se eliminará el registro “Redmi YS” y “Alcatel ONE”.

Mensaje del sistema al momento de eliminar un producto.



Consulta del trigger para los productos eliminados.

72 • `SELECT * FROM log_eliminacion_productos;`

| id   | producto_id | titulo     | eliminado_en        |
|------|-------------|------------|---------------------|
| 1    | 3           | Redmi YS   | 2025-05-30 22:38:04 |
| 2    | 5           | Acatel One | 2025-05-30 22:38:17 |
| NULL | NULL        | NULL       | NULL                |

Se muestran todos los datos de los productos eliminados IdProducto, titulo y la fecha de eliminación que se determina automáticamente.

## Resultados

### Formulario de cliente.

Se introducen los datos (nombre, apellido, email, INE, idCliente), una vez creado el cliente se le asigna un ID automático dentro de la base de datos. Una vez creado el cliente se muestra una lista con todos los clientes existentes, y se muestra el numero de puntos que cada uno tiene.

Y posteriormente se muestra la opción para poder editar un cliente ya creado solo utilizando el ID automático asignado y colocando código JSON con los atributos nuevos que se quieren cambiar.

Se muestra el ej. de la creación de un usuario nuevo y la actualización del mismo.

**Clientes**

Juan  
Perez  
perez@gmail.com  
232324455  
203

**Crear Cliente**

Cliente creado: {  
"id": 5,  
"nombre": "Juan ",  
"apellido": "Perez",  
"email": "perez@gmail.com",  
"ine": 232324455,  
"numCliente": "203",  
"puntosTotales": 0,  
"createdAt": "2025-05-30T20:47:10.848791"  
}

**Miguel Eduardo Urena Nieto** -  
vesper1098@gmail.com  
Puntos totales: 0  
**Eliminar**

**Arleth Cruz** - cruz@gmail.com  
Puntos totales: 0  
**Eliminar**

**Moises Tafolla** - tafolla@gmail.com  
Puntos totales: 0  
**Eliminar**

**Nuevo Cliente Prueba** - pruea@email.com  
Puntos totales: 0  
**Eliminar**

**Actualizar Cliente por ID**

5

{  
"nombre": "Nuevo Cliente",  
"apellido": "Prueba",  
"email": "pruea@email.com",  
"ine": 209303,  
"numCliente": 100  
}

**Actualizar Cliente**

Cliente actualizado: {  
"id": 5,  
"nombre": "Nuevo Cliente",  
"apellido": "Prueba",  
"email": "pruea@email.com",  
"ine": 232324455,  
"numCliente": "203",  
"puntosTotales": 0,  
"createdAt": "2025-05-30T20:47:10.848791"  
}

Se muestra la cantidad de puntos que tiene, como es nuevo tienen 0.

Se muestra la parte del código JSON del cliente actualizado.

## Formulario de productos

Dentro del formulario productos se introducen los datos (nombreProducto, descripción, precio y stock) una vez creado el producto se visualiza una lista de los productos existentes y un pedacito de código JSON con las características, el ID del producto se genera automáticamente.

### Productos

Crear Producto

Producto creado: {  
"id": 3,  
"titulo": "OPPO Vivo",  
"descripcion": "Normal",  
"precio": 700,  
"stock": 10,  
"categoria": null  
}

Producto nuevo creado.

iPhone 15 - \$1500 - Stock: 3 Eliminar

Samsung S25 - \$1000 - Stock: 5 Eliminar

OPPO Vivo - \$700 - Stock: 10 Eliminar

### Actualizar Producto por ID

```
{  
  "titulo": "Redmi YS",  
  "descripcion": "PRO",  
  "precio": 900,  
  "stock": 20  
}
```

Actualizar Producto

```
Producto actualizado: {  
  "id": 3,  
  "titulo": "Redmi YS",  
  "descripcion": "PRO",  
  "precio": 900,  
  "stock": 20,  
  "categoria": null  
}
```

Se muestra la parte del código JSON del producto actualizado.

En el formulario de actualizar producto, se introduce el Id del producto que se quiere actualizar y en código JSON los atributos que se quieren actualizar con sus valores nuevos.

## Formulario de categoría.

Dentro del formulario de categorías se crean categorías nuevas con el dato (nombreCategoría), el Id de la categoría se establece automáticamente al crearla una vez creada se visualiza una lista de las categorías ya existentes.

Dentro del formulario asignar categoría, por medio del Id del producto y el Id de la categoría se establece el producto a una categoría en específico.

Smartphones

Crear Categoría

Categoría creada: {  
"id": 3,  
"nombre": "Smartphones"  
}

Listado de Categorías

ID: 1 - Nombre: Apple

ID: 2 - Nombre: Samsung

ID: 3 - Nombre: Smartphones

Se muestra la parte del código JSON de la categoría creada.

Lista de las categorías existentes.

Asignar Producto a Categoría

ID del Producto: 1

ID de la Categoría: 1

Asignar Categoría

Eliminar Categoría por ID

ID de Categoría

Eliminar Categoría

Actualizar Categoría por ID

ID de Categoría

{  
"nombre": "Nueva categoría"  
}

Actualizar Categoría

{  
"id": 1,  
"titulo": "iPhone 15 ",  
"descripcion": "PRO",  
"precio": 1500,  
"stock": 3,  
"categoria": {  
"id": 1,  
"nombre": "Apple"  
}  
}

Al asignar categoría a un producto se muestra un código JSON con los atributos nuevos asignados al producto.

## Formulario de ventas.

En el formulario de ventas solo basta el Id del cliente que quiere realizar una compra, y se tiene que hacer uso de un fragmento de código JSON para determinar el producto que requiere comprar y la cantidad de productos a comprar. Una vez realizada la compra se crea la factura para el cliente, mostrando el monto total y la cantidad de puntos obtenidos.

Código JSON para generar la venta.

5

[{"producto": {"id": 3}, "cantidad": 1}]

Crear Venta

Venta creada:

```
{
  "id": 4,
  "cliente": {
    "id": 5,
    "nombre": "Nuevo Cliente",
    "apellido": "Prueba",
    "email": "pruea@email.com",
    "ine": 232324455,
    "numCliente": "203",
    "puntosTotales": 10,
    "createdAt": "2025-05-30T20:47:10.848791"
  },
  "detalles": [
    {
      "id": 4,
      "producto": {
        "id": 4,
        "titulo": "Redmi YS",
        "descripcion": "PRO",
        "precio": 900,
        "stock": 16,
        "categoria": null
      },
      "cantidad": 1,
      "precioUnitario": 900
    }
  ],
  "fecha": "2025-05-30T20:59:31.7370851",
  "precioTotal": 900,
  "cantidadProductos": 1
}
```

Facturada generada al realizar la venta

Venta #1 - Cliente ID: 1

Eliminar

Venta #2 - Cliente ID: 3

Eliminar

Venta #3 - Cliente ID: 4

Eliminar

Venta #4 - Cliente ID: 5

Eliminar

Clientes

Jesus

Salinas

sali@gmail.com

1234344

205

Crear Cliente

Cliente creado: {  
 "id": 6,  
 "nombre": "Jesus",  
 "apellido": "Salinas",  
 "email": "sali@gmail.com",  
 "ine": 1234344,  
 "numCliente": "205",  
 "puntosTotales": 0,  
 "createdAt": "2025-05-30T21:00:49.450943"  
}

Miguel Eduardo Urena Nieto -  
vesper1098@gmail.com  
Puntos totales: 10  
Eliminar

Arleth Cruz - cruz@gmail.com  
Puntos totales: 15  
Eliminar

Moises Tafolla - tafolla@gmail.com  
Puntos totales: 15  
Eliminar

Nuevo Cliente Prueba - pruea@email.com  
Puntos totales: 10  
Eliminar

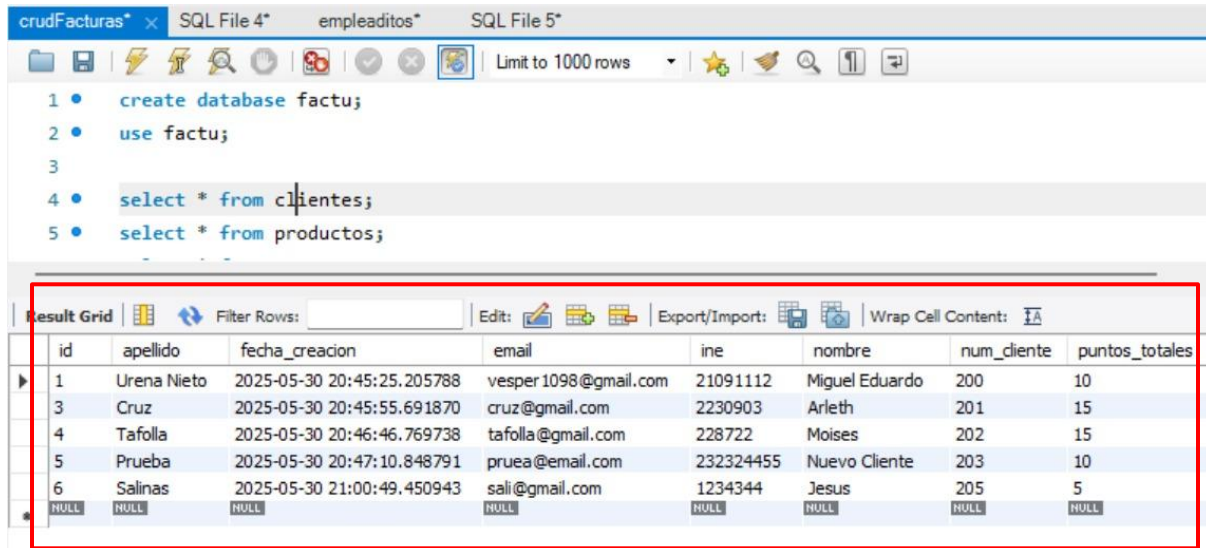
Jesus Salinas - sali@gmail.com  
Puntos totales: 0  
Eliminar

Volviendo al formulario Clientes, podemos observar que se actualizaron los datos de los clientes registrados y hay un incremento de puntos dependiendo del monto de la venta generada es la asignación de puntos. En el ejemplo la venta tiene un valor de \$900 por lo tanto se le otorgan 10 puntos al cliente.

## Consultas.

### Consultar clientes.

Al consultar se muestran los datos de los clientes, incluyendo los puntos acumulados que tiene.

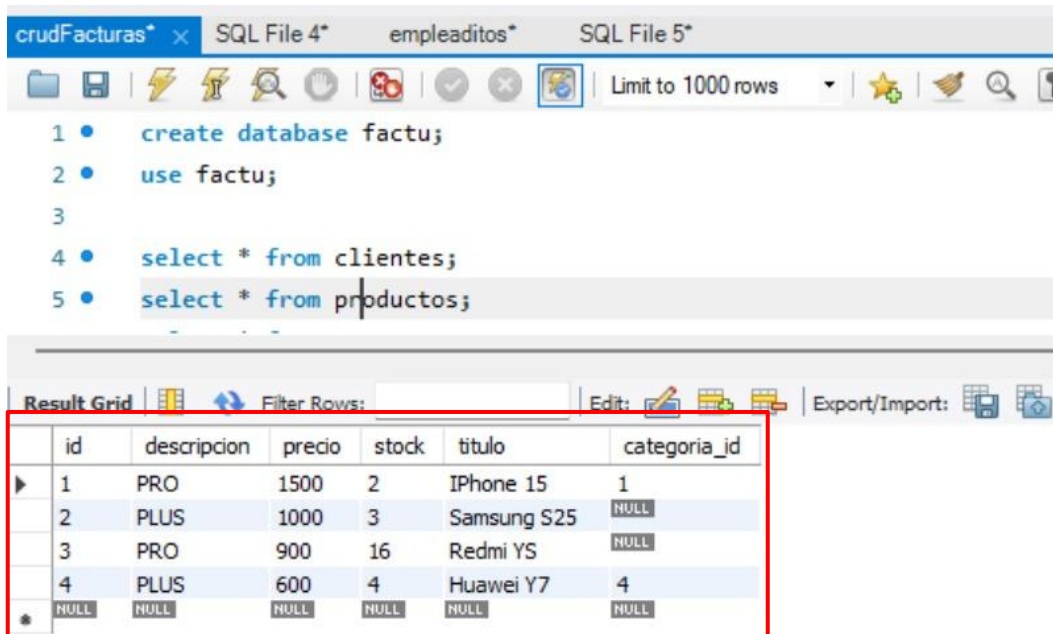


```
1 • create database factu;  
2 • use factu;  
3  
4 • select * from clientes;  
5 • select * from productos;
```

|   | id   | apellido    | fecha_creacion             | email                | ine       | nombre         | num_cliente | puntos_totales |
|---|------|-------------|----------------------------|----------------------|-----------|----------------|-------------|----------------|
| ▶ | 1    | Urena Nieto | 2025-05-30 20:45:25.205788 | vesper1098@gmail.com | 21091112  | Miguel Eduardo | 200         | 10             |
|   | 3    | Cruz        | 2025-05-30 20:45:55.691870 | cruz@gmail.com       | 2230903   | Arleth         | 201         | 15             |
|   | 4    | Tafolla     | 2025-05-30 20:46:46.769738 | tafolla@gmail.com    | 228722    | Moises         | 202         | 15             |
|   | 5    | Prueba      | 2025-05-30 20:47:10.848791 | pruea@email.com      | 232324455 | Nuevo Cliente  | 203         | 10             |
|   | 6    | Salinas     | 2025-05-30 21:00:49.450943 | sali@gmail.com       | 1234344   | Jesus          | 205         | 5              |
| * | NULL | NULL        | NULL                       | NULL                 | NULL      | NULL           | NULL        | NULL           |

### Consulta productos.

Al realizar la consulta de los productos existentes se muestran los datos, y la categoría a la que ha sido asignado.

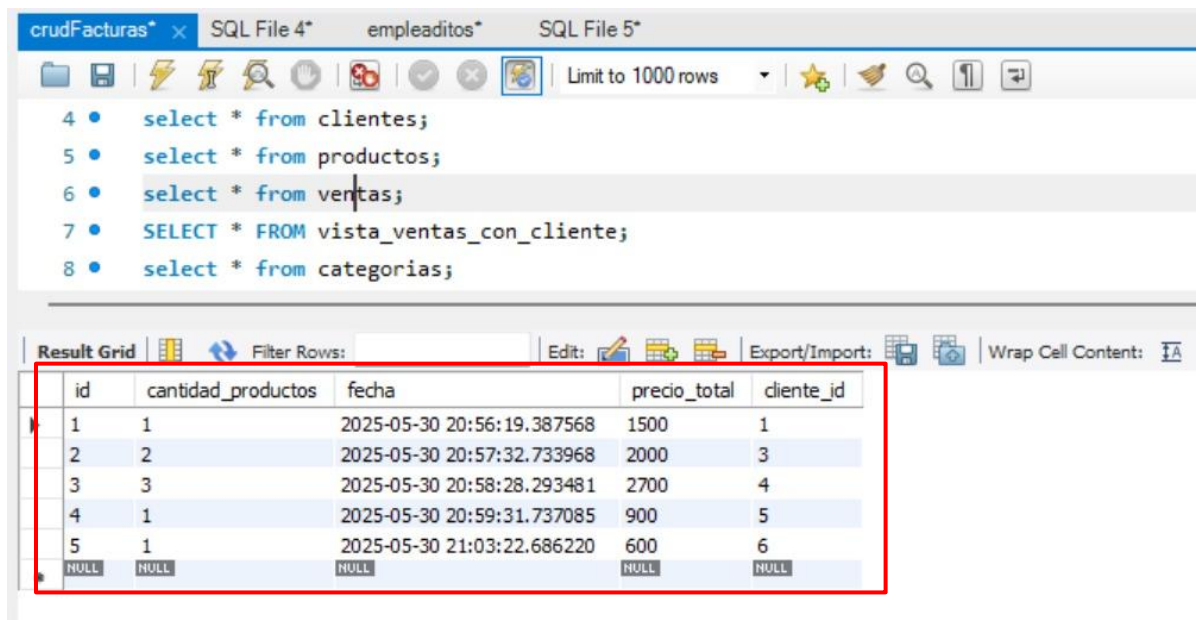


```
1 • create database factu;  
2 • use factu;  
3  
4 • select * from clientes;  
5 • select * from productos;
```

|   | id   | descripcion | precio | stock | titulo      | categoria_id |
|---|------|-------------|--------|-------|-------------|--------------|
| ▶ | 1    | PRO         | 1500   | 2     | IPhone 15   | 1            |
|   | 2    | PLUS        | 1000   | 3     | Samsung S25 | NULL         |
|   | 3    | PRO         | 900    | 16    | Redmi YS    | NULL         |
|   | 4    | PLUS        | 600    | 4     | Huawei Y7   | 4            |
| * | NULL | NULL        | NULL   | NULL  | NULL        | NULL         |

### Consulta ventas.

Al consultar las ventas se reflejan todos los datos en la tabla (Id, cantidadProductos, fecha, precioTotal y IdCleinte).



The screenshot shows a SQL IDE window titled "SQL File 4\*" with a toolbar and a list of queries. The query list includes:

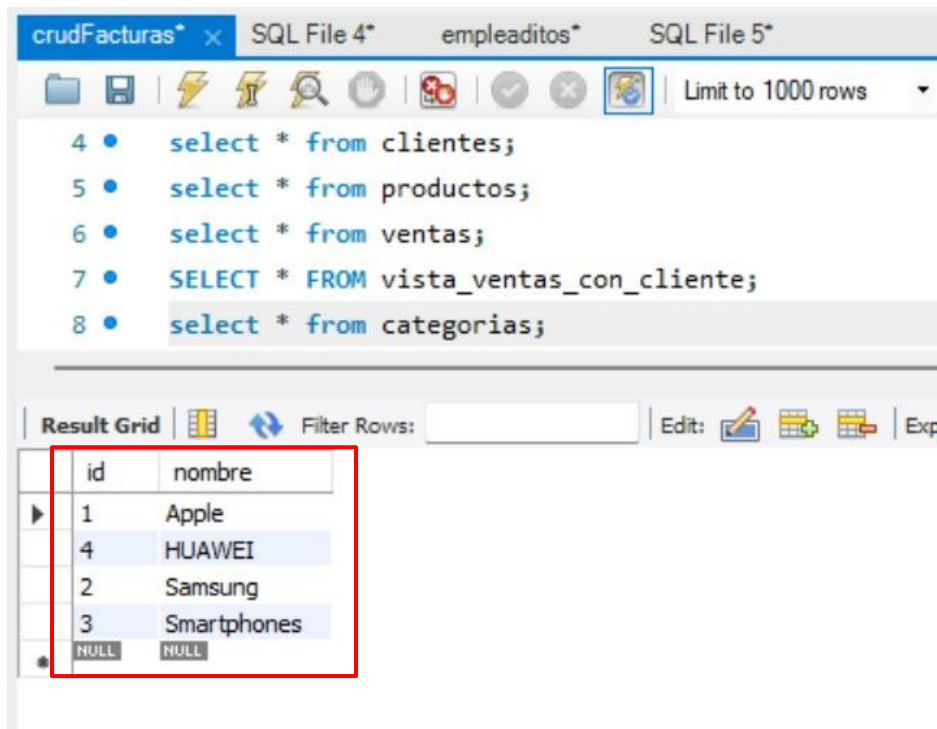
- 4 • select \* from clientes;
- 5 • select \* from productos;
- 6 • select \* from ventas;
- 7 • SELECT \* FROM vista\_ventas\_con\_cliente;
- 8 • select \* from categorias;

The "Result Grid" is displayed below the queries, showing the results of the selected query. The table has 6 columns: id, cantidad\_productos, fecha, precio\_total, and cliente\_id. The data is as follows:

| id   | cantidad_productos | fecha                      | precio_total | cliente_id |
|------|--------------------|----------------------------|--------------|------------|
| 1    | 1                  | 2025-05-30 20:56:19.387568 | 1500         | 1          |
| 2    | 2                  | 2025-05-30 20:57:32.733968 | 2000         | 3          |
| 3    | 3                  | 2025-05-30 20:58:28.293481 | 2700         | 4          |
| 4    | 1                  | 2025-05-30 20:59:31.737085 | 900          | 5          |
| 5    | 1                  | 2025-05-30 21:03:22.686220 | 600          | 6          |
| NULL | NULL               | NULL                       | NULL         | NULL       |

### Consulta categorías.

Dentro de la consulta a categorías se muestra en la tabla (IdCategoria y nombreCategoria)



The screenshot shows the same SQL IDE window as above, but with the query "select \* from categorias;" selected. The "Result Grid" displays the results of this query in a table with 2 columns: id and nombre. The data is as follows:

| id   | nombre      |
|------|-------------|
| 1    | Apple       |
| 4    | HUAWEI      |
| 2    | Samsung     |
| 3    | Smartphones |
| NULL | NULL        |

### Consulta producto\_cliente.

En esta consulta se reflejan los Id de los productos y sus clientes.

The screenshot shows a SQL query editor with the following queries:

```
3
4 • select * from clientes;
5 • select * from productos;
6 • select * from ventas;
7 • SELECT * FROM vista_ventas_con_cliente;
8 • select * from categorias;
9 • select * from producto_cliente;
10 • select * from producto_venta;
```

The result grid for the 'producto\_cliente' query is displayed below the editor:

| producto_id | cliente_id |
|-------------|------------|
| 1           | 1          |
| 2           | 3          |
| 3           | 4          |
| 3           | 5          |
| 4           | 6          |

### Consulta producto\_venta

Se muestra los productos que están asociados con ventas.

The screenshot shows a SQL query editor with the following queries:

```
4 • select * from clientes;
5 • select * from productos;
6 • select * from ventas;
7 • SELECT * FROM vista_ventas_con_cliente;
8 • select * from categorias;
9 • select * from producto_cliente;
10 • select * from producto_venta;
11 • select * from detalle_venta;
```

The result grid for the 'producto\_venta' query is displayed below the editor:

| id   | descripcion | precio | stock | titulo      | categoria_id |
|------|-------------|--------|-------|-------------|--------------|
| 1    | PRO         | 1500   | 2     | iPhone 15   | 1            |
| 2    | PLUS        | 1000   | 3     | Samsung S25 | NULL         |
| 3    | PRO         | 900    | 17    | Redmi YS    | NULL         |
| 4    | PRO         | 900    | 16    | Redmi YS    | NULL         |
| 5    | PLUS        | 600    | 4     | Huawei Y7   | 4            |
| NULL | NULL        | NULL   | NULL  | NULL        | NULL         |

### Consulta detalle\_venta.

Dentro de esta consulta se muestran los datos de las ventas realizadas como (Id, cantidad, precio, IdProducto, IdVenta).

The screenshot displays a SQL IDE interface with a query editor and a results pane. The query editor shows a series of SQL statements, with the last one being a SELECT query on the 'detalle\_venta' table. The results pane shows the output of this query as a table with 5 columns: id, cantidad, precio\_unitario, producto\_venta\_id, and venta\_id. The table contains 5 rows of data, with the last row showing NULL values. Below the results pane, there is an 'Output' section showing a log of SQL actions performed, including the creation of a view and several SELECT queries with LIMIT clauses.

```
5 • select * from productos;
6 • select * from ventas;
7 • SELECT * FROM vista_ventas_con_cliente;
8 • select * from categorias;
9 • select * from producto_cliente;
10 • select * from producto_venta;
11 • select * from detalle_venta;
12 • SELECT * FROM productos WHERE id IN (1, 4);
```

| id   | cantidad | precio_unitario | producto_venta_id | venta_id |
|------|----------|-----------------|-------------------|----------|
| 1    | 1        | 1500            | 1                 | 1        |
| 2    | 2        | 1000            | 2                 | 2        |
| 3    | 3        | 900             | 3                 | 3        |
| 4    | 1        | 900             | 4                 | 4        |
| 5    | 1        | 600             | 5                 | 5        |
| NULL | NULL     | NULL            | NULL              | NULL     |

detalle\_venta 79 x

Output

Action Output

| #  | Time     | Action                                                                              |
|----|----------|-------------------------------------------------------------------------------------|
| 8  | 21:07:34 | CREATE OR REPLACE VIEW vista_ventas_con_cliente AS SELECT v.id, v.cantidad_producto |
| 9  | 21:07:37 | SELECT * FROM vista_ventas_con_cliente LIMIT 0, 1000                                |
| 10 | 21:08:31 | select * from producto_cliente LIMIT 0, 1000                                        |
| 11 | 21:09:00 | select * from producto_venta LIMIT 0, 1000                                          |
| 12 | 21:10:58 | select * from detalle_venta LIMIT 0, 1000                                           |

## Conclusión

El presente proyecto ha representado un ejercicio integral en el diseño, desarrollo e implementación de una interfaz web funcional para la gestión dinámica de información a través de una API REST. Aplicando principios fundamentales tanto de frontend como de backend, se logró construir una solución coherente y modular que cumple con los estándares actuales del desarrollo web.

Uno de los pilares fundamentales de este sistema es su estructura modular, la cual consiste en dividir el código en componentes independientes que cumplen funciones específicas. Esta arquitectura aporta beneficios cruciales, como:

- **Mantenibilidad:** permite identificar y corregir errores de manera rápida.
- **Reutilización:** los módulos pueden ser aplicados en distintos contextos sin modificaciones.
- **Escalabilidad:** es posible incorporar nuevas funcionalidades sin afectar el resto del sistema.
- **Colaboración:** facilita el trabajo en equipo.

Por otro lado, el HTML cumple su rol como lenguaje base de la web, actuando como el esqueleto de la aplicación. En este proyecto, se utilizó para estructurar el contenido mediante formularios, secciones y tarjetas, apoyándose en etiquetas semánticas como `<header>`, `<main>`, `<section>`, `<form>` y `<input>`. Además, se enlazaron hojas de estilo CSS y scripts de JavaScript que aportan interactividad y dinamismo a la experiencia del usuario.

La conexión entre el frontend y la API REST permitió implementar operaciones CRUD (crear, leer, actualizar y eliminar), logrando así una interacción fluida y en tiempo real con la base de datos. Cada formulario está enlazado con funciones en JavaScript que envían peticiones HTTP (mediante métodos como `POST`, `GET`, `PUT` y `DELETE`) a los endpoints correspondientes, lo cual garantiza una comunicación efectiva sin necesidad de recargar la página.

Gracias a esta implementación, el sistema es capaz de gestionar ventas, asignar categorías, actualizar registros específicos y presentar resúmenes de forma eficiente y visualmente amigable.

En resumen, este proyecto no solo consolidó conocimientos técnicos, sino que también fomentó el desarrollo de habilidades como el análisis, la organización modular, la lógica de programación y la documentación profesional. Ha sido una experiencia educativa enriquecedora, con aplicación directa en entornos reales y escalables.